				sitemaps/lastmod Output: Live URL + updated sitemaps	
9. Translate	Appinventiv BE	Azure Translator/Chat GPT	Post-publish EN	Create DE/FR/IT/PL/ES + hreflang	Localized pages
10. Metrics loop	Appinventiv BE	GA4 + GSC + GPT	Nightly	Flag low CTR/Bounce; suggest targeted re-gen	optimization_queue
11. Near-pages	Appinventiv AI	GSC (+Semrush) + embeddings + OpenAI	Weekly	Create near-page drafts for real demand (no cannibalization)	New drafts
12. Internal similarity	SEO QA	Screaming Frog N-gram	Monthly	Check <15% overlap; flag templates/clusters	Similarity report
13. LLM feeds	Appinventiv	ai-sitemap, ai-summary, feeds	Nightly	Refresh AI endpoints for ChatGPT/Bing/Perplexity	Fresh feeds

2. Roles in short:

- appinventive (Rosotravel): reliable raw ingest only (draft_raw).
- Appinventiv BE: diff, routing, gen pipeline, validation, publishing, translations, metrics loop, near-pages, AI feeds.
- Appinventiv FE: templates, schema injection, sitemaps/hreflang, performance.
- Rosotravel SEO/Marketing: Human Optimization panel, prompt feedback, brand tone, Copyscape resolution, final acceptance of systemic changes.

3. System Contract: Ownership, Safety, and Deterministic Outputs

(tu jest „co wolno systemowi”, a co jest zakazane; ownership; compliance; zero UI)

This section defines mandatory system-level fields and rules required for correct operation of:

RAW ingestion, Diff Engine, AI processing, SEO governance, indexation safety, and MCP/LLM exposure:

➢ This is not a UI specification.
➢ It is a binding contract between:
➢ Data ingestion (RAW snapshots),
➢ AI generation,
➢ SEO ownership (keyword/FAQ/schema maps),
➢ Publication logic (Published Record),
➢ Editorial actions (manual edits + locks),
➢ Safety enforcement (indexing + LLM exposure).

A.09.1 Field Ownership Model (Hard Rule)

Every field MUST have exactly one ownership type:

Owner Type	Meaning	Can AI write?	Can human lock?	Can supplier override?
RAW-owned	Ingested snapshot fields and seeds	No	No (RAW immutable)	No (new snapshot creates new RAW version)
AI-owned	Generated/paraphrased content blocks	Yes	Yes	No
Human-owned	Manually edited + locked blocks	No	Already locked	No
System-owned	Derived/governance/control fields	No	No	No

Ownership determines:

- regeneration eligibility,
- override precedence,
- indexing eligibility,

- MCP/LLM exposure eligibility.

A.09.2 Mandatory System-Owned Fields (Must exist for every entity)

A) Identity & provenance

- entity_id (immutable internal ID)
- entity_type
- source_type (INTERNAL | EXTERNAL)
- stable_code (deterministic export key; required for white-label)

B) Version domains (see A.09.4)

- raw_version (integer)
- raw_ingested_at (timestamp)

Conditional (PRODUCT only):

- offer_version (integer)
- realtime_offer_version (integer)

C) Published tracking pointers (no “version explosion”)

Published is tracked as pointers to domain versions, not new versions:

- published_for_raw_version (int | null)
- published_for_offer_version (int | null, PRODUCT only)
- published_for_realtime_offer_version (int | null, PRODUCT only)
- published_at (timestamp)
- published_by (user/system)

D) Canonical & index governance (system-owned)

- index_state (draft / index / noindex)
- canonical_target (url/entity ref)
- sitemap_included (bool, computed)
- exposure_state (public_html / public_api / ai_feed / mcp) (computed)

E) Flags & signals (system-owned, persistent until resolved)

- blocking_flags[] (e.g., OFFER_MISSING, RELATIONSHIP_MISSING)
- warning_flags[] (e.g., MEDIA_REVIEW_REQUIRED)
- change_signals[] (see A.09.10)

Hard rule:
Acknowledging notifications does not clear flags. Only resolution clears flags.

A.09.3 RAW Seeds vs Published Candidate Fields (No mixing)

A) RAW seed fields (RAW-owned, immutable per raw_version)

These are stored as seeds and must never be published verbatim if restricted by licensing:

- PRODUCT: title_seed, short_description_seed, long_description_seed, highlights_seed[], itinerary_overview_seed
- ATTRACTION: name_canonical, introduction_seed, overview_sections_seed[]
- CITY: name_canonical, short_description_seed (optional)

RAW payloads and supplier-protected blocks remain internal-only and must never be exposed publicly.

B) Published candidate fields (AI-owned or Human-owned)

These are the editable/publishable fields that can be generated by AI or edited and locked by humans:

- title
- snippet / short_description

- long_description
- highlights[]
- itinerary_summary (non-stepwise)
- faq[]
- meta_title, meta_description

Rule:
If a human edits any block, that block becomes Human-owned (locked) for publishing precedence.

A.09.4 Unified Versioning Domains (Binding)

The system MUST separate freshness into domains to prevent raw_version explosion:

1) raw_version domain (slow snapshot)

Represents changes in:

- RAW seed content (where applicable),
- factual snapshot (CITY/ATTRACTION place truth),
- media snapshot,
- relationship snapshot (hierarchy + edges).

raw_version increments ONLY when any raw-domain hash changes (see A.09.5).

2) offer_version domain (PRODUCT only, medium cadence)

Represents changes in:

- options structure,
- inclusions/exclusions,
- cancellation policy structure,
- meeting point structure,

- other offer components that affect booking contract.

offer_version increments ONLY when offer_hash changes.

3) realtime_offer_version domain (PRODUCT only, high cadence)

Represents changes in:

- availability refs/calendars,
- price_from/currency (as realtime signals),
- start_times (if present).

Hard rule:
realtime_offer_version MUST NOT force raw_version increments.

A.09.5 Hashes (Change Detection Contract)

Hashes are computed on normalized blocks and are the only allowed trigger inputs for Diff Engine severity.

A) Hash set (must exist)

Entity Type	Required hashes
CITY	content_hash, factual_hash, media_hash, relationship_hash
ATTRACTION	content_hash, factual_hash, media_hash, relationship_hash
PRODUCT	content_hash, media_hash, relationship_hash, offer_hash, realtime_offer_hash

Notes:

- content_hash is computed over RAW seed content fields only (not AI/Human blocks).
- relationship_hash is computed over parent_relations + relationship_edges snapshot.

- offer_hash excludes realtime availability/pricing signals.
- realtime_offer_hash includes only availability/pricing refs and realtime signals.

A.09.6 Manual Locks (Block-level, not global)

Locks MUST exist per publishable block (not “one lock for everything”).
Minimum lock scopes:

- lock.title
- lock.snippet
- lock.long_description
- lock.highlights
- lock.itinerary_summary
- lock.faq
- lock.meta
- lock.media_override
- lock.factual_override (CITY/ATTRACTION only)
- lock.offer_override (PRODUCT only)

Each lock MUST store:

- locked_by
- locked_at
- lock_reason
- requires_approval (configurable by role)

Rules:

- Locks never expire automatically.

- Unlock requires explicit action + audit.
- Locked blocks are skipped by regeneration automatically.

A.09.7 AI State & Eligibility (Non-versioned)

AI regeneration never creates a new version number.
AI always references the current domain versions.

Required system fields:

- ai_allowed_content (bool)
- ai_allowed_meta (bool)
- ai_allowed_faq (bool)
- ai_last_generated_at
- ai_based_on_raw_version (int)
- ai_prompt_template_id
- ai_regeneration_log[] (audit only)

Eligibility rules:
AI may operate only if:

- block is not locked,
- required dependencies exist (no missing contract fields),
- either Diff Engine marked MAJOR for the relevant domain or user explicitly triggered regeneration.

A.09.8 SEO Governance Fields (System-owned, derived from maps)

SEO ownership is not edited here and is not written by AI.
These fields are derived from governance maps and are read-only in CMS:

- primary_keyword

- secondary_keywords[]
- keyword_owner_ref (pointer to keyword_map row)
- faq_owner_refs[] (pointers to faq_map rows)
- schema_owner_refs[] (pointers to schema_map rows)

Rule:
Pipeline regeneration never changes keyword ownership.

A.09.9 Published Record Assembly (Deterministic Winner Rule)

The system MUST maintain a Published Record per entity.
This is the only object served to:
frontend HTML,
public APIs,
AI feeds / AI sitemap,
MCP tools.

Winner precedence (per block):

1. Human-owned (locked)
2. AI-owned (if exists for current domain versions)
3. RAW seed-derived (only if explicitly allowed by licensing/indexing rules)
4. Empty + safety flags

Safety enforcement during assembly:

- Restricted supplier content must never leak into Published Record.
- If a valid Published Record cannot be assembled for required contract fields:
  page remains online but becomes noindex, follow and excluded from sitemaps.

A.09.10 Change Signals & Review Flags (System Contract)

Change signals are system fields consumed by UI layers (not UI definitions).

A) Trigger conditions

A change signal MUST be created when:

- a new raw_version / offer_version / realtime_offer_version is detected for a published entity,
- Diff Engine severity is MEDIUM or MAJOR,
- a human-locked block was bypassed due to supplier update,
- AI regeneration was executed on a published entity,
- indexation-relevant governance fields changed (canonical/index_state).

B) Payload (must store)

- entity_id, entity_type
- raw_version / offer_version / realtime_offer_version (as applicable)
- change_severity (MINOR / MEDIUM / MAJOR)
- changed_blocks[] (content / factual / media / relationships / offer / realtime)
- requires_human_review (bool)
- created_at
- resolved_at (nullable)
- resolved_by (nullable)
- resolution_action (reviewed / ignored / escalated)

C) Resolution rules

- Every change signal must be explicitly resolved by a human.
- Resolution never modifies content; it confirms review only.
- Unresolved signals remain active indefinitely.

A.09.11 MCP / LLM Safety Enforcement (Binding)

Only Published Record fields may be exposed to:

- AI summaries,
- MCP servers,
- external LLM datasets.

The following must never be exposed:

- RAW payloads,
- supplier unique content blobs (e.g., viatorUniqueContent as-is),
- raw itineraries/steps,
- raw reviews,
- restricted UGC/supplier media.

Violation response:

- force noindex + exclude from feeds + create blocking flag.

01. Global JSON-LD builders Tokens, Schema, Relations system

a. Definition Token

Each web page is generated using dynamic tokens placeholders that are replaced by real data (from API, AI generation, or manual marketing input). Schemas are generated automatically from these tokens using template-based JSON-LD builders.

Element	Description
Token	A dynamic variable used in meta titles, headings, schema fields, and prompts.
Schema Type	JSON-LD structured data object (e.g., `TouristTrip`, `FAQPage`, etc.).
Data Source	Origin of the data: Rosotravel API, AI content, manual input.
Responsibility	Who maintains or updates the value — Marketing or IT.

b. Global Token Library

IT: AI & programmatic SEO - schemes

c. Schema Mapping by Page Typ

Rules:

- **Canonical-only**: JSON-LD must reference `{{Canonical_URL}}` as the canonical identity; never treat parameter URLs as identity.
- **UI ↔ schema parity**: If a fact appears in JSON-LD, it must be present in the UX (or in a clearly visible "About/Details" section), and must not contradict UX.
- **Index eligibility enforcement**: If `{{Robots_Directive}} = noindex,follow`, the page must not appear in AI sitemap / inventory feeds as a first-class entity.
- **Fail-closed**: If required tokens for a page type are missing → block publish (or force `noindex,follow` depending on your publishing rules).
- **No supplier leakage**: No supplier IDs, supplier URLs/endpoints, raw partner payloads, verbatim supplier descriptions/reviews/photos unless explicitly licensed.
- `Main_Entity_ID` must be emitted as valid `@id` (URN or canonical#entity)

IT: AI & programmatic SEO - schemes

d. Hierarchy & Linking (Relations)

Hard rule (JSON-LD identity):

- Every primary entity node MUST have an `@id` that is a **valid absolute IRI** and is **stable forever**.
  Recommended forms (choose one and standardize site-wide):
- **Schema property naming rule:** In JSON-LD, always emit `offers` (plural) on Product/TouristTrip nodes. In feeds you may use `offer` as an internal object name, but the on-page schema must use `offers`.

IT: AI & programmatic SEO - schemes

e. Relations → Implementation by Page Type (Matrix)

IT: AI & programmatic SEO - schemes

Core rule (applies to every page type)

UI↔Schema parity (hard rule): If a relation is emitted in JSON-LD, the matching concept must be visible in the UI **or** be a strictly technical invariant (canonical URL, language, identity).
No hidden lists: If a page emits an `ItemList`, the list must be rendered on the page and match ordering (when indexable).

No fabricated facts: Geo coords, ratings, counts, opening hours, and “sameAs” must come from SSOT sources (Places/Wikidata/Reviews API/CMS) and must not be guessed.

Node layout (use this on every page)

Each page publishes **at minimum**:

1. a **WebPage node** (or `CollectionPage` / `Article` where relevant)
2. one **main entity node** referenced by WebPage `mainEntity`

WebPage node MUST include (all page types):

- `@id` (stable web page id / canonical URL as IRI)
- `url` (canonical absolute URL)
- `inLanguage` (BCP-47 like `en-US`)
- `breadcrumb` (BreadcrumbList) **for indexable pages**
- `mainEntity` (points to the entity `@id`)

Main entity node MUST include (all page types):

- `@id` (stable entity IRI, e.g. `urn:rt:product:...`)
- `identifier` (string id)
- `name`
- `url` (canonical absolute URL)

Hard gate (all pages): If `@id`, `identifier`, `url`, `inLanguage`, or `mainEntity` missing → **BLOCK publish** (or force `noindex` + Inbox critical).

Implementation matrix by Page Type

1) Country Destination Page (Template: Country)

WebPage node

- `mainEntity` → Country Place entity `@id`
- `breadcrumb` → BreadcrumbList (Home → Country)

Country entity node (Place-like: use `TouristDestination` if you already do)

- `containedInPlace` (optional if you model Continent; otherwise omit)
- `containsPlace` (list Regions or top Cities) **ONLY if UI shows them**
- `sameAs` (recommended: Wikidata/Wikipedia/official)
- `hasPart` / `isPartOf` (for page graph):
    - Page `hasPart` = links to Region/City pages (top N)

ItemList

- If the page has “Top cities / Top experiences” modules → publish `ItemList` for each visible list.

Hard gates

- If page is indexable and a “Top list” module is visible → `ItemList` must exist and be non-empty, else **BLOCK**.
- BreadcrumbList must match UI crumbs, else **BLOCK**.

---

2) Region Destination Page (Template: Region)

Same structure as Country, but:

- Region entity `containedInPlace` → Country entity (strong GEO)
- Region page `isPartOf` → Country page (page graph)
- `containsPlace` → Cities (only if UI lists)

Hard gates

- If Region page shows Cities list → must publish `containsPlace` and/or `ItemList` matching UI, else **BLOCK**.

---

3) City Destination Page (Template: City)

WebPage node

- `mainEntity` → City entity
- `breadcrumb` (Home → Country → Region (if any) → City)

City entity node

- `containedInPlace` → Region (if exists) else Country
- `geo` (optional but recommended if you have city centroid)
- `containsPlace` → top POIs (only if UI lists)
- `nearby` (optional; can be POI-to-POI primarily)
- `sameAs` (recommended)

ItemList (strongly recommended)

- "Top tours" list → `ItemList` (Tour URLs)
- "Top attractions" list → `ItemList` (POI URLs)

Hard gates

- Any visible list module on an indexable City page must have matching `ItemList` (same items, same ordering), else **BLOCK**.
- If breadcrumbs visible → BreadcrumbList must match, else **BLOCK**.

---

4) City "Things to do" Page (Template: City Things-to-do)

This is a **listing page**. Treat as `CollectionPage` WebPage.

WebPage node

- `@type`: `CollectionPage`
- `mainEntity` → an `ItemList` node (preferred) OR a City entity + embedded ItemList

ItemList

- Must represent the primary product listing the user sees.
- Each item must include:
  - `position`
  - `url`
  - (optional) `name`
- If you do segment-based ordering (DEE), emit only the **default** ordering in schema unless you have a safe deterministic rule.

Relations

- Page `isPartOf` → City page
- City entity `hasOfferCatalog` (optional) pointing to the same `ItemList` **if** you keep it valid and consistent.

Hard gates

- If page is indexable: `ItemList` must not be empty → **BLOCK**.
- If filters create URL variants: those variants must be `noindex` and canonicalized (but that's governed by SEO rules; still: **schema must use canonical URL only**).

5) Attraction / POI Page (Template: Attraction)

WebPage node

- `mainEntity` → POI entity
- `breadcrumb` → (Home → Country → City → POI)

POI entity node (`TouristAttraction`)

- `containedInPlace` → City entity (**recommended**)
- `geo` → GeoCoordinates (**required if POI page exists**)
- `sameAs` → Wikidata/Wikipedia/official (**recommended**)
- `nearby` → nearby POIs (optional; only if UI renders “Nearby”)
- `tourBookingPage` → relevant booking page(s) **only if UI shows a booking CTA path** (e.g., “Best tours for this attraction”)

ItemList (optional but recommended)

- If POI page shows “Top tours for this attraction” → publish `ItemList`.

Hard gates

- Missing `geo` for POI → **BLOCK**.
- If “Nearby” UI module renders → schema must include `nearby` for the same entities (top N), else **BLOCK** (or disable module).
- If “Top tours” list renders and page is indexable → `ItemList` required, else **BLOCK**.

---

6) Tour / Product Page (Template: Product)

WebPage node

- `mainEntity` → Product entity (your `TouristTrip` node)
- `breadcrumb` → (Home → Country → City → Product)

Product entity node (your commercial entity)

- Identity: `@id`, `identifier`, `url`, `name` (required)
- Place grounding:
    - `containedInPlace` → City or POI (conditional; use if you genuinely anchor it)
    - optional `about` → POI entity (if your product is “about the Colosseum”)
- Commerce:
    - `offers` → Offer (**required**)
    - `provider` → Organization (**required**)
    - `potentialAction` → ReserveAction/BuyAction (**conditional**, only if CTA exists)
- Trust:
    - `aggregateRating` / `review` (**only if displayed** and compliant)
- Community:
    - `discussionUrl` / `commentCount` (**only if module exists**)
- Similarity:
    - `isSimilarTo` / `relatedLink` (**optional**, must come from your similarity engine)

Offer node MUST include

- `price`, `priceCurrency`, `availability`, `url` (CTA target), and (if you have it) `validFrom` / `priceValidUntil`.

Hard gates

- Offer invalid/missing → **BLOCK**.
- Offer must match **Booking Panel price/currency/availability** → mismatch → **BLOCK**.

- If CTA exists in UI → `offers.url` and/or `potentialAction.target` must equal the CTA target → mismatch → **BLOCK**.
- If you display rating/count → schema must match exactly → mismatch → **BLOCK**.
- If Q&A/comments module exists → discussionUrl must exist; if you emit discussionUrl but module absent → **BLOCK**.

---

7) Package / Bundle Page (Template: Package, purchasable)

This is both a listing and a purchasable unit.

WebPage node

- `@type`: `CollectionPage` (recommended)
- `mainEntity` → Package entity node

Package entity node

- Identity: `@id`, `identifier`, `url`, `name`
- Included items:
  - `hasPart` (optional) linking to included Product entities **or**
  - `ItemList` of included products (recommended if you show the list)
- Commerce:
  - `offers` (**required if package is purchasable**)
  - `provider` Organization (required)
  - `potentialAction` ReserveAction/BuyAction (conditional, only if CTA exists)
- Editorial (if you show curator/author UI):
  - If you present it as editorial content: add `author`/`publisher` **only if UI shows author** (curator block)

Hard gates

- Package purchasable but no Offer → **BLOCK**.
- Included tours list rendered → ItemList required and must match UI ordering, else **BLOCK**.
- CTA exists → offers.url or potentialAction.target must match CTA, else **BLOCK**.

---

8) Experience Category Page (Template: Category)

WebPage node

- `@type`: `CollectionPage`
- `mainEntity` → ItemList (recommended)

Relations

- `isPartOf` → City page or Hub page (depending on your IA)
- ItemList must represent visible listing.

Hard gates

- Indexable category listing must have non-empty ItemList, else **BLOCK**.

---

9) Blog / Inspiration Article (Template: Blog)

WebPage node

- `@type`: `WebPage` (optional) + Article node as mainEntity (preferred: `mainEntity` = Article)

Article node

- `author` Person (**required**)
- `publisher` Organization (**required**)
- `datePublished` (**required**)

- `dateModified` (**required**)
- `inLanguage` (**required**)
- `about` (optional: City/POI entity)
- `subjectOf` (optional: VideoObject)
- FAQPage only if FAQ block is rendered

Hard gates

- Missing author identity (name + profile URL), datePublished, dateModified, H1/title → **BLOCK**.
- If FAQPage emitted but FAQ UI not rendered → **BLOCK**.

---

10) Travel Guides (Country / City / Itinerary) (Template: Guide)

Same as Blog, plus:

- For itinerary-type guides: may use HowTo (see section B) **only when the Steps module renders**.

Hard gates

- Same as blog gates for author/dates.
- If Steps module renders and HowTo emitted → must pass HowTo gates (section B).

---

11) Expert Page (Template: Expert)

WebPage node

- `mainEntity` → Person node

Person node

- `@id`, `identifier`, `url`, `name` (required)

- `image` (recommended)
- `sameAs` (optional)
- `worksFor` / link to Organization (recommended)
- `knowsAbout` (optional: cities/POIs)

Hard gates

- Person core identity fields missing → **BLOCK**.

---

12) Global Experiences Hub (Template: Hub /experiences/)

WebPage node

- `@type`: `CollectionPage`
- `mainEntity` → ItemList (categories) and/or multiple ItemLists

Relations

- Page `hasPart` → category pages
- ItemList must match visible UI category tiles ordering.

Hard gates

- If Hub is indexable and shows categories list → ItemList required and non-empty, else **BLOCK**.

02. Development Deliverables Checklist

Deliverable	Description	Owner
Schema Builder Engine	Generates all JSON-LD types (TouristDestination, TouristTrip, Attraction, Person, Offer, ItemList) from tokens	IT

AI Connector	Connects API → AI → CMS (OpenAI + internal models)	IT
Keyword Mapping Database	Locks keyword ownership for Country/Region/City/Attraction/Product/Near-page	IT
AI Feed Generator	Generates `/ai-summary.json`, `/ai-sitemap.xml`, `/explore/ai-dataset/`	IT
Data Diff Engine	Compares API version N vs N-1 and triggers reprocessing only on changed fields	IT
Content Safety & Uniqueness Module	Integrates Copyscape API, N-gram similarity, internal duplication checks	IT
Slug Normalization Engine	Generates clean slugs (no taxonomy, no category IDs) and prevents duplicates	IT
Multilingual URL + Hreflang Generator	Generates hreflang sets + alternate URLs + canonical rules	IT
Filter Routing Logic	Handles filter parameters → non-indexable URLs → canonical rewrite	IT
Experts & Creators Graph Builder	Creates schemas for Person entities, authored content, links to cities/countries	IT
Image Pipeline (Upload + AI Optimization)	Integrates Cloudinary + auto-alt-text + WebP generation	IT
Variant & Offer Schema Engine	Handles product variations (API: Viator + Rosotravel native offers)	IT
Content Validation Scripts	Validates schema syntax, required fields, meta length, broken URLs, CTR alerts	IT
QA Dashboard	Human approval queue: sampling, criteria scoring, approve/publish workflow	IT
Sitemap Builder	Builds `/sitemap.xml` segmented by entity type (destination, attraction, product, guides, experts…)	IT

Graph Sync Monitor	Ensures hasPart / isPartOf / nearby relations remain correct after content updates	IT

03. For marketing team to prepare before implementation

Deliverable	Description	Owner
Prompt Library	Complete library of AI prompts for each page type (Product, City, Attraction, Package, Blog, Expert, Guide). Must include tone rules, tokens, length rules, and fallback prompts.	Marketing
SEO Keyword Hierarchy File	Final keyword ownership map: transactional vs mid-funnel vs informational. Includes keyword clustering per entity (City/Attraction/Product) and rules to prevent cannibalization.	Marketing
H1–H3 Template Book	Structural outline templates for each page type using tokens: {{City_Name}}, {{Attraction_Name}}, {{Tour_Type}}, {{nearby}}, etc. Defines semantic headers, ordering, and internal linking placement.	Marketing
FAQ Repository (by entity type)	Centralized file of approved FAQs for: Country, Region, City, Attraction, Tour, Bundle, Expert. Includes fallback FAQs for AI generation.	Marketing
LLM Entity Map	Full mapping of: hasPart, isPartOf, nearby, sameAs relations for the whole site (Country → Region → City → Attraction → Tour → Package). Required for AI Feed + Schema Builder.	Marketing + IT
Image Style Guide + Alt Text Rules	Visual guidelines for all auto-generated images: composition, framing, lighting, color palette. Includes alt-text structure rules and naming conventions.	Marketing
Slug Style Guide	Rules for generating URL slugs: no stopwords, no category IDs, no redundancy, lowercase, hyphens only, ≤65 characters. Includes conflict-resolution logic.	Marketing
Content Governance Matrix	Defines which content is auto-published vs requires human QA. Establishes sampling %, quality thresholds, and criteria triggering regeneration.	Marketing
Canonical + Hreflang Policy	Full ruleset for multilingual URLs, canonical tags, alternate links, parameters allowed/not allowed, non-indexable filters, and domain-level locale matching.	Marketing

A.10.3 Content Ownership Model (Critical for SEO & AI)

Every piece of indexable content MUST have **exactly one owner entity**.

Ownership applies to:

- keywords
- FAQs
- schema blocks
- SEO text sections

Ownership is enforced via dedicated mapping tables.

4.1 Workflow Map (Top Level): Preparation → Execution → Post-Publish Loop

1. Top Level overview: of SEO/LLM programmatic concept

a. Preparation

01. Upload predefined destinations and Ingestions Raw objects for seed

The system ingests a predefined list of destinations (countries and regions only; never cities).

Each destination record MUST include:

- internal_id (system)
- external_id (e.g., Viator destinationId)
- destination_type (COUNTRY or REGION)
- parent_id (required for regions)
- canonical_name_en (English canonical name)

Hard rule: during upload, the system MUST create a versioned RAW destination snapshot (RAW object) for each record.This RAW destination snapshot is used later only for seed-based keyword retrieval and destination page generation (Run SEO Auto).

Note: this list is authoritative and is not created dynamically by products. It defines the top-level tree:
COUNTRY → REGION

02. Destination normalization and hierarchy mapping

After upload, the system runs an automatic normalization pass that:

- standardizes destination names to a single canonical English form,
- resolves duplicates (e.g., “UK” vs “United Kingdom”),
- builds the parent–child hierarchy between countries and regions.
  Hard rule: no SEO content is generated at this stage.

03. Seed creation for destination-level keyword retrieval (Country/Region only; per entity)

For each destination entity (country or region), the system creates a destination seed used ONLY for broad destination keyword retrieval. Seed creation rules (high-level, no templates repeated here):

- seed inputs come only from destination metadata: canonical_name_en, destination_type, and (for regions) parent country name
- seed MUST NOT contain: city names, POI/attraction names, product titles, or transactional modifiers (“tickets”, “price”, “book”)
- seed is created separately per destination entity (one country = one seed set; one region = one seed set)

Important: this is not a “batch seed” and does not interact with Pipeline batches.

04. Connection to product inventory

While products are added (the product ingestion flow is described in the Execution section), the system links destinations to inventory using `external_id / destinationId` mappings (and internal product mappings).

For each destination, the system computes and continuously maintains:

- `product_count`
- `destination_state` = `ACTIVE` if `product_count ≥ 1`,
- otherwise `INACTIVE`

05. Basic publication (regardless from Run SEO Auto or not)

A destination is published and indexed (follow) automatically when at least one product linked to that destination is published. Destination is published when even 1 product is linked. Inactive destinations remain stored in the system but are never published, indexed, or included in sitemaps. Basic publication create the landing page of the destination which includes only the H1 and products listing.

A destination landing page is published automatically when at least one linked product is published:

- `ACTIVE destinations`: published and **indexable follow** (per your indexation rules)
  `INACTIVE destinations`: stored but never published, never indexed, never included in sitemaps

Basic publication outputs only:

- H1 (canonical destination name)
- product listing module (inventory-driven)

06. AI processing: destination-level keyword retrieval and opt. with “Run SEO Auto"

For active destinations, a marketer can click “Run SEO Auto” in the backend UI.
This triggers destination-level keyword retrieval from SEMrush using the destination seed and retrieves only

Governance rules:

- blocked until product_count ≥ 1
- regeneration is allowed only for Marketing Admin (role-gated)
- regeneration creates a new AI version but does not change indexation state by itself
- images are not sourced from suppliers; destination visuals (if used) come from your approved generation/asset pipeline

07. Automatic keyword selection (no clustering, no embeddings at destination level)

The system automatically selects:

- primary_keyword (exactly 1)
- secondary_keywords (up to 2)

Selection is rule-based using volume and relevance (destination-level rules). Hard rule: no embeddings, no semantic clustering, no manual keyword picking at the destination level.

08. Destination FAQ question retrieval (destination-wide only)

The system retrieves a small set of destination-level questions from SEMrush "Questions" and, if available, Google Search Console.

Rules:

- questions must be destination-wide (visa, safety, currency, transport, seasonality, culture)
- dedupe uses simple text matching (no embeddings at this level)
- any city/POI/transactional leakage is removed

09. AI content preparation

Using the selected keywords and approved FAQ question set, AI prepares destination content for:

➢ Snippet
➢ meta title + meta description
➢ FAQ answers (template-driven, fact-checked against your factual layer if applicable for destinations)
➢ stable highlights blocks (non-dynamic)
➢ according to fields country page

All outputs are stored on the destination entity and never propagated to child entities.

10. Destination update - API calls

Destination changes are rare but must be supported. The system MUST allow destination refresh to handle examples like:

- currency / defaultCurrencyCode changes
- time zone changes
- administrative hierarchy change (region affiliation changes)
- destination renames/aliases normalization adjustments

Rules:

- refresh updates the RAW destination snapshot (new RAW version only if values changed)
- refresh MUST NOT trigger any Pipeline/batch logic (destinations remain outside batches)
- any downstream SEO regeneration happens only through explicit Run SEO Auto (role-gated), not automatically

b. Execution

11. Entry points (how data enters Execution)

Execution starts only when at least one of the following happens:

a. New INTERNAL product is added (manual / CMS)

- Publication team adds a product
- assigns: country + city (must exist) and optionally selects/creates an attraction via Google Maps
- product is saved as INTERNAL source

b. New EXTERNAL product is ingested (Viator / OTA)

- ingestion job pulls product data from external API
- destination chain comes from external destination IDs
- products are saved as EXTERNAL source

Hard rule: regardless of entry point, every entity (Product/City/Attraction/Near-page) MUST enter the system as a versioned RAW snapshot first (no direct publish).

12. RAW ingestion & entity spawning (what gets created automatically)

When a new product is ingested, the system MUST ensure the minimum entity graph exists:

12.1 Product RAW snapshot

- create RAW_PRODUCT_Version (with correct source_type, external_id if any, and version fields)

12.2 City RAW snapshot (spawn if missing)

- if the city does not exist yet in canonical store → create RAW_CITY_Version
- city identity is attached using the best available keys (viator_destination_id / google_place_id / wikidata_id)

12.3 Attraction RAW snapshot (spawn if missing)

Attraction can be created in two ways:

- INTERNAL: user selects/creates a POI using Google Places (Google place_id becomes the primary identity key)
- EXTERNAL: attraction comes from external attraction object OR inferred linkage to POI if resolvable

Hard rule: RAW ingestion MUST also write relationship fields immediately:

- Product → City (mandatory)
- Product → Attraction (optional but recommended)

- City → Region → Country (mandatory chain where available)
  These relationships are stored as edges + relationship_hash.

13. Versioning domains + refresh cadence (no version explosion)

Execution operates on separate freshness domains:

- raw_version (slow snapshot: content seed + factual + media + relationships)
- offer_version (offer snapshot: options, inclusions/exclusions, cancellation/meeting point structure)
- realtime_offer_version (high cadence: pricing/availability refs)

Hard rule: changes in realtime_offer_version MUST NOT increment raw_version.

14. Batch creation (RAW batch is created immediately, then activated manually)

14.1 Initial batch creation (no cycle, no staging)

The first incoming product for a destination scope MUST create a RAW batch automatically.

- batch is a persistent container for RAW records
- batch remains “RAW / inactive” until a human runs batch processing

14.2 Adding records to an existing RAW batch

As more products arrive for that scope:

- they are appended to the same RAW batch
- no automatic batch AI processing is triggered

14.3 Manual activation

A user triggers “Run AI Batch Processing”.
Once executed:

- batch becomes “ACTIVE”
- its SEO ownership outcomes become the authoritative baseline for that destination scope

Governance rule: batch-level re-run (full reset) is restricted (Marketing Manager/Admin only) and allowed only for strategy-level mistakes (e.g., wrong SEMrush settings). Regular new products are processed as single records.

15. Single-record processing (late-arriving products after batch activation)

When a batch is ACTIVE and a new product arrives:

- the product still enters RAW and is appended to batch history
- AI processing is executed per single record (not batch-wide)

Hard rule: single-record processing MUST respect existing ownership locks:

- it cannot reclaim keywords already locked by earlier batch run
- it cannot spawn near-pages that would collide with already-consumed keyword ownership for that destination

---

16. **Seed generation (Execution-level: cities, major attractions, optionally gated products)**

    Seeds exist only to control keyword retrieval scope and cost.

    16.1 City seeds
    Generated only if the city is active (published product_count ≥ 1).

    - minimal fixed set (city discovery intent)

    16.2 Major-attraction seeds
    Generated only if attraction qualifies as “major” (per your thresholds / override rules).

    - includes controlled informational + commercial patterns for routing (not ownership drift)

    16.3 Product seeds (exception only, gated)
    Disabled by default.
    Can be enabled only under strict gates (bookable + linked to major attraction + SEO Admin toggle).

    - used for transactional routing only, never for broad discovery

    Hard rule: seeds are English-only source; translations happen later.

17. **Keyword retrieval (SEMrush + GSC) with ownership gates**

    The system retrieves candidate keywords using SEMrush and later validates/expands with GSC.

    17.1 Retrieval scope

    - City/Things-to-do/Attraction/Near-page/Product each pulls only intents allowed by ownership rules described in different subject

    17.2 Mix-of-two (SEMrush) handling
    If SEMrush returns “mixed intent”:

    - keyword is eligible only if both intents are allowed for that page type
    - otherwise it is routed to the closest eligible owner type or rejected

    Hard rule: keyword ownership assignment is deterministic, logged, and locked.

18. Embeddings + clustering (only after keyword retrieval)

This is where semantic structure is built NOT where batches are created.

18.1 Embeddings

- compute embeddings for keywords + entity centroids
- used for similarity gates, dedupe, and routing confidence

18.2 Clustering

- cluster within destination scope to avoid cannibalization
- produce candidate sets per page type

18.3 Ownership decision

- apply rules (intent allowlist + similarity thresholds + keyword locking)
- write winners into keyword_map (single source of truth)

19. Factual enrichment (Google Places + Wikidata; cost-controlled)

Factual enrichment runs independently from AI writing and is versioned via factual_hash.

- Attractions & Cities: Google Places is Tier-1 for name/address/hours/coords/categories/website (per your rules)
- Wikidata: historical facts, relations, external IDs, aliases, sameAs

Cost rule: refresh cadence differs by field criticality (e.g., opening hours more frequent than name). Factual updates MUST NOT trigger AI rewriting unless a dependent indexable block is configured to regenerate.

20. Diff engine & pipeline actions (selective processing only)

On any new RAW snapshot or factual/media refresh:

- compute hashes (content_hash / factual_hash / media_hash / relationship_hash / offer_hash / realtime_offer_has
- compare RAW vN vs vN-1
- classify severity (MINOR / MEDIUM / MAJOR)
- enqueue only required downstream actions

Hard rule: only MAJOR changes can enqueue AI drafting modules. MINOR and MEDIUM bypass AI drafting.

21. AI drafting (content + schema) + validations

21.1 AI drafting

- AI generates only allowed blocks and never invents facts
- schema generation uses verified factual + operational winners

21.2 Validations

- duplication / similarity checks
- policy & SEO safety checks
- optional plagiarism checks (as a gate, not a generator)
- structured data validation

22. FAQ pipeline (questions → dedupe → AI answers → QA)

- candidates from SEMrush Questions + GSC queries
- dedupe via similarity + uniqueness rules (global uniqueness enforced)
- AI generates answers (and only answers when questions are locked/unique per your governance)
- human QA sampling approves

23. Human QA + publishing (deterministic assembly)

23.1 QA sampling

- review only what is flagged (OUTDATED / CONFLICT / MISSING / LOCKED)
- approve batch output or selected records

23.2 Published Record assembly

Published output is assembled using deterministic winner rules:

- human-locked > AI > RAW (only if allowed) > empty
  RAW payload is never exposed.

Publishing outputs:

- landing pages (all eligible types)
- sitemaps (standard + AI sitemap)
- AI feeds / API feed (only Published Record)

24. Post-publish operations (translation, monitoring, optimization)

- translate from EN winners to other languages (auto)

- manual language edits lock that language block
- monitor mismatches (raw_version vs published_for_raw_version)
- monitor keyword ownership conflicts and near-page caps
- ongoing performance loop (GSC-driven improvements

c. Post-Publish Optimization & Controlled Expansion

25. Always-on Monitoring (system starts immediately after first publish)

After any entity becomes Published, the system continuously monitors:

- RAW vs Published mismatch (published_for_raw_version < raw_version)
- AI backlog / AI failure states (ai_status = FAILED or REQUIRED but not generated)
- schema validity (structured data errors, required fields missing)
- factual completeness (missing critical factual fields required by indexing rules)
- media integrity (broken URLs, invalid assets, license flags)
- ownership conflicts (keyword_map conflicts, FAQ duplicates) as alerts only (no auto-fix here)

Hard rule: Monitoring creates signals + jobs, but does not silently change keyword ownership.

26. Optimization Inbox (CMS UI surface for all post-publish signals)

- The system writes every post-publish issue into a unified “Optimization Inbox” (one queue):
    - OUTDATED (Published is behind RAW)
    - MISSING (critical data missing)
    - CONFLICT (ownership / duplication risk)
    - LOCKED_BLOCKED_UPDATE (RAW changed but manual lock prevents update)
    - AI_FAILED / AI_BACKLOG
    - SCHEMA_INVALID
    - MEDIA_REVIEW_REQUIRED
    - API_OUTAGE / RATE_LIMIT (non-blocking banner + backlog)
- Each signal must store: entity_id, entity_type, raw_version, severity, affected_blocks, created_at, resolution_state.

Hard rule: Every signal must be explicitly resolved by a human (acknowledged/reviewed), even if no content changes were made.

27. Automatic Optimization Engine (backend, daily evaluation)

- Once per day, the system evaluates GSC + GA4 + technical health and decides whether to enqueue safe jobs:

  GSC triggers (SEO):

  - CTR drop on stable impressions → enqueue meta/snippet refresh candidate
  - position stagnation with growing impressions → enqueue snippet/meta test candidate
  - impressions collapse → enqueue technical audit signals (index/canonical/schema checks)
  - query overlap spikes → create cannibalization warning signal (governance-only resolution)

  GA4 triggers (UX):

  - bounce up / dwell down / scroll down → enqueue “content refresh candidate” (limited blocks only)
  - conversion down → enqueue “offer integrity check” (operational fields + schema consistency)

Hard rule: Automatic engines can enqueue only allowed job types (see next point) and must respect locks.

28. Automatic Actions (job queue execution, not a content editor)

- Jobs created by the engine run through a queue with backpressure and caps.
- Allowed automatic job types (examples):
  - regenerate specific AI blocks (meta/snippet/etc.) using approved prompt templates
  - refresh schema blocks from verified factual + operational winners
  - refresh factual cache (Google/Wikidata) per cost-controlled cadence
  - media re-optimization (Cloudinary processing / fallback substitution)
- Disallowed automatic actions:
  - changing keyword ownership
  - creating new page types that expand scope (except Near-Pages via gated module below)
  - overriding manual locks
  - auto-publishing high-risk changes without queue + cap enforcement

29. Publishing Queue + Daily Limits (prevents “Google downgrade” spikes)

- Any action that results in a new Published Record enters a Publishing Queue.
- The queue enforces configurable caps, e.g.:
  - max_new_indexable_pages_per_day
  - max_updated_indexable_pages_per_day
  - max_per_destination_per_day (prevents city-wide spikes)

    - max_ai_regeneration_jobs_per_day (token/cost control)

- If caps are exceeded:

    - the system auto-splits into multiple jobs/days
    - low-priority improvements are deferred first (CTR tuning), critical safety fixes first (schema/facts)

Hard rule: No “mass publish now” bypass unless Admin override is used and logged.

30. Weekly Classification Engine (LangChain / deterministic outputs)

- Once per week, the system reclassifies entities into operational buckets for prioritization:
    - NEEDS_META_REFRESH
    - NEEDS_SNIPPET_REFRESH
    - NEEDS_SCHEMA_FIX
    - NEEDS_FACTUAL_REFRESH
    - HIGH_CANNIBALIZATION_RISK (manual governance)
    - READY_FOR_NEAR_PAGE_WINDOW (only when time window + gates pass)

Hard rule: Classification changes priorities and queues, not ownership.

31. Near-Page Generation Module (controlled expansion after observation window)

- After the defined observation window (e.g., 5 weeks), the system evaluates real demand (GSC queries) and can create Near-Page candidates.
- Mandatory gates before creation:

    - similarity / cannibalization gate (per D.05 thresholds)
    - intent eligibility gate (Near-Pages never own pure transactional)
    - inventory coverage gate (must have enough relevant published products)
    - per-entity caps (prevents index bloat)

- Creation behavior:

    - Near-Page is created as draft with default noindex, follow
    - candidate ownership is reserved (tentative), not hard-locked until validation passes
    - human review is required before becoming indexable (per your indexing/governance rules)

32. Incremental Supply After Initial Batch (new products & late arrivals)

- When new products arrive after the initial batch has already become ACTIVE:

    - they still enter RAW and are appended to the destination scope

    - they run single-record AI processing (not a batch re-run)
    - they must respect existing keyword locks and cannot “reclaim” already-owned head terms

- Near-page eligibility for these late arrivals:

    - can only add candidates if they do not collide with already-consumed ownership
    - otherwise they enrich existing eligible owners (per your rules

33. Manual Optimization (SEO/Marketing actions, role-gated)

- Humans work post-publish through:
    - Optimization Inbox (resolve signals)
    - Pipeline Content Actions overlay (mass regenerate specific blocks)
    - Content & Structure view (manual edits + locks)
    - Governance flows (ownership/canonical decisions)

- Manual actions must always log:
    - who did it, why, which blocks, before/after diff, and which version domain was affected

34. Blog Topic Suggestions (separate editorial lane)

- Informational queries discovered post-publish may generate blog topic suggestions.
- Blog creation/editing happens outside Pipeline:
    - blog keywords are blog-locked
    - Pipeline can optionally regenerate blog meta only (role-gated), never blog body
- Blog suggestions never change programmatic ownership automatically.

35. Reliability & Outages (non-blocking, no silent SEO changes)

- If any API is rate-limited or down (Viator/Google/Wikidata/SEMrush/GSC):

    - ingestion/jobs are queued
    - retries use exponential backoff
    - last valid cached data remains active
    - CMS shows non-blocking banner and backlog count

- Indexation impact:
    - no automatic index changes due to outage alone
    - safety fallback still applies: if a valid Published Record cannot be assembled → force noindex + remove from sitemaps

36. The loop repeats (continuous refinement)

- Daily: monitoring + triggers → enqueue safe jobs (within caps)
- Weekly: classification + prioritization updates
- Per observation window: Near-Pages (gated) + controlled expansion

- Always: publish queue throttling + audit logs + signal resolution discipline

d. Short description of each step

Destination submit

Source list of destinations from Viator API. Submit in the rosotravel backend and map it properly to destination types we want to support (country, region etc it will be defined in document). It is important as mapped destinations need to be weekly refreshed. It creates a destination tree.

Once all the destinations are submitted those which have min. 1 product will be automatically published only as a simple landing page with h1 and product list.

Raw product ingestion

All internal and external products are fetched through API or internal database and stored as RAW entries with version numbers, hashes and source type. RAW entries are grouped into batches based on country so keyword allocation and clustering run at scale. All products important and own are batched into one group for example product Italy raw.

The system stores all RAW data in a draft state, without generating any SEO elements yet.

A factual data enrichment placeholder is created during ingestion for future attraction/city metadata filling (Google Places + Wikidata).

Product classification and version rules

Each product receives product_version and source_type so the system can track changes over time. Internal products are rewritten by AI one time, then locked permanently, and all future edits are manual only unless a marketer explicitly unlocks a section.

External products allow AI rewriting for all sections unless a human edits a specific field, in which case that field becomes manual-override and remains locked.

Diff check for updated products

The system compares previous RAW versions with new RAW versions using hashes and field-level comparisons. If only non-text fields change (price, schedule, availability) the CMS updates only those fields without triggering AI rewriting.

If text fields change and the affected section is not manually locked, that section is sent back into the AI drafting pipeline for regeneration.

Image optimization

All images (external API images and internal images) pass through Cloudinary for automatic conversion to WebP, CDN delivery, lazy-loading and compression.If cities, regions, attractions or categories are missing hero images, the system generates them with Midjourney.

Alt text placeholders are created for all images for later AI filling.

Keyword data retrieval

The system calls SEMrush and GSC APIs to fetch keyword lists, search volumes, intent classification, competitor SERP data and related queries. The system retrieves tnavigational, informational, commercial, transactional, mix of 2 keywords for each product, attraction, city, region and country, near pages and blog.

All keywords are normalized and cleaned before embedding.

Factual Data Enrichment (Google Places + Wikidata)

The system enriches attractions, cities and regions with authoritative factual data:

➢ Google Places API

Used to retrieve operational and logistical facts:

- opening hours (weekday_text) → automation of FAQ
- formatted address
- geo coordinates (lat/lng)
- international_phone_number
- official website
- rating + review_count
- place_id for stable referencing

➢ Wikidata API

Used to retrieve stable encyclopedic metadata for schema + contextual facts:

- inception / construction year
- architectural style
- instance_of (cathedral, museum, palace, etc.)
- official website

- coordinates (fallback)
- isPartOf / hasPart relations
- identifiers for sameAs in schema.org

Purpose of the module

This module feeds:

- FAQ generation (“What are the opening hours?”)
- Schema enrichment (TouristAttraction → factual properties)
- Context for AI drafting (accurate facts vs invented)
- Global attractions map dataset accuracy

No manual insertion is required all data is fetched automatically.

Embeddings and vector storage

All product metadata, attraction metadata and keyword phrases are embedded using OpenAI embeddings. Embeddings are stored in Qdrant to enable semantic similarity checks, clustering, duplicate detection, and topic graph creation.

Qdrant becomes the single semantic memory for products, attractions, cities, things-to-do pages and near-pages.

LangChain clustering and keyword allocation

LangChain processes all embeddings, performs semantic clustering and assigns funnel stages to each keyword.
Keywords are allocated based on page type:
• products receive bottom-funnel keywords
• attractions receive mid-funnel keywords
• things-to-do pages receive broad transactional keywords
• immediate near-pages receive long-tail commercial keywords
• blog topics receive informational keywords
The system ensures that keyword clusters map cleanly to the correct page type.

Keyword ownership locking

The system checks if any keyword or keyword cluster is already owned by another page. Used or owned keywords cannot be assigned again, preventing keyword cannibalization.

New keywords can only be assigned if they are free, relevant and match the correct funnel stage and page type.

AI drafting

OpenAI generates full content drafts including:
• title
• short description
• long description
• highlights
• itinerary (if applicable)
• H1, H2 and H3 structure
• meta title and meta description
• initial FAQ placeholder container
• pro tips and expert insights
• alt text for images
• snippets for LLM previews
AI uses RAW data, keyword maps, token library, schema mapping and embeddings as its context. The system generates drafts for products, attractions, things-to-do pages, immediate near-pages, city pages, region pages, country pages, category pages and expert pages. FAQ generation also integrates Google Places + Wikidata facts where available.

Schema drafting

The system generates schema JSON-LD for each entity type including TouristAttraction, TouristDestination, TouristTrip, Offer, FAQPage, Person and Organization.

Schemas include placeholders for final URLs to be inserted during the publishing stage. Schemas also generate isPartOf, hasPart and nearby relations where applicable. Schemas now include enhanced factual fields from Google Places (openingHoursSpecification) and Wikidata (sameAs, inception, architecturalStyle).

Quality validation

A second GPT validation pass checks factual accuracy, consistency with the input data, natural writing quality and SEO balance. Copyscape checks English product pages for external duplicates.

Screaming Frog N-gram scanning checks for internal repetition, template patterns and low-diversity issues.

FAQ keyword retrieval

After AI QC, the system retrieves FAQ-intent keywords using SEMrush “Questions”, SEMrush People Also Ask, and GSC queries from real users. The system embeds these questions, deduplicates them via Qdrant similarity (to avoid duplicate FAQs across products/cities/attractions), assigns FAQ ownership, and prepares them for AI drafting.

FAQ AI drafting

OpenAI generates final FAQ content using:

- locked FAQ keywords,
- page context,
- RAW data (opening hours, practical info),
- existing content to avoid contradictions.

This ensures FAQ accuracy and prevents cannibalization.

Content status assignment

Each product and page receives a QC status flag: passed or needs_review.
Sections edited by humans in the CMS become permanently locked and excluded from future AI rewrites.

Human QA sampling

The system selects 3–5 percent of pages from each batch for manual evaluation.
A marketer or SEO specialist evaluates clarity, accuracy, keyword usage, UX quality and style.
If the sample passes, the full batch is approved.
If the sample fails, prompts or keyword allocation rules are adjusted and drafts are regenerated.

Publishing

The CMS generates final URL slugs for all page types.
The system publishes:
• product pages
• attraction pages
• things-to-do pages
• category pages
• city, region and country pages

• expert pages
• immediate near-pages
• internal supporting pages
Structured data is injected into each page, and all sitemaps and AI-sitemaps are updated.
The ai-summary.json dataset is regenerated to improve LLM visibility and integration with ChatGPT and Perplexity.

Factual schema fields from Google Places + Wikidata are now inserted into the live JSON-LD.

Embedding the final published version

All published pages are embedded again using OpenAI embeddings and stored in Qdrant.
Relations (isPartOf, hasPart, nearby, related) are embedded and stored for future contextual search and clustering.
This enables accurate reranking, SEO refinement and dynamic near-page creation.

Global attractions map generation

After publishing, the system aggregates all attractions, cities and regions with coordinates into a unified dataset:

- ➢ Google Places coordinates
- ➢ Wikidata identifiers
- ➢ Attraction metadata

This dataset powers the Explore Map interface and the /explore/ai-dataset/ feed for LLMs.
The map updates automatically whenever new attractions or regions are published.

Translation and localization

Azure Translator or OpenAI translation models generate localized versions of each page with semantic preservation and SEO-safe rewriting.

Keywords that are locked in English remain locked in translated versions but are translated appropriately. All localized pages enter the publishing pipeline.

Monitoring phase

GSC and GA4 collect key performance metrics for every page including impressions, clicks, CTR, bounce rate, reading time and conversions.

LangChain classifies pages weekly into OK, needs optimization or A/B test candidate based on real performance.

Optimization queue

Pages with low CTR, high bounce rate or no impressions for 30 days are added to the optimization queue.
GPT generates improved meta titles, meta descriptions or introductory paragraphs while respecting locked sections.
The revised pages are republished and embedded again.

Dynamic near-page generation

Exactly 5 weeks after initial publishing, GSC and SEMrush data are analyzed to detect new keywords and long-tail search behavior.
Embedding similarity checks determine whether an existing page covers the topic; if not, the system creates a new dynamic near-page draft.
Dynamic near-pages target long-tail queries that emerge only after real user activity.

Near-page publishing

Dynamic near-pages follow the same full pipeline: AI drafting, schema generation, quality validation, sampling, publishing and translations.
These pages expand organic visibility based on confirmed user demand rather than predictions.

Blog topic generation

Informational keywords from SEMrush are clustered separately into blog-topic groups.
Blog topics never overlap with products, attractions or near-pages.
All blog content is written manually by expert writers to preserve E-E-A-T.
Blog topics are prepared before publishing but written and released on a separate editorial calendar.

Continuous improvement loop

As new batches publish and mature, the system continuously refines keyword ownership, AI prompting, clustering logic, preview snippets, schema structures and internal linking.

Each full cycle improves the next batch, making the entire ecosystem more accurate, more scalable and more competitive over time.

4.2 External Data Inputs & Failover Contract (APIs, Caching, Rate Limits, Outages)

Właściciel treści: Viator/OTA, Google Places, Wikidata, SEMrush, GSC, GA4, media/CDN
Tu ma być: retry/backoff, cache, “last valid data”, outage banners, brak utraty danych, brak silent SEO changes.

01. Integrations

SEO & Performance Data

- **Google Search Console API** — impressions, CTR, queries, missing keywords, performance signals.
- **GA4** — behavior metrics: bounce, dwell time, conversions, engagement patterns.

Keyword Intelligence

- **SEMrush API** — external volumes, KD, intent categories, competitive SERPs.
- **OpenAI Embeddings** — semantic clustering of keywords, duplicate detection, topic mapping.
- **Qdrant Vector DB** — stores all embeddings for anti-cannibalization, retrieval, topic ownership.

AI Generation & Orchestration

- **OpenAI GPT** — main LLM for all content: meta, descriptions, headers, FAQs, snippets, alt text.

- **LangChain** — orchestrator for chain logic: keyword → content → QA → routing.
    - Similarity checks (e.g., 0.92 threshold)
    - Regeneration triggers
    - Funnel assignment routing

Media & Asset Infrastructure

- **Cloudinary API** — CDN, WebP/AVIF optimization, auto-compression, lazy-loading.
- **Nano Banana pro** — only for Country / Regions / other small entities oraz near pages

Content Safety & Uniqueness

- **Copyscape API** — plagiarism scanning for English product/tour pages.
- **Screaming Frog N-gram Check** — detects template repetition and AI footprint.

Translation

- **Open AI API** — multi-language translation with semantic preservation across all pages.

Factual / Local

- **Google Places API** – opening hours, address, phone, website, rating.
- **Wikidata API**

Error Handling & Fallback

8) Broken or Invalid Media Assets - failover

8.1 Definition

- 404/403, expired URLs, invalid dimensions, suspected license violations.

8.2 Fallback order

1. Retry fetch (limited attempts).
2. Use last_valid_cache media.
3. Use licensed placeholder where allowed.

8.3 SEO rules (binding)

- Supplier media must not be included in image sitemaps.
- Supplier media must not be reused on other pages/entities.
- Placeholder images must not be added to image sitemaps.

8.4 UI notification

- MEDIA_REVIEW_REQUIRED (Warning or Blocking depending on policy)

10) API Rate Limits / Outages (System-level)

10.1 Affected APIs

- OTA feeds (products, offers, availability)
- Google Places
- Wikidata
- SEMrush
- Google Search Console

10.2 Handling

- Queue ingestion tasks.
- Retry with exponential backoff.
- Use last_valid_cache data for Published assembly where allowed.

4.3 Destination Preupload (Country/Region): Create RAW Destination Snapshots + Normalize Hierarchy

Tu ma być: upload → RAW_DESTINATION snapshot (versioned) → canonicalization → region→country relacje (bez batchy).

a. Preupload: preparation Destination:

01. Defined Entity and mapping

"AREA"
"CITY"
"COUNTRY"
"COUNTY"
"DISTRICT"
"HAMLET"
"ISLAND"
"NATIONAL PARK"
"NEIGHBORHOOD"
"PENINSULA"
"PROVINCE"

"REGION"
"STATE"
"TOWN"
"UNION TERRITORY"
"VILLAGE"
"WARD"

02. Ingestions for Countries and Region and Raw objects

Preupload is the foundation layer for destination entities that must exist before batch ingestion of Products / Cities / Attractions and before any keyword ownership or AI generation. It provides:

- A stable destination graph (Country → Region → City) used by all relationship rules.
- A clean seed for Run SEO Auto at destination level (Country/Region only).
- A cost-controlled ingestion path (destination API first; paid sources only when needed).

1. Preupload ingestion (Country + Region)

- System fetches destination rows from the Destination API (same schema as City “destinations[]” response).
  System creates/updates RAW destination entities:
    - `RAW_COUNTRY_Version`
    - `RAW_REGION_Version`

- System builds destination hierarchy using `destinationId` + `parentDestinationId` and stores it in `relationship_fields`.

2. Activation gate (controls whether Run SEO Auto is allowed)

- A destination becomes Active only if `product_count >= 1` (computed internally from Published Products mapped to this destination via the hierarchy).

- If `product_count = 0`:
    - entity stays stored
    - entity can remain unpublished / unsitemapped (per your global rules)
    - Run SEO Auto is disabled

3. Run SEO Auto (Country/Region only; separate run per entity)

- Marketer can run Run SEO Auto independently for each destination entity:

    - Italy (Country) = 1 run
    - Lazio (Region) = 1 run
    - (Island if modeled as Region-like destination) = 1 run

- Run SEO Auto uses only destination metadata (canonical English name + destination type + parent country name for Region).
- Destination-level Run SEO Auto does not use embeddings/clustering and does not use Product data for seed construction.

4. Paid source usage policy (cost control)

- Destination API ingestion is the default baseline and runs on a slow cadence.
- Paid sources (Google / Wikidata) are executed only when needed:
    - when destination becomes Active and is being prepared for publication
    - when generating “facts_and_curiosities” (if enabled)
    - on a long refresh cadence (e.g., 180 days) to avoid overspending

Column meaning (binding, keep consistent across all RAW tables)

- Source = system origin of the value (API / SYSTEM / WIKIDATA, etc.).
- Refresh (days) = maximum intended refresh cadence. New versions occur only if data changes.
- Required = hard validation (if missing → ingestion flags error / blocks promotion if required).
- Derived = computed from other stored fields (not fetched directly).

---

1) RAW_COUNTRY_Version (versioned snapshot)

A) Core identity & snapshot meta

Field	Source	Refresh (days)	Derived	Notes
entity_id	SYSTEM	once	Yes	Internal immutable ID
stable_code	SYSTEM	once	Yes	CTR_<ULID>
entity_type	SYSTEM	once	Yes	COUNTRY
source_type	SYSTEM	once	Yes	INTERNAL / EXTERNAL
external_id (destinationId)	API OTA	API OTA	No	*Required for EXTERNAL destinations (preupload)
destination_type_raw	API OTA	API OTA	No	Expected values include

					COUNTRY
raw_version	SYSTEM	on change	Yes		Increments only when stored RAW snapshot changes
raw_ingested_at	SYSTEM	on ingest	Yes		Timestamp

B) content_fields (seed inputs only; indexing rules live elsewhere)

Field	Source	Refresh (days)	Required	Derived	Notes
canonical_name_en	API OTA	API OTA	Yes	No	Used for Run SEO Auto seeds (EN canonical)
destination_url	API OTA	API OTA	No	No	destinationUrl
lookup_id	API OTA	API OTA	No	No	lookupId if available

C) factual_fields (baseline; no paid enrichment required at preupload)

Field	Source	Refresh (days)	Required	Derived	Notes
default_currency_code	API OTA	API OTA	No	No	defaultCurrencyCode
time_zone	API OTA	API OTA	No	No	timeZone
languages[]	API OTA	API OTA	No	No	Destination languages list (if provided)
country_calling_code	API OTA	API OTA	No	No	countryCallingCode
iata_codes[]	API OTA	API OTA	No	No	May be empty for countries
center_coordinates {lat,lng}	API OTA	API OTA	No	No	center.latitude/longitude

D) relationship_fields (MUST exist; hierarchy graph)

Field	Source	Refresh (days)	Required	Derived	Notes

parent_relations.country (self)	SYSTEM	90	Yes	Yes	Self pointer for graph consistency
parent_relations.region	SYSTEM	90	No	Yes	Not applicable for Country (null)
relationship_edges[]	SYSTEM	90	Yes	Yes	Must include COUNTRY_ROOT or equivalent root edge
relationship_hash	SYSTEM	on change	Yes	Yes	Hash over relationship_fields

E) enrichment_identity_fields (optional; paid sources are NOT mandatory at preupload)

Field	Source	Refresh (days)	Required	Derived	Notes
wikidata_id (QID)	WIKIDATA (optional)	180	No	No	Fetch only when destination becomes Active / publish-prep requires facts
sameAs[]	WIKIDATA	180	No	No	External IDs/links, optional

(optio
nal)

F) hashes (naming MUST match the global convention)

Field	Source	Required	Notes
content_hash	SYSTEM	Yes	Hash of content_fields only
factual_hash	SYSTEM	Yes	Hash of factual_fields only
media_hash	SYSTEM	Yes	Countries may have empty media; still hash empty set deterministically
relationship_hash	SYSTEM	Yes	Hash of relationship_fields only
offer_hash	SYSTEM	No	Not used for Country (null)
realtime_offer_hash	SYSTEM	No	Not used for Country (null)

---

2) RAW_REGION_Version (versioned snapshot)

A) Core identity & snapshot meta

Field	Source	Refresh (days)	R	Derived	Notes

entity_id	SYSTEM	once	Y	Yes	Internal immutable ID
stable_code	SYSTEM	once	Y	Yes	REG_<ULID>
entity_type	SYSTEM	once	Y	Yes	REGION
source_type	SYSTEM	once	Y	Yes	INTERNAL / EXTERNAL
external_id (destinationId)	API OTA	API OTA	Y	No	*Required for EXTERNAL destinations (preupload)
parent_external_id (parentDestinationId)	API OTA	API OTA	Y	No	*Required for EXTERNAL regions to bind hierarchy
destination_type_raw	API OTA	API OTA	Y	No	Expected values include REGION / ISLAND (if modeled)

raw_version	SYSTEM	on change	Y	Yes	Increments only when stored RAW snapshot changes
raw_ingested_at	SYSTEM	on ingest	Y	Yes	Timestamp

B) content_fields (seed inputs only; indexing rules live elsewhere)

Field	Source	Refresh (days)	Required	Derived	Notes
canonical_name_en	API OTA	API OTA	Yes	No	Used for Run SEO Auto seeds (EN canonical)
destination_url	API OTA	API OTA	No	No	destinationUrl
lookup_id	API OTA	API OTA	No	No	lookupId if available

C) factual_fields (baseline; no paid enrichment required at preupload)

Field	Source	Refresh (days)	Required	Derived	Notes
default_currency_code	API OTA	API OTA	No	No	Often inherited at runtime; keep if present
time_zone	API OTA	API OTA	No	No	timeZone
languages[]	API OTA	API OTA	No	No	May be empty
iata_codes[]	API OTA	API OTA	No	No	Optional
center_coordinates {lat,lng}	API OTA	API OTA	No	No	center.latitude/longitude

D) relationship_fields (MUST exist; hierarchy graph)

Field	Source	Refresh (day	Required	Derived	Notes

		s)			
parent_relations.country	SYSTEM (from parentExternalId resolution)	90	Yes	Yes	Region must resolve to a Country
parent_relations.region (self)	SYSTEM	90	Yes	Yes	Self pointer
relationship_edges[]	SYSTEM	90	Yes	Yes	Must include REGION_IN_COUNTRY
relationship_hash	SYSTEM	on change	Yes	Yes	Hash over relationship_fields

E) enrichment_identity_fields (optional; paid sources are NOT mandatory at preupload)

Field	Source	Refresh (days)	Required	Derived	Notes

wikidata_id (QID)	WIKIDATA (optional)	180	No	No	Fetch only when Active / publish-prep needs facts
sameAs[]	WIKIDATA (optional)	180	No	No	External IDs/links, optional

F) hashes (naming MUST match the global convention)

Field	Source	Required	Notes
content_hash	SYSTEM	Yes	Hash of content_fields only
factual_hash	SYSTEM	Yes	Hash of factual_fields only
media_hash	SYSTEM	Yes	Regions may have empty media; hash empty set deterministically
relationship_hash	SYSTEM	Yes	Hash of relationship_fields only
offer_hash	SYSTEM	No	Not used for Region (null)
realtime_offer_hash	SYSTEM	No	Not used for Region (null)

Run SEO Auto (Destination Entities Only: Country / Region / Island)

Purpose (binding)
Run SEO Auto generates **destination-level SEO ownership + destination page blocks** for **preuploaded destination entities** only (Country / Region / Island if modeled as destination).
It is designed to capture **broad navigational + broad commercial** demand for the destination entity **without** entering city/POI/product discovery logic.

Hard scope exclusions
Run SEO Auto MUST NOT be used for:

- City
- Attraction / POI
- Product
- Near-pages
  Those entity types are handled by the batch + clustering pipeline.

---

1) Execution granularity (hard)

Run SEO Auto is executed **per entity**, always separately:

- 1 Country entity → 1 Run SEO Auto job
- 1 Region entity → 1 Run SEO Auto job

If UI allows multi-select (many entities), backend still creates **one job per entity** with separate:

- strategy_version_id
- ai_version_id
- unit estimate + actual units
- audit log entries

---

2) Eligibility gate (“Active” rule)

Run SEO Auto is available only when:

- `destination.product_count ≥ 1` published product linked to this destination via relationship graph.

If `product_count = 0`:

- entity remains stored (preupload)
- Run SEO Auto disabled
- entity remains unpublished / unsitemapped (per index rules)

Counting rule (binding):

- Country `product_count` = all published products mapped to the Country (directly or through child Regions/Cities, depending on mapping rules)
- Region `product_count` = all published products mapped to the Region (directly or through child Cities)

---

3) Inputs & source of truth (NO seed phrase repetition)

Run SEO Auto uses **only destination entity metadata** as inputs.
Seed phrases and forbidden inputs are defined in the section:
“Seed for countries, regions and other entities (all separate)” (this document).
Run SEO Auto MUST reference that section and MUST NOT redefine seed lists.

Allowed inputs (hard) — where it comes from:

- Destination canonical name (EN):
  - `RAW_COUNTRY_Version.content_fields.canonical_name_en` OR
  - `RAW_REGION_Version.content_fields.canonical_name_en`
- Destination type:
  - `entity_type` = COUNTRY / REGION / ISLAND
- For Region disambiguation:
  - parent country resolved via `relationship_fields.parent_relations.country` → country

canonical_name_en

- Language for keyword retrieval:
    - EN only (translations happen later)

Forbidden inputs (hard):

- any city name injection
- any POI name injection
- any product title injection
- any factual enrichment content (Google/Wikidata text) used for seeding

---

4) SEMrush intent taxonomy handling (includes “Mixed”)

SEMrush intent values are treated as authoritative:

- Informational
- Navigational
- Commercial
- Transactional
- Mixed (mix of 2)

Destination ownership allowlist (hard):

- Allowed: Navigational, Commercial
- Allowed: Mixed ONLY if it contains ONLY allowed intents and **excludes Transactional**
- Conditionally allowed: Informational ONLY when it matches **destination-wide planning** patterns (travel guide / travel / visit / itinerary) AND contains destination name
- Forbidden: Transactional always

- Forbidden: Mixed if it includes Transactional

Mixed intent rule (binding):

- Mixed(Navigational + Commercial) → allowed
- Mixed(Informational + Commercial) → allowed only if it matches destination-wide planning patterns (otherwise reject)
- Mixed(* + Transactional) → reject always

---

5) SEMrush retrieval calls (destination-level, low complexity)

Destination-level Run SEO Auto uses **no embeddings, no clustering, no cross-entity analysis**.

Calls performed:

1. **Destination keyword pool retrieval**

- Uses the seed phrases defined in the destination seed section (no duplication here).
- Export columns MUST include at least: keyword, volume, KD, intent (if available), CPC (optional)

2. **Destination questions retrieval (FAQ candidates)**

- Query phrase MUST be `canonical_name_en` only (single phrase) for cost stability.

Post-filtering is performed in Rosotravel backend (binding).

---

6) Filtering rules (hard, destination-safe)

A keyword can be considered for destination ownership only if ALL are true:

1. Contains destination name OR matches allowed destination templates (as defined in the seed section)
2. Does NOT contain:

- any known city names for that destination (from destination graph)

- any attraction/POI names
- transactional modifiers: tickets, ticket, price, discount, skip the line, entry fee, booking, book, promo
- OTA brand terms: viator, getyourguide, tripadvisor

3. Intent passes destination allowlist rules (including Mixed handling above)

Hard reject cases:

- Any Transactional keyword
- Any Mixed containing Transactional
- Any Informational long-tail (“opening hours”, “how to get”, “history”, etc.) unless explicitly allowed as destination-wide planning pattern

---

7) Selection logic (deterministic; no duplication of formulas)

The selector chooses:

- `primary_keyword` = exactly 1
- `secondary_keywords[]` = up to 2

Selection uses the **existing deterministic scoring model already defined in your document** (Volume + TemplateFit + Penalties), with these binding constraints:

- Primary keyword MUST be from the core destination patterns (things to do / tours / attractions / sightseeing), unless none exist.
- Secondary keywords MUST be non-duplicates:
    - similarity(primary, candidate) < 0.92
- Prefer template diversity for secondaries (avoid choosing three near-identical variants).

(Do NOT reintroduce embeddings/clustering at destination level.)

---

8) Output per destination entity (Run SEO Auto writes)

Each execution stores outputs ONLY on that destination entity:

- primary_keyword (1) — destination-level ownership keyword
- secondary_keywords (≤2)
- destination_snippet (180–260 chars)
- meta_title, meta_description
- destination_faq_questions (exactly 7, destination-wide only)
- destination_faq_answers (template-based; editable + lockable per QA)
- highlights (5–7 bullets)
- facts_and_curiosities (5–7 factual bullets; allowed only if factual enrichment data exists; AI may phrase but never invent)
- ai_version_id, generated_at

Non-propagation rule (hard):
These outputs MUST NOT propagate to cities, attractions, products, or near-pages.

---

9) Governance & regeneration rules (hard)

- Role-gated: only SEO Admin / Marketing Admin can run or regenerate.
- Every run creates a new `ai_version_id` and writes an audit event.
- Regeneration does NOT auto-change publishing/indexation state.
- The last approved AI version is rendered publicly.

4.4 Destination Seeds + Run SEO Auto (Country/Region only, per entity)

Tu ma być: seed rules dla destination entities + SEMrush calls + rule-based selection + destination outputs (snippet/meta/FAQ/highlights/facts).
Hard rule: nadal poza batchami.

01. Seed for countries, regions and other entities (all separate)

Scope & Intent

The destination-level seed is used **only** for country / region / island / others entities triggered via **Run SEO Auto**. Its purpose is to retrieve **broad navigational and commercial keywords** for the destination as a whole.

This seed **must not** attempt to discover:

- city-level intent,
- attraction-level intent,
- product-level transactional intent.

Source of Truth for Seed Construction

Seed phrases are built **exclusively from destination entity metadata**, never from products.

Each destination entity provides:

- canonical_name (English, normalized)
- destination_type (country | region | island)
- parent_name (for regions / islands)
- language = EN (fixed; translations handled later)

Country-Level Seed Construction

For a **country** destination, the seed consists of **fixed template phrases** combined with the country canonical name.

Seed phrase set (country):

- `things to do in {{Country_Name}}`
- `{{Country_Name}} tours`
- `{{Country_Name}} attractions`
- `{{Country_Name}} travel`
- `{{Country_Name}} sightseeing`
- `visit {{Country_Name}}`

Rules:

- Always use canonical English name.
- No cities, regions, or attractions included.

- No plural/singular expansion manually (SEMrush handles variants).
- Seed list size intentionally small (6–8 phrases max).

Region / Island-Level / others entity seed Construction

For a **region or island**, the seed must include **parent country disambiguation** to avoid SERP pollution.

Seed phrase set (region / island):

- `things to do in {{Region_Name}} {{Country_Name}}`
- `{{Region_Name}} {{Country_Name}} tours`
- `{{Region_Name}} attractions`
- `visit {{Region_Name}} {{Country_Name}}`
- `{{Region_Name}} sightseeing`

Rules:

- Parent country name is mandatory in at least 50% of seed phrases.
- Do not include city names even if the region contains only one city.
- No product or attraction names.

What Must NEVER Be Included in Destination Seeds

The following inputs are explicitly forbidden at destination level:

- City names
- Attraction / POI names
- Product titles
- “tickets”, “price”, “opening hours”
- Dates, seasons, or pricing modifiers
- Any content from Google Places or Wikidata

Reason:
Destination pages must **own only navigational and broad commercial intent**, not transactional or informational long-tail.

SEMrush Query Mode (Destination-Level)

- Database: English (US or Global EN, per configuration)
- Query type: Keyword Overview + Questions
- Intent filtering:
    - Include: navigational, commercial
    - Exclude: transactional, informational long-tail
- No embeddings, no clustering, no cross-entity analysis at this level.
- Output Per Destination Entity

AI-Generated Outputs for destinations (per execution)

- Remember this is only for country, region, islands, village etc.
- Not eligible for cities or attractions

Each run produces the following fields:

1. **primary_keyword**
    - Exactly 1 keyword
    - Broad, destination-level intent only
    - Navigational or broad commercial
    - Locked at destination level
2. **secondary_keywords**
    - Maximum 2 keywords
    - Close variants of the primary keyword
    - Must pass similarity rules (< 0.92)
3. **destination_snippet**
    - 180–260 characters
    - Used as:
        - SEO intro
        - H1-adjacent description
        - Canonical destination summary
    - Supports later manual edits and CHAIN / UNCHAIN control
4. **meta_title**
    - AI-generated, destination-specific
    - Editable manually
    - Regenerable if AI remains chained
5. **meta_description**
    - AI-generated
    - Editable manually
    - Regenerable if AI remains chained
6. **destination_faq_questions**
    - Exactly 7 informational questions
    - Destination-wide only (country / region scope)
    - Topics aligned with:
        - travel planning

        - logistics
        - safety
        - seasonality
        - culture
    - No transactional intent

7. **destination_faq_answers**
    - Answers generated using country / region FAQ templates
    - Fact-checked against factual data sources
    - Editable and lockable per question

8. **highlights (structured inspiration)**
    - 5–7 bullet points
    - Non-dynamic, stable SEO block
    - AI-generated, manually editable

9. **facts_and_curiosities (factual block)**
    - 5–7 verified factual bullets
    - No AI hallucinations allowed
    - Sourced from factual enrichment layer
    - AI may phrase but not invent facts

10. **ai_version_id**
    - Unique identifier for this Run SEO Auto execution

11. **generated_at**
    - Timestamp of generation
- generated_at timestamp

These outputs are stored **only on the destination entity** and never propagated to child cities or pages.

01. Run SEO Auto - Semrush API calls (logical mapping)

1) Destination keyword retrieval (broad keywords)

Call type: “Keyword Ideas / Phrase match” (SEMrush endpoint depending on plan). Request parameters (must be enforced):

- `database`: `{{semrush_database}}` (fixed EN index)
- `phrase`: each seed phrase is sent as an independent request OR batched if SEMrush supports multi-phrase in one call
- `export_columns`: keyword, search_volume, keyword_difficulty, intent (if available), cpc (optional), competitive_density (optional)
- `display_limit`: 200 per seed phrase (cap)
- `display_offset`: 0

- `device`: desktop (optional; fixed)
- `country`: not used (database already defines market)

Post-filtering rules (done in Rosotravel backend, not in SEMrush):

- Include only: navigational + broad commercial keywords
- Exclude: transactional modifiers (tickets, price, discount, skip-the-line), and informational patterns (opening hours, how to get, history)
- Deduplicate by lowercase normalization + basic similarity string match

2) Destination questions retrieval (FAQ candidates)

Call type: “Questions / Keyword Questions”

Request parameters:

- `database`: `{{semrush_database}}`
- `phrase`: `{{canonical_name_en}}` (NOT all seed phrases; keep cost stable)
- `export_columns`: question, volume (if available)
- `display_limit`: 100
- `display_offset`: 0

Post-filtering rules:

- Keep only destination-wide questions (visa, safety, currency, best time, transport, tipping, itinerary length, cost level)

- Remove questions that clearly belong to:
    - a city (contains city names)
    - an attraction/POI
    - transactional intent (tickets/price/book)

- Deduplicate using simple text similarity (no embeddings at destination level)

3) Selection logic (rule-based scoring)

The destination-level selector chooses 1 primary and up to 2 secondary keywords from the SEMrush output using a deterministic scoring model (no embeddings, no clustering).

Pre-filters (must pass to be scored)

A keyword is eligible only if all conditions are true:

- Contains the destination name (country/region) OR is an exact “things to do in {{destination}}” pattern.
- Does NOT contain:

    - city names (any known city list for that country/region)

    - attraction/POI names (any known attraction list for that destination)
    - transactional modifiers: tickets, ticket, price, discount, skip the line, skip-the-line, entry fee, booking, book, promo
    - shopping/OTA terms: viator, getyourguide, tripadvisor

- Intent must be navigational or broad commercial. If SEMrush intent is missing, infer intent by pattern:

    - Broad commercial allowed patterns: things to do, tours, attractions, sightseeing, travel, travel guide, itinerary (borderline, see rule 4).

Keyword normalization (before scoring)

- lowercase
- trim spaces
- remove punctuation
- collapse multiple spaces
- standardize destination name variants to canonical (e.g., “u.k.” → “united kingdom”)

Scoring components (deterministic). For each eligible keyword k, compute:

A) VolumeScore (0–50)

- Take SEMrush volume for k.

- Normalize within the eligible set for this destination:
    - VolumeScore = 50 * (volume(k) / max_volume_in_set)
    - (If SEMrush volume is missing: set VolumeScore = 10.)

B) TemplateFitScore (0–30)
Assign exactly one bucket:

- 30 points: exact match things to do in {{destination}}

- 26 points: {{destination}} tours
- 24 points: {{destination}} attractions
- 22 points: {{destination}} sightseeing
- 18 points: visit {{destination}} OR {{destination}} travel
- 12 points: {{destination}} itinerary OR {{destination}} travel itinerary OR {{destination}} travel guide (borderline)

C) CleanlinessPenalty (0 to -25)
Subtract points if any apply:

- -10 if keyword contains “best” / “top” / “recommended” (often overlaps with other page types; still allowed but deprioritized)
- -15 if keyword contains “itinerary” (allowed, but should rarely become primary)

- -25 if keyword contains a second location entity (e.g., “lazio rome”, “lazio naples”) even if still destination-related

D) FinalScore
FinalScore = VolumeScore + TemplateFitScore + CleanlinessPenalty

4) Primary keyword selection rule

- Sort eligible keywords by FinalScore (desc).
- Primary keyword is the top keyword that satisfies:
    - must be in one of these patterns:
      things to do in {{destination}} OR {{destination}} tours OR {{destination}} attractions OR {{destination}} sightseeing
    - “itinerary” keywords can be primary ONLY if none of the four patterns exist in the eligible set.

5) Secondary keyword selection rule (up to 2)

After selecting primary:

- Remove duplicates using keyword similarity:
    - If similarity(primary, candidate) ≥ 0.92 → reject (duplicate)

- Prefer candidates that cover different templates:

    - If primary is “things to do…” → prefer “{{destination}} tours” and “{{destination}} attractions”
    - If primary is “{{destination}} tours” → prefer “things to do…” and “{{destination}} attractions”

- If multiple candidates tie, choose higher FinalScore.

6) Worked selection example (Italy)

Eligible scored keywords (illustrative):

- things to do in italy
    - TemplateFitScore 30, VolumeScore highest → FinalScore highest → selected as primary

- italy tours
    - TemplateFitScore 26, high volume → selected as secondary #1

- italy attractions
    - TemplateFitScore 24, high volume → selected as secondary #2

- italy sightseeing
    - TemplateFitScore 22 → loses to attractions/tours due to lower score

- best places to visit in italy
    - CleanlinessPenalty -10 (“best”) and weaker template fit → not selected

- italy travel itinerary
    - TemplateFitScore 12 + Penalty -15 (“itinerary”) → allowed but strongly deprioritized

4) Output selection rules (no clustering)

From the filtered keyword pool:

- primary_keyword: choose 1 keyword with best mix of relevance + volume (rule-based scoring)
- secondary_keywords: choose up to 2 close variants (must not be duplicates; enforce similarity < 0.92)

Store outputs per destination entity:

- primary_keyword (1)
- secondary_keywords (≤2)
- faq_questions (set)
- ai_version_id, generated_at
- generated snippet/meta/FAQ answers using destination templates

Regeneration & Governance Rules

- Run SEO Auto can be executed only when product_count ≥ 1.
- Only Marketing Admin can regenerate.
- Each regeneration creates a new ai_version_id.
- Regeneration does **not** change indexation or publishing state.
- The last approved AI version is the one rendered publicly.

02. Example Validation Italy (Country) vs Lazio (Region) Seeds + Resulting Keyword Sets

BItaly (Country) — Seed phrases sent to SEMrush

1. things to do in Italy
2. Italy tours
3. Italy attractions
4. Italy travel
5. Italy sightseeing

6. visit Italy

Lazio (Region) — Seed phrases sent to SEMrush

1. things to do in Lazio Italy
2. Lazio Italy tours
3. Lazio attractions
4. visit Lazio Italy
5. Lazio sightseeing

Example returned keyword pool (illustrative) and how it gets filtered

Italy — example SEMrush returns (mixed)

Potential returned keywords (illustrative):

- things to do in italy
- italy tours
- italy travel itinerary
- best places to visit in italy
- italy sightseeing
- italy attractions
- italy guided tours
- italy day trips
- italy tickets (transactional modifier)
- colosseum tickets (POI transactional)
- rome tours (city-level)

Filtering outcome (Italy destination rules):

- Keep (navigational/broad commercial):
    - things to do in italy
    - italy tours
    - italy attractions
    - italy sightseeing
    - best places to visit in italy
    - italy travel itinerary (borderline; keep only if treated as broad planning)
- Remove (not allowed at destination level):
    - italy tickets (transactional)
    - colosseum tickets (POI)
    - rome tours (city-level)

Selected outputs (example):

- primary_keyword: “things to do in italy”
- secondary_keywords: “italy tours”, “italy attractions”

FAQ questions retrieved (illustrative):

- is italy safe for tourists
- best time to visit italy
- do you need a visa for italy
- how many days in italy is enough
- what is the currency in italy
- how to get around italy
- Is the italy worth visit?

Stored AI version fields:

- primary_keyword, secondary_keywords
- destination_faq_questions (deduped)
- destination_snippet + meta title/description + FAQ answers

Lazio — example SEMrush returns (mixed)

Potential returned keywords (illustrative):

- things to do in lazio
- lazio italy tours
- lazio attractions
- lazio sightseeing
- rome day trips (city-level)
- vatican tours (POI/city-context)
- lazio travel guide
- lazio tickets (transactional)

Filtering outcome (Lazio destination rules):

- Keep:
    - things to do in lazio
    - lazio italy tours
    - lazio attractions
    - lazio sightseeing
    - lazio travel guide (broad planning)
- Remove:

    - lazio tickets (transactional)
    - rome day trips (city-level)
    - vatican tours (POI/city-context)

Selected outputs (example):

- primary_keyword: “things to do in lazio”
- secondary_keywords: “lazio italy tours”, “lazio attractions”

FAQ questions retrieved (illustrative):

- what to see in lazio besides rome
- best time to visit lazio
- how to travel around lazio
- is lazio good for families
- how many days in lazio
- Is the lazio worth visiting in winter?
- What to see with family in Lazio?

What this proves (validation logic)

Even with only 5–6 seed phrases, SEMrush typically returns:

- many close variants,
- related planning keywords,
- some city/POI leakage,
- some transactional modifiers.

4.5 Product Entry Points (Execution Start): INTERNAL Add vs EXTERNAL Ingest

Purpose

This section defines the execution backbone of Model C. It formalizes how products enter the system, how they are counted, validated, filtered, and selected before any decision logic, UX rendering, or SEO processes are applied.

This section does NOT define:

- UI behavior (see Chapter 3),

- Workflow governance (later section),

- Archetype/intent/bundle logic (Phase C).

It defines RAW → VALIDATION → POOL → SET → BATCH TRIGGER rules.

4.5.0 Core Execution Principles

1. Premium Filtering Principle

Rosotravel never lowers quality thresholds to increase coverage.

2. Strictest-Category Rule

If a product qualifies for multiple Viator category groups, it must satisfy the strictest threshold (Group 01).

3. No Silent Swaps

Products are not automatically removed from SET due to minor feed changes. Stability is preserved.

4. RAW is a Mirror

External provider payload is stored as-is. Field names are not renamed.

5. Only SET triggers SEO and AI batch.

4.5.1 Viator Field Contract (Exact Field Usage)

The system must use exact provider field paths. No renaming.

Core fields (NOW – required in Phase A):

status

productCode

createdAt

lastUpdatedAt

title

description

inclusions

exclusions

additionalInfo

timeZone

tags[]

destinations[] (ref, primary)

supplier (name, reference)

reviews.totalReviews

reviews.combinedAverageRating

itinerary

itinerary.itineraryItems[]

itinerary.privateTour

itinerary.duration.fixedDurationInMinutes

logistics.start

logistics.end

logistics.redemption

logistics.travelerPickup.pickupOptionType

pricingInfo.ageBands[]

ticketInfo.ticketTypes[]

bookingRequirements.minTravelersPerBooking

bookingRequirements.maxTravelersPerBooking

bookingQuestions[]

cancellationPolicy.type

images[]

productOptions[]

productOptions[].languageGuides

TBD (Phase C or later refinement):

pricingInfo.type

pricingInfo.unitType

itinerary.skipTheLine

itinerary.itineraryItems[].admissionIncluded

bookingConfirmationSettings

translationInfo

4.5.2 A0 — Ingest (RAW Layer)

All products are ingested from the provider API into RAW_ALL_PRODUCTS.

Rules:

- Provider payload is stored 1:1.

- Delta refresh uses lastUpdatedAt.

- No thresholds are applied.

- No band calculation occurs here.

- RAW does not trigger SEO.

- RAW is not indexed.

- RAW is not directly visible in B2C.

4.5.3 A1 — Validated City Attribution

destinations[].primary is NOT sufficient for city attribution.

A product belongs to a city only if:

validated_start_city == city_id

Validated start city is determined by:

- logistics.start

- logistics.redemption

- logistics.travelerPickup

- pickup location mapping

- meeting point mapping

- internal mapping layer (engineering-provided)

Example:

If a product starts in Salzburg and visits Vienna,

it does NOT count toward Vienna RAW_count.

4.5.4 A1.1 RAW_count(city)

RAW_count(city) =

Number of RAW products where:

- status == ACTIVE

- validated_start_city == city_id

RAW_count measures market density.

It does NOT apply rating or review filters.

RAW_count determines city band classification.

4.5.5 A1.2 City Band Classification

Band A: 0–299

Band B: 300–599

Band C: 600+

Bands influence threshold matrix only.

4.5.6 A2 — POOL (Eligibility Layer)

POOL contains products eligible for selection.

Entry requirements:

- status == ACTIVE
- validated_start_city == city_id
- rating >= band + group threshold
- reviews >= band + group threshold
- strictest-category rule satisfied
- transport exclusions applied

Transport exclusions:

- sightseeing cruise excluded
- hop-on hop-off excluded

Automatic Behavior:

If rating drops below threshold:

→ product is automatically removed from POOL.

If new product meets thresholds:

→ automatically added to POOL.

POOL does NOT:

- trigger SEO batch
- generate near pages
- trigger keyword retrieval

4.5.7 A3 — SET (Curated Selection Layer)

SET is built from POOL.

SET includes products selected for:

- City page
- Things to do page
- Attraction page
- Curated portfolio

SET triggers:

- Keyword retrieval
- AI drafting
- Near pages generation
- FAQ generation
- Internal linking expansion

SET is stable.

If a product’s rating drops:

→ it is flagged

→ it is NOT automatically removed from SET

→ removal requires governance decision

4.5.8 A4 — Runtime Availability Fallback

If a SET product:

- has no availability for selected date
- does not support selected language
- fails runtime constraints

System behavior:

- selects substitute from POOL
- does not modify SET
- does not trigger SEO
- does not lower quality threshold

4.5.9 B2B Execution Layer

B2B uses:

- POOL
- B2B Expanded eligibility (rating ≥ 4.5 and reviews ≥ 100)
- All Rosotravel private tours

B2B does NOT:

- trigger SEO batch
- generate near pages
- affect RAW_count
- affect B2C SET

4.5.10 Language Clarification (Execution-Level)

Site language is controlled by hreflang and separate translated URLs.

Tour language is a runtime filter.

Language filtering logic (final selection logic defined in Phase C):

If two products compete:

- Product A rating 4.8, 2 languages
- Product B rating 4.6, 5 languages

Selection may favor broader language coverage.

Language logic does not create new URLs.

4.5.11 Ticket vs Tour-Like Classification

This is NOT Tour vs Activity. It is Ticket vs Tour-like.

Temporary guard rule:

If itinerary.itineraryItems count > 2

→ product is treated as Group 01 (tour-like).

Final classification logic is defined in Phase C.

4.5.12 Activities in Group 01

Outdoor Activities are part of Group 01.

No strong distinction between activity vs tour is required in Phase A.

Differentiation is deferred to Phase C (bundle logic, pacing logic, segment logic).

4.5.13 Delta Refresh Contract

If rating changes:

- Below threshold → automatic removal from POOL.

- Already in SET → flagged only.

If new product meets threshold:

→ automatically added to POOL.

If city mapping changes:

→ RAW_count recalculated

→ band recalculated

→ products flagged

→ SET remains stable unless manually reviewed.

4.5.14 SEO Trigger Summary

Triggers SEO:

- SET collections only.

Does NOT trigger SEO:

- RAW

- POOL

- B2B Expanded

- Runtime fallback substitutions.

4.6 RAW Ingestion & Entity Spawning: Create RAW_PRODUCT + Spawn RAW_CITY/RAW_ATTRACTION + Write Relationships

At this stage, the RAW object is defined conceptually according to feed assumptions and architectural goals. However, its final structure depends on completion of Stage 1 (Data Model Design) led by appinventive. **You should get from appinventive the final version of raw feed.**

A single product is not sourced from one endpoint only. It consists of:

- product-specific endpoints (core product data),
- and multiple global endpoints (e.g. tags, taxonomies, availability, reviews).

For example, the tags endpoint is global and products are linked to it via identifiers. The complete RAW feed will therefore be an aggregated structure composed of several coordinated endpoints.

The final list of fields, relationships, and RAW schema structure will be delivered after appinventive completes the full endpoint analysis and defines how the combined feed is assembled into a unified RAW object.

Implementation of ingestion mechanisms must follow that finalized specification.

c. Ingestion of RAW data for Cities, Products and Attractions

01. Change logic of current adding product at some points

Website Rosotravel MVP - featuer list 2025 y. here you can see screenshots and a feature list of current log in or you can log in to test environment and see what is the process of adding product on www.rosotravel.at (not www.rosotravel.com which is live website)

➢ Location:

- Select city from where tour starts: editor should add only City -> region -> country should be mapped automatically. Logic is same if city, region, country are not pre-uploaded it shows the error
- Type attraction which customer will visit during their trip (all attractions names in English) this has the same logic still is connected with google and create missing attraction if not exists

02. RAW Data Model Object

This section defines the **minimum complete RAW data model** required to ingest and maintain **CITY, ATTRACTION, PRODUCT** entities in a unified way (internal + external), including:

- deterministic **identity** (stable IDs and codes)
- deterministic **hierarchy & relationships** stored already in RAW (Country → Region → City → Attraction → Product)
- field-level **data source priority** (Viator vs Google vs Wikidata)
- **versioning** and **refresh cadence**, including high-frequency availability/pricing updates without RAW version explosion
- dedupe/merge rules for City and Attraction

Hard rule: Relationships and hierarchy MUST be present already at RAW stage. Downstream SEO/AI/publishing cannot “guess” hierarchy.

---

1) Ingestion Entry Points (how RAW creation is initiated)

1.1 Internal product creation (Rosotravel-owned)

Flow:

1. The Publication Team creates a Product in CMS (internal).
2. Team selects:
    - **City** (must exist as destination records; preuploaded)
    - It automatically makes a connection with Country and Region (if region exists)
    - **Attraction (optional)**:
        - choose existing Attraction, OR

        - create new Attraction via Google Places search (place_id) → RAW ATTRACTION is created if missing

3. System creates/updates RAW_PRODUCT_Version and ensures relationship edges:
    - Product → City
    - Product → Attraction
    - Attraction → City

Hard rule: Internal products MUST mint stable internal codes at creation time (see §3).

1.2 External product ingestion (Viator)

Flow:

1. System imports Product from Viator (productCode).
2. System also imports/updates:
    - City/Destination graph (destinationId, parentDestinationId)
    - Attraction objects when available (attractionId + destinations[])
3. System creates/updates RAW objects:
    - RAW_PRODUCT_Version (external_id = viator productCode)
    - RAW_CITY_Version (external_id = viator destinationId for type=city)
    - RAW_ATTRACTION_Version (external_id = viator attractionId)
4. System resolves and stores parent_relations in each RAW object:
    - Country → Region → City (from destination graph)
    - Attraction → City (from attraction.destinations primary)
    - Product → City (from product.destinations)
    - Product → Attraction (explicit link if possible; otherwise deterministic match + “needs_review” if ambiguous)

---

2) Data Sources & Priority (binding)

2.1 Source tiers (unchanged)

Tier 1 — Google Places API (Cities & Attractions only)
Used for: official name, address, coordinates, categories, phone, official website, opening/holiday hours.

Tier 2 — Wikidata API (Cities & Attractions primarily)
Used for: historical facts, classification (“instance of”), aliases, parent relations (“isPartOf”), external IDs, official languages.

Tier 3 — Wikipedia (Safe Extract Mode, optional)
Used only if explicitly allowed: neutral short summary / historical context only.
Never used for operational data (hours, prices, ticketing, schedules).

External operational source — Viator (Products primarily, plus minimal city/attraction references)
Used for: product commercial content + offer/booking structure.

2.2 Field-level priority rules (binding)

Field	Source Order
City/Attraction canonical name	Google → Wikidata
Opening hours	Google only (if available)
Coordinates	Google → Wikidata
Address	Google only
Website	Google → Wikidata
Historical facts	Wikidata → Wikipedia (if allowed)

Parent entity relations	Viator destination graph → Wikidata (fallback)
Product title/description	ViatorUniqueContent (preferred) → Viator product.description (fallback)
Reviews & UGC	Stored RAW only (never indexable)

---

3) Definitions (binding)

Required = Yes
Field must be present to accept RAW entity and proceed.

Required = No
Field is ingested if available, but missing value does not block ingestion (warning only).

Derived
Field is computed deterministically by the system (from relationships, normalization, or rules). Not copied 1:1 from one payload.

---

4) Identity, IDs & White-Label Stable Codes (Hard rules)

4.1 Mandatory identifiers (all entities)

Each entity MUST have:

- **entity_id** (internal immutable ID; recommended ULID/UUID)
- **entity_type** = CITY | ATTRACTION | PRODUCT
- **source_type** = INTERNAL | EXTERNAL
- **external_id** (nullable for INTERNAL; required for EXTERNAL when available)
- **stable_code** (mandatory for BOTH internal and external, used for white-label exports)

4.2 stable_code format (recommended)

Stable code must be:

- immutable
- not derived solely from name (names can change)
- export-safe for white-label

Recommended:

- CITY: `CTY_<ULID>`
- ATTRACTION: `POI_<ULID>`
- PRODUCT: `PRD_<ULID>`
- LINK (product-attraction association): `LNK_<ULID>`

Hard rule: Internal products MUST have stable_code minted at creation time (not later).

---

5) Relationship & Hierarchy

5.1 Canonical hierarchy model stored in RAW

Even if Region/Country are not ingested as standalone RAW objects, their **references** MUST exist in `parent_relations`.

Canonical chain:
Country → Region → City → Attraction → Product

5.2 Relationship fields (mandatory on all three entities)

All three RAW objects MUST include a relationship block:

- parent_relations (required)
- relationship_edges[] (required)
- relationship_hash (required)

5.3 relationship_edges[] (standard edge types)

Edge types (minimum):

- `CITY_IN_REGION`
- `REGION_IN_COUNTRY`
- `ATTRACTION_IN_CITY`
- `PRODUCT_IN_CITY`
- `PRODUCT_VISITS_ATTRACTION` (optional but recommended)
- `PRODUCT_RELATED_ATTRACTION` (optional secondary relations; can exist but must not change canonical chain)

Each edge record MUST include:

- edge_id (ULID/UUID)
- edge_type
- from_entity_id + from_stable_code
- to_entity_id + to_stable_code (or reference node id for region/country)
- source (VIATOR | GOOGLE | WIKIDATA | CMS | SYSTEM)
- confidence (HIGH | MEDIUM | LOW)
- created_at / updated_at

5.4 Viator hierarchy usage (hard requirement)

- CITY parent chain is built from Viator `destinationId` + `parentDestinationId` traversal.
- ATTRACTION must map to CITY via Viator attraction.destinations primary (or deterministic fallback).
- PRODUCT must map to CITY via product.destinations (or deterministic fallback).

If conflict occurs (e.g., Google says different city than Viator):

- keep both in raw_payload

- choose canonical parent_relations using deterministic priority:
    1. Viator destination graph for “destination hierarchy”
    2. Google address components for “geo place truth”
    3. Wikidata isPartOf fallback
- set `parent_relations.confidence` accordingly and raise a warning flag.

---

6) Unified Versioning & Refresh Cadence (prevents version explosion)

6.1 Version domains (must exist)

The system MUST track separate freshness/version domains (independent counters):

1. **raw_version**
   Slow-changing snapshot for:

- content seeds
- factual snapshot
- media snapshot
- relationship snapshot

2. **offer_version** (PRODUCT only)
   Medium cadence booking offer snapshot for:

- options structure
- included/excluded
- cancellation policy
- meeting point structure
- confirmation rules

3. **realtime_offer_version** (PRODUCT only)
   High cadence runtime snapshot for:

- pricing signals

- availability/calendar refs
- start times (if provided)
  This domain may refresh every 15–30 minutes.

6.2 Increment rules (binding)

raw_version increments ONLY when ANY of these change (by hash comparison):

- `content_hash` changed
- `factual_hash` changed
- `media_hash` changed
- `relationship_hash` changed

offer_version increments ONLY when:

- `offer_hash` changed

realtime_offer_version increments ONLY when:

- `realtime_offer_hash` changed

Hard rule (non-negotiable):
`realtime_offer_version` MUST NOT force `raw_version` increments.

6.3 Hash domains (single source of truth)

Hashes MUST exist and MUST be computed on normalized, deterministic JSON (stable ordering + canonical serialization).

Core hashes (CITY, ATTRACTION, PRODUCT):

- **content_hash** = hash(content_fields_seed only)
- **factual_hash** = hash(factual_fields only)
- **media_hash** = hash(media_fields only)
- **relationship_hash** = hash(relationship_fields only)

Offer hashes (PRODUCT only):

- **offer_hash** = hash(operational_fields / offer snapshot only)
- **realtime_offer_hash** = hash(realtime_offer_fields only)

Hard rule (prevents duplicate logic):

- `relationship_fields` MUST NOT be included in `factual_hash`.
  Relationship changes are tracked only via `relationship_hash`.

6.4 Entity → applicable hashes matrix (hard enforcement)

Entity type	Required hashes
CITY	content_hash, factual_hash, media_hash, relationship_hash
ATTRACTION	content_hash, factual_hash, media_hash, relationship_hash
PRODUCT	content_hash, factual_hash, media_hash, relationship_hash, offer_hash, realtime_offer_hash

6.5 Where hashes live (storage rule)

- `content_hash`, `factual_hash`, `media_hash`, `relationship_hash` are stored alongside the RAW snapshot and participate in `raw_version`.
- `offer_hash` is stored alongside the offer snapshot and participates in `offer_version`.
- `realtime_offer_hash` is stored alongside realtime cache/snapshot and participates in `realtime_offer_version`.

This separation guarantees:

- no RAW version explosion from realtime changes
- deterministic diffing per domain

- predictable pipeline triggers

Refresh Policy Keys (CONFIG)

Policy Key	Source	TTL / Cadence	Paid-cost sensitivity	Notes (hard)
ONCE_ID	SYSTEM	once	none	IDs/keys are set once; re-fetch only if missing
GP_HOURS_7D	GOOGLE PLACES	7 days	high	Opening + holiday/special hours; must be fresh
GP_CONTACT_60D	GOOGLE PLACES	60 days	high	phone, website; keep last-known-good on outage
GP_IDENTITY_90D	GOOGLE PLACES	90 days	high	official name, address; low volatility
GP_GEO_365D	GOOGLE PLACES	365 days	high	coordinates; refresh only if missing/invalid

GP_CATEGORIES_180D	GOOGLE PLACES	180 days	high	place types/categories (taxonomy only)
WD_CORE_365D	WIKIDATA	365 days	low	instance_of, isPartOf, external IDs, core relations
WD_ALIASES_180D	WIKIDATA	180 days	low	aliases/labels
OTA_PRODUCT_CONTENT_90D	EXTERNAL API (OTA)	90 days	medium	title/desc/highlights seeds; avoid polling content too often
OTA_OFFER_30D	EXTERNAL API (OTA)	Scope API determine	No have	included/excluded, cancellation, meeting point, options
OTA_STABLE_180D	EXTERNAL API (OTA)	Scope API determine	No have	duration, languages (unless supplier is known volatile)
OTA_MEDIA_30D	EXTERNAL API (OTA)	Scope API	No have	product images/videos can rotate

		determine		
OTA_RELATIONS_180D	EXTERNAL API (OTA)	Scope API determine	No have	destination graph IDs & parent links rarely change
REALTIME_15M	EXTERNAL API (OTA)	Scope API determine	No have	availability/price cache; MUST NOT bump raw_version

Hard rule (binding): **REALTIME_15M never increments raw_version and never affects content_hash/factual_hash/media_hash.**

RAW OBJECTS (FINAL TABLES)

7.1 RAW_CITY_Version (versioned snapshot)

A) Core identity & snapshot meta

Field	Source	Refresh Policy	Required	Derived	Notes
entity_id	SYSTEM	ONCE_ID	Yes	Yes	Internal immutable ID
stable_code	SYSTEM	ONCE_ID	Yes	Yes	CTY_<ULID>
entity_type	SYSTEM	ONCE_ID	Yes	Yes	CITY
source_type	SYSTEM	ONCE_ID	Yes	Yes	INTERNAL/EXTERNAL
external_id (viator_destination_id)	EXTERNAL API (OTA)	ONCE_ID	No*	No	Required if source_type=EXTERNAL

google_place_id	GOOGLE	ONCE_ID	No	No	Identity key if available
wikidata_id (QID)	WIKIDATA	ONCE_ID	No	No	Identity key if available
raw_version	SYSTEM	on change	Yes	Yes	Slow snapshot version
raw_ingested_at	SYSTEM	on ingest	Yes	Yes	Timestamp

* “Required if EXTERNAL” = hard validation per source_type.

B) content_fields (seed only; not indexing rules here)

Field	Source Order	Refresh Policy	Required	Derived	Notes
name_canonical	GOOGLE → WIKIDATA	GP_IDENTITY_90D	Yes	No	Canonical display name
name_aliases[]	WIKIDATA	WD_ALIASES_180D	No	No	Alternate names
short_description_seed	WIKIDATA → (Wikipedia optional)	WD_CORE_365D	No	No	Seed only; can be empty initially

C) factual_fields

Field	Source Order	Refresh Policy	Required	Derived	Notes

coordinates {lat,lng}	GOOGLE → WIKIDATA	GP_GEO_365D	Yes	No	Mandatory for geo linking
address_structured	GOOGLE	GP_IDENTITY_90D	No	Yes	Normalized from components
time_zone	EXTERNAL API (OTA) → GOOGLE	OTA_RELATIONS_180D	No	No	Prefer OTA if consistent
iata_codes[]	EXTERNAL API (OTA)	OTA_RELATIONS_180D	No	No	Optional
categories[]	GOOGLE → WIKIDATA	GP_CATEGORIES_180D	No	No	Place categories
official_website	GOOGLE → WIKIDATA	GP_CONTACT_60D	No	No	Optional

D) relationship_fields (MUST exist)

Field	Source	Refresh Policy	Required	Derived	Notes
parent_relations.country	OTA graph →	OTA_RELATIONS_180D	Yes	Yes	Must exist as reference

	Wikidata				
parent_relations.region	OTA graph → Wikidata	OTA_RELATIONS_180D	No	Yes	Optional if not available
parent_relations.city (self)	SYSTEM	ONCE_ID	Yes	Yes	Self pointer
relationship_edges[]	SYSTEM	Derived on compute	Yes	Yes	CITY_IN_REGION + REGION_IN_COUNTRY
relationship_hash	SYSTEM	on change	Yes	Yes	Hash over relationship_fields

E) media_fields

Field	Source Order	Refresh Policy	Required	Derived	Notes
images[]	GOOGLE → WIKIDATA → Wikipedia(opt)	GP_CATEGORIES_180D	No	No	References only; avoid frequent paid pulls
videos[]	GOOGLE (if any)	GP_CATEGORIES_180D	No	No	Optional

F) hashes

Field	Source	Required	Notes
content_hash	SYSTEM	Yes	Hash of City content_fields only (name + aliases + desc seed)
factual_hash	SYSTEM	Yes	Hash of factual_fields only
media_hash	SYSTEM	Yes	Hash of media_fields
relationship_hash	SYSTEM	Yes	Hash of relationship_fields

7.2 RAW_ATTRACTION_Version (versioned snapshot)

A) Core identity & snapshot meta

Field	Source	Refresh Policy	Required	Derived	Notes
entity_id	SYSTEM	ONCE_ID	Yes	Yes	Internal immutable ID
stable_code	SYSTEM	ONCE_ID	Yes	Yes	POI_<ULID>
entity_type	SYSTEM	ONCE_ID	Yes	Yes	ATTRACTION

source_type	SYSTEM	ONCE_ID	Yes	Yes	INTERNAL/EXTERNAL
external_id (viator_attraction_id)	EXTERNAL API (OTA)	ONCE_ID	No*	No	Required if EXTERNAL and provided
google_place_id	GOOGLE	ONCE_ID	No	No	Created via Google for internal creation
wikidata_id (QID)	WIKIDATA	ONCE_ID	No	No	Alternative identity key
raw_version	SYSTEM	on change	Yes	Yes	Slow snapshot version
raw_ingested_at	SYSTEM	on ingest	Yes	Yes	Timestamp

* One-of identity rule (hard): attraction must have at least one of (viator_attraction_id OR google_place_id OR wikidata_id).

B) content_fields (seed only)

Field	Source Order	Refresh Policy	Required	Derived	Notes
name_canonical	GOOGLE → WIKIDATA	GP_IDENTITY_90D	Yes	No	
introduction_seed	OTA unique content	OTA_PRODUCT_CONTENT_90D	No	No	RAW only; indexing

	→ Wikidata				handled elsewhere
overview_sections_seed[]	OTA unique content → Wikidata	OTA_PRODUCT_CONTENT_90D	No	No	RAW only

C) factual_fields

Field	Source Order	Refresh Policy	Required	Derived	Notes
coordinates {lat,lng}	GOOGLE → WIKIDATA	GP_GEO_365D	Yes	No	
address_structured	GOOGLE	GP_IDENTITY_90D	No	Yes	
opening_hours	GOOGLE only	GP_HOURS_7D	No	No	Optional if unavailable
official_website	GOOGLE → WIKIDATA	GP_CONTACT_60D	No	No	
phone	GOOGLE	GP_CONTACT_60D	No	No	

categories[]	GOOGLE → WIKIDATA	GP_CATEGORIES_180D	No	No	
sameAs[]	WIKIDATA	WD_CORE_365D	No	No	External IDs/links

D) relationship_fields (MUST exist)

Field	Source	Refresh Policy	Required	Derived	Notes
parent_relations.city	OTA destinations → destination_map OR GOOGLE address	OTA_RELATIONS_180D	Yes	Yes	Canonical city required
parent_relations.region	from City chain	Derived on compute	No	Yes	Optional
parent_relations.country	from City chain	Derived on compute	Yes	Yes	Derived via city parent
parent_relations.attraction (self)	SYSTEM	ONCE_ID	Yes	Yes	Self pointer
relationship_edges[]	SYSTEM	Derived on compute	Yes	Yes	ATTRACTION_IN_CITY + inherited city chain
relationship_hash	SYSTEM	on change	Yes	Yes	

E) media_fields

Field	Source	Refresh Policy	Required	Notes
images[]	OTA → GOOGLE → WIKIDATA	OTA_MEDIA_30D (OTA part) + GP_CATEGORIES_180D (Google part)	No	Keep source attribution
videos[]	OTA (if any)	OTA_MEDIA_30D	No	Optional

F) hashes

Field	Required	Notes
content_hash	Yes	content_fields
factual_hash	Yes	factual_fields
media_hash	Yes	media_fields
relationship_hash	Yes	relationship_fields

7.3 RAW_PRODUCT_Version (versioned snapshot + offer + realtime)

A) Core identity & snapshot meta

Field	Source	Refresh Policy	Required	Derived	Notes

entity_id	SYSTEM	ONCE_ID	Yes	Yes	Internal immutable ID
stable_code	SYSTEM	ONCE_ID	Yes	Yes	PRD_<ULID>
entity_type	SYSTEM	ONCE_ID	Yes	Yes	PRODUCT
source_type	SYSTEM	ONCE_ID	Yes	Yes	INTERNAL/EXTERNAL
external_id (viator_productCode)	EXTERNAL API (OTA)	ONCE_ID	No*	No	Required if EXTERNAL
raw_version	SYSTEM	on change	Yes	Yes	Slow snapshot version (content/media/relations changes)
offer_version	SYSTEM	on change	Yes	Yes	Medium cadence offer snapshot (operational offer)
realtime_offer_version	SYSTEM	frequent	Yes	Yes	High cadence pricing/availability
raw_ingested_at	SYSTEM	on ingest	Yes	Yes	Timestamp

* Required if source_type=EXTERNAL.

B) content_fields (seed)

Field	Source Order	Refresh Policy	Required	Derived	Notes
title_seed	OTA unique content → OTA title	OTA_PRODUCT_CONTENT_90D	Yes	No	INTERNAL: CMS title_seed required
short_description_seed	OTA unique content → OTA description	OTA_PRODUCT_CONTENT_90D	No	No	Optional seed
long_description_seed	OTA unique content → OTA description	OTA_PRODUCT_CONTENT_90D	No	No	Stored RAW; indexing handled elsewhere
highlights_seed[]	OTA tags/desc → SYSTEM extract	OTA_PRODUCT_CONTENT_90D	No	Yes	Extract bullets seed if available
itinerary_overview_seed	OTA itinerary → SYSTEM summary seed	OTA_PRODUCT_CONTENT_90D	No	Yes	Seed only

C) operational_fields (offer snapshot; medium cadence)

Field	Source	Refresh Policy	Required	Derived	Notes
duration	EXTERNAL API (OTA)	OTA_STABLE_180D	Yes	No	
languages[]	EXTERNAL API (OTA)	OTA_STABLE_180D	No	No	
meeting_point	OTA logistics → (optional Google verify)	OTA_OFFER_30D	No	No	Verify only if needed (do not poll Google)
cancellation_policy	EXTERNAL API (OTA)	OTA_OFFER_30D	No	No	
included[]	EXTERNAL API (OTA)	OTA_OFFER_30D	No	No	Optional; warn if missing
excluded[]	EXTERNAL API (OTA)	OTA_OFFER_30D	No	No	Optional; warn if missing
product_options[]	EXTERNAL API (OTA)	OTA_OFFER_30D	No	No	Offer structure
confirmation_type	EXTERNAL API (OTA)	OTA_OFFER_30D	No	No	

D) realtime_offer_fields (high cadence; runtime cache)

Field	Source	Refresh Policy	Required	Derived	Notes
price_from	OTA pricing	REALTIME_15M	Yes	No	Must not bump raw_version
currency	OTA pricing	REALTIME_15M	Yes	No	
availability_refs	OTA availability	REALTIME_15M	Yes	No	Calendar endpoint refs
start_times[]	OTA availability	REALTIME_15M	No	No	Optional; not always explicit

E) media_fields

Field	Source	Refresh Policy	Required	Notes
images[]	EXTERNAL API (OTA)	OTA_MEDIA_30D	No	
videos[]	EXTERNAL API (OTA)	OTA_MEDIA_30D	No	

F) relationship_fields (MUST exist; includes internal white-label links)

Field	Source	Refresh Policy	Required	Derived	Notes

parent_relations.city	INTERNAL: CMS selection / EXTERNAL: OTA destinations	OTA_RELATIONS_180D	Yes	Yes	Canonical city required
parent_relations.region	from City chain	Derived on compute	No	Yes	
parent_relations.country	from City chain	Derived on compute	Yes	Yes	
parent_relations.attraction	INTERNAL: CMS/Google selection / EXTERNAL: link if resolvable	OTA_RELATIONS_180D	No	Yes	Optional but recommended
relationship_edges[]	SYSTEM	Derived on compute	Yes	Yes	PRODUCT_IN_CITY + optional PRODUCT_VISITS_ATTRACTION
product_attraction_link_code	SYSTEM	ONCE_ID	No	Yes	LNK_<ULID> created when link exists

relationship_hash	SYSTEM	on change	Yes	Yes	Hash over relationship_fields

G) hashes

Field	Required	Notes
content_hash	Yes	content_fields only (NO realtime signals)
offer_hash	Yes	operational_fields only
realtime_offer_hash	Yes	realtime_offer_fields only (NO raw_version bump)
media_hash	Yes	media_fields
relationship_hash	Yes	relationship_fields

---

8) Dedupe / Merge Rules (CITY & ATTRACTION)

8.1 City dedupe keys (priority)

Priority	Key	Source	Rule
1	viator_destination_id	VIATOR	If matches → same CITY
2	google_place_id	GOOGLE	If matches → same CITY

3	wikidata_id (QID)	WIKIDATA	If matches → same CITY
4	(name + coordinates within radius)	SYSTEM	Only if none above exist; requires manual review if collision

Hard rule: If two keys conflict (e.g., same google_place_id but different viator_destination_id), keep one canonical City and store the other as alias reference; raise conflict flag.

8.2 Attraction dedupe keys (priority)

Priority	Key	Source	Rule
1	viator_attraction_id	VIATOR	If matches → same ATTRACTION
2	google_place_id	GOOGLE	If matches → same ATTRACTION
3	wikidata_id (QID)	WIKIDATA	If matches → same ATTRACTION
4	(name + coordinates within radius + same city)	SYSTEM	Only if none above exist; requires review

Hard rule: Attraction must always have canonical City relationship; merge cannot produce attraction without city.

8.3 Merge precedence (field winners) — RAW level

This is NOT publishing logic; it only unifies RAW entity identity.

Field group	Winner precedence
name/address/geo	GOOGLE first
hierarchy destination chain	VIATOR first
historical facts	WIKIDATA first
external IDs	WIKIDATA first
product commercial content	VIATOR first
offer/realtime offer	VIATOR first

---

9) Hard Requirements Checklist (developer enforcement)

1. Every CITY/ATTRACTION/PRODUCT must have:

- entity_id, stable_code, raw_version, raw_ingested_at
- relationship_fields + relationship_edges[] + relationship_hash
- hashes required by entity type (see 6.4 matrix)

2. Every CITY/ATTRACTION/PRODUCT must store:

- parent_relations (including country reference)
- relationship_edges[]
- relationship_hash

3. PRODUCT must always have:

- offer_version + offer_hash
- realtime_offer_version + realtime_offer_hash
- hard separation: realtime never increments raw_versi

4. INTERNAL products must mint stable codes immediately:

- product stable_code PRD_<ULID>
- attraction stable_code POI_<ULID> (if created)
- link_code LNK_<ULID> if product↔attraction exists
  This enables deterministic white-label exports.

5. Viator hierarchy must be stored:

- destinationId and parentDestinationId chain snapshot (via parent_relations + edges)

A.10.1 Hierarchical Entity Model

All entities MUST exist within a single, deterministic hierarchy:

Country → Region → City → Attraction → Product → Near-Page

Rules:

- Each entity has **exactly one parent** (except Country).
- No entity may skip hierarchy levels.
- Near-pages are always terminal nodes (no children).
- Products MUST always belong to exactly one Attraction or City (never directly to Region or Country).
- An entity may not be published unless its parent entity is published (except Countries).

This hierarchy is the **single source of truth** for:

- internal linking
- canonical logic

- sitemap eligibility
- LLM context windows
- topic graph construction

A.10.2 Horizontal Relationships (Non-Hierarchical)

In addition to the strict hierarchy, the system supports **controlled horizontal relations** for discovery, UX, and AI context enrichment.

Allowed horizontal relationships:

- **Attraction ↔ Nearby Attractions**
  Geo-based relation derived from coordinates and distance thresholds.
- **Attraction ↔ Related Tours (Products)**
  Based on explicit attraction-product linkage and intent compatibility.
- **City ↔ Experience Categories**
  Categories act as thematic aggregators (e.g. “Museums”, “Food Tours”).
- **Category ↔ Tours**
  A tour may belong to multiple categories, but category pages never own transactional keywords.

Rules:

- Horizontal relations MUST NOT create new canonical paths.
- Horizontal relations MUST NOT override hierarchy-based ownership.
- Horizontal relations are advisory and contextual only.

4.7 Unified Version Domains + Hash Naming (raw_version / offer_version / realtime_offer_version + all hashes)

Tu ma być: reguły bumpów + “no version explosion” + standard naming:

G.1 Core principle (single shared version number)

The system uses one shared version number (N) representing the current source snapshot state (RAW vN).
AI and Published states are tracked against the same RAW version N, without creating additional version numbers.

Hard rule: Versioning represents source snapshots, not editorial work.

G.2 RAW Version (raw_version = N) — when does N increment?

`raw_version` increments **ONLY** when a new snapshot causes a change in at least one of:

- `content_hash`, or
- `factual_hash`, or
- `media_hash`.

G.2.1 RAW version increase triggers (RAW only)

RAW vN++ happens when:

1. **External API (OTA) snapshot change** affects normalized fields that impact `content_fields`, `factual_fields`, or `media_fields`
   → raw_version++
2. **Internal product snapshot change** affects the above normalized fields
   → raw_version++
3. **Google Places factual refresh** changes factual_fields (name/address/hours/coords/website/phone/categories etc.)
   → raw_version++ (because `factual_hash` changes)
4. **Wikidata factual refresh** changes factual_fields (aliases, parent relations, classification, external IDs, historical facts stored as factual)
   → raw_version++ (because `factual_hash` changes)
5. **Media refresh** changes images/videos refs or ordering
   → raw_version++ (because `media_hash` changes)

G.2.2 Explicitly NOT version triggers

These actions do **NOT** increment raw_version:

- Manual SEO edits in CMS (meta, text, highlights edits, etc.)
- AI regeneration / paraphrase
- Publishing / unpublishing
- Review/approval workflow actions
- Real-time offer refresh (availability updates, short-term pricing signals)

Reason: those are editorial/processing/runtime operations, not a new source snapshot for RAW vN.

Stamp: **content_hash / factual_hash / media_hash / relationship_hash / offer_hash / realtime_offer_hash written in 4.6 under each table.**

4.8 Portfolio choosing / product classification

4.8.0 Purpose

This section is the selection “heart” of Model C. It defines how Rosotravel deterministically chooses its curated portfolio and classifies products before any batching.

4.8 defines, in operational detail

- how products enter **RAW_ALL_PRODUCTS** (provider mirror)
- how **Pool eligibility** is computed (auto add/remove)
- how **Set** is built (stable curated portfolio)
- how products are classified (**TICKET / PASS / TOUR**)
- how **Archetype Core vs Archetype Variant** works (competition vs differentiation)
- how winners are selected (within archetype core; language advantage always-on)
- how **City / Things-to-do / Attraction Sets** are constructed with hard anti-duplicate rules
- how bundles are built (Anchor + Complement + Flex; time geometry; difficulty)
- how Phase B “More Options” uses broader pools without triggering batch

- which collections trigger SEO/AI batch and which never do
- how single-product processing is triggered when a product enters Set after batch completion

Non-goals

- UI screens and editable parameter tables (Chapter 3 Product Ops Settings).
- Workflow execution steps and queue orchestration (Chapter 4 workflow and Chapter 4.9+).

4.8.1 Definitions and Data Boundaries (Read/Write, Version Domains)

Definition Block: Product Entity

A product is a single entity keyed by provider stable id (e.g., Viator productCode). Products must never be duplicated into separate records for RAW/Pool/Set; those are memberships/flags.

Definition Block: raw_version (provider snapshot)

raw_version is the provider payload snapshot for the product (refreshed from API). raw_hash is computed in 4.7.

Definition Block: offer_version (curated/publishable layer)

offer_version is Rosotravel's transformed/publishable representation (AI layer + governance locks), hashed in 4.7.

Definition Block: realtime_offer_version (runtime commerce layer)

realtime_offer_version represents runtime availability/pricing constraints (not editorial), hashed in 4.7.

4.8 reads

- raw_version + raw_hash
- validated start city attribution (from 4.5 mapping rules)
- quality thresholds and weight multipliers (from Chapter 3 Product Ops Settings)

- manual flags and overrides (from Product Ops operations)

4.8 writes (selection outputs)

- membership flags: pool_member, set_member, blocked, pinned, flagged_reason[]
- classification: TICKET / PASS / TOUR (locked logic)
- category group: 01 / 02 / 03 (locked mapping)
- archetype_core_id and archetype_variant_id (deterministic)
- per-core winners: default_winner, best_value_winner, premium_winner, private_upgrade_winner
- portfolio_version (per city) and portfolio_hash(city_id) as stable boundary for 4.9 batch input
- selection audit artifacts (why selected / why excluded) for debugging and trust

Important: Selection outputs are a stable boundary for batching.
4.9 must consume portfolio_hash(city_id) + Set membership lists as stable inputs.

4.8.2 Hard System Locks (must never be editable)

A) Inventory architecture

- RAW → Pool (auto) → Set (stable)
- Pool membership recalculates automatically (auto add/remove)
- Set membership is stable: threshold drops only FLAG Set items; no auto removal

B) Trust gates

- Set requires bookingConfirmationSettings.confirmationType = INSTANT (no exceptions)
- MANUAL products are never allowed in the main stack (City/Things/Attraction core). They may appear only in Phase B “More Options”.

C) Core decision model

- 4-layer model: Audience / Intent / Constraints / Value Tier
- Max 2 active intents at a time
- hidden_gems is separate and must not be merged with active/adventure

D) Archetype rules

- Minimum candidates per Archetype Core = 4 (global)
- Merge order locked: duration → pickup/meeting → POI (city-level only)
- POI merge forbidden on attraction pages

E) Anti-duplicate rules

- R1–R6 are enforced at Set construction time (no product twice per page; Top Tickets ticket-only; rails vs top picks; “2-of-many” must be materially different; etc.)

F) Winner swap policy

- If product is already in Set, algorithm changes do not silently replace it; they FLAG only. Manual review required.

4.8.3 End-to-End Selection Sequence (overview)

This section describes the full selection pipeline in the exact order it must run:

- Stage A — RAW ingest and validated city attribution
- Stage B — city banding (RAW_count)
- Stage C — Pool eligibility (quality gates by band + group)
- Stage D — product classification (TICKET/PASS/TOUR) and INSTANT gate for Set
- Stage E — archetype assignment (Core vs Variant; merge rules)

- Stage F — profile model (Audience/Intent/Constraints) and eligibility states
- Stage G — within-core winner selection (BQS + language advantage always-on + fit tie-breaks)
- Stage H — build Sets per page type (City/Things/Attraction) with R1–R6 enforcement
- Stage I — build bundles (Anchor/Complement/Flex + difficulty + time geometry)
- Stage J — Phase B pool usage rules (no batch)
- Stage K — outputs (portfolio_version/hash) → handoff to 4.9

4.8.4 Stage A — RAW_ALL_PRODUCTS (how products arrive)

A1 Ingest

All provider products are ingested into RAW_ALL_PRODUCTS as raw_version snapshots (see 4.7). Provider field names remain unchanged.

A2 Validated Start City Attribution (reference 4.5)

Each product is attributed to a validated_start_city using logistics-based mapping (meeting/pickup logic), not provider primary destination alone.

A3 RAW_count(city) (banding input)

RAW_count(city) counts RAW products after validated attribution:

- status active
- validated_start_city == city_id

RAW_count measures market density and is used for banding only.

4.8.5 Stage B — City Banding

Band assignment uses RAW_count(city):

- Band A: 0–299

- Band B: 300–599
- Band C: 600+

Band affects which threshold rows are used in Stage C and affects some exposure counts in Phase B.
Band does not affect URLs, canonical, or indexing.

4.8.6 Stage C — Pool Eligibility (quality gates)

Definition Block: Category Groups 01/02/03

Group 01 (broad tours pool):
Art & Culture, Food & Drink, Outdoor Activities, Seasonal & Special Occasions, Tours/Sightseeing/Cruises

Group 02 (tickets/workshops):
Classes & Workshops, Tickets & Passes

Group 03 (transport):
Travel & Transportation Services

C1 Group assignment

Each product is mapped to exactly one primary group for threshold selection. If multiple groups apply, the strictest-category rule applies (treated as Group 01).

C2 Threshold application (parameterized; values live in Chapter 3)

Pool eligibility checks:

- min_rating by band+group
- min_reviews by band+group
- exclusions policy (Group 03 transport exclusions)
- structural minimums (rating and reviews must exist)

C3 Pool membership behavior (hard)

- If eligibility drops → product is automatically removed from Pool

- If eligibility rises (new product or improved stats) → product is automatically added to Pool

- Pool changes do not automatically alter Set. Set is flagged-only.

4.8.7 Stage D — Deterministic Classification + INSTANT Gate for Set

Stage D is about “what the product is” and whether it is safe for the main stack.

4.8.7.1 Definition Block: Classification output

classification ∈ { TICKET, PASS, TOUR }

4.8.7.2 Definition Block: Guide Mode normalization

Guide information may exist at:

- productOptions[].languageGuides[]
  and/or

- languageGuides[] at product level

Engineering must normalize both into an effective guide_mode set for classification.

4.8.7.3 Classification rules (locked; provider-fields only)

Rule D1 — Guided implies TOUR
If any normalized languageGuides.type == GUIDE → classification = TOUR

Rule D2 — Itinerary density implies TOUR
If itinerary.itineraryItems.length > 2 → classification = TOUR
Exception handled by Rule D3.

Rule D3 — PASS exception (multi-attraction pass)
If:

- guide_mode does not include GUIDE, AND

- ticketInfo exists, AND

- product represents a pass/city card/multi-attraction entitlement,
  THEN classification = PASS.

Note: “pass/city card” detection uses provider fields and/or an engineering-owned allowlist mapping finalized during discovery. If detection is UNKNOWN, treat as TOUR to avoid Top Tickets contamination.

Rule D4 — Ticket-only
If guide_mode does not include GUIDE AND itinerary.itineraryItems.length ≤ 2
→ classification = TICKET

4.8.7.4 Definition Block: INSTANT gate for Set

Set eligibility requires:
bookingConfirmationSettings.confirmationType == INSTANT

If confirmationType == MANUAL:

- product can remain in RAW and Pool (if it meets quality gates)
- product is not eligible for Set
- product can appear only in Phase B “More Options” contexts
- product must not trigger batch

ConfirmationType eligibility matrix

confirmationType	Allowed in RAW	Allowed in Pool (if quality passes)	Allowed in Set (main stack)	Allowed in Phase B “More Options”	Triggers SEO/AI batch
INSTANT	Yes	Yes	Yes	Yes	Yes (via Set only)
MANUAL	Yes	Yes	No	Yes	No

4.8.8 Stage E — Archetype engine (Core vs Variant; competition integrity)

Stage E prevents “global ranking chaos” by enforcing competition within meaningful groups.

4.8.8.1 Definition Block: Archetype Core (competition bucket)

Archetype Core defines where competition happens. Products only compete within the same Archetype Core.

Core ID must be built from deterministic fields (no title heuristics):
CORE_ID = {group_family} + {experience_mode} + {poi_cluster} + {format_core} + {duration_bucket}

Components

- group_family: 01/02/03
- experience_mode:
    - Group01 + classification TOUR → tour_like
    - Group02 + classification TICKET/PASS → ticket_like
    - Group03 → transfer_like
- poi_cluster:
    - attractionId cluster if available (from itinerary items)
    - otherwise UNKNOWN_POI (city-level only)
- format_core:
    - private/shared from itinerary.privateTour
    - pickup/meeting from logistics.travelerPickup.pickupOptionType, else unknown_pickup
- duration_bucket:
    - derived from itinerary.duration.* if available, else unknown_duration

4.8.8.2 Definition Block: Archetype Variant (dedup/display differentiation)

Variant is used to distinguish materially different formats without splitting competition.

Variant may include only provider-field signals, for example:

- pickupOptionType
- itinerary.skipTheLine (if present)
- guide_mode (GUIDE vs AUDIO/WRITTEN)
- confirmationType (INSTANT/MANUAL)

Additional variant signals are discovery-defined (Tag Mapping Registry).

4.8.8.3 Minimum candidates per Core (hard)

Each Core must have at least 4 Pool candidates.

4.8.8.4 Merge order (locked)

If Core candidates < 4:

1. merge duration_bucket away
2. merge pickup/meeting into unknown_pickup
3. merge POI into UNKNOWN_POI (city-level only; forbidden on attraction pages)

If still < 4:

- mark Core as sparse and do not force it into rails; it remains accessible via Phase B.

4.8.9 Stage F — Profile model (Audience/Intent/Constraints + eligibility states)

4.8.9 Core Classification Layer — Final Tag Sets

This section defines the final classification layer used by the Rosotravel Decision Platform.

This is one of the most important functional foundations of the system. These tags define how the platform understands the traveler, interprets intent, applies constraints, evaluates product fit, and then selects, ranks, and presents products, bundles, rails, dynamic snippets, trust elements, and future CRM / retargeting logic.

The classification layer is divided into four separate dimensions:

Audience — who the traveler is / what type of traveler they are
Intent — what the traveler is trying to optimize for right now
Constraints — what the system must respect, avoid, or filter by
Eligibility state — how well a product fits a specific audience / intent / constraint profile

These dimensions must not be merged into one flat tagging system.

4.8.9.1 Definition Block: Audience tags

Audience describes the dominant traveler type or travel style.

```
audience ∈ {
 first_time_visitor,
 family_traveler,
 couple_traveler,
 comfort_easy_pace_traveler,
 solo_social_traveler,
 interest_deep_dive_traveler,
 active_adventure_traveler
}
```

`first_time_visitor` is the default audience.

If the system cannot confidently detect a stronger audience signal, it must fall back to `first_time_visitor`.

Audience tags are used for:

- DEE snippet selection
- product ordering profiles
- bundle selection
- rail emphasis
- review-summary emphasis
- trust-proof emphasis
- CRM / retargeting audience logic
- future personalization logic

Audience tags should not be used as hard filters by default. They should primarily guide ordering, emphasis, and presentation unless a separate constraint requires filtering.

4.8.9.2 Definition Block: Intent tags

Intent describes what the traveler is trying to optimize for in the current context.

```
None
intent ∈ {

 best_value,

 premium_comfort,

 hidden_gems,

 less_crowded,

 bucket_list_icons,

 deep_dive_learning,

 activity_intensity,

 family_compatibility,

 private_upgrade

}
```

Intent tags are additive.

A user, page, product, rail, or recommendation context may have:

- no strong intent,
- one primary intent,
- or up to two supporting intents where evidence is strong enough.

Intent tags are used for:

- product ranking
- rail logic
- bundle composition
- near-page matching
- “why this pick” explanations
- trade-off explanations
- CTA emphasis
- CRM / retargeting messaging

Hard rule: `hidden_gems` must not be merged with adventure or activity-intensity logic.

`hidden_gems` is a separate intent dimension. A hidden gem can be calm, cultural, family-friendly, premium, or active. It should not automatically be treated as adventure.

4.8.9.3 Definition Block: Constraint tags

Constraints describe explicit requirements or limitations that the system must respect.

```
None
constraints ∈ {

 private_only,

 small_group_preferred,

 low_walking_required,

 stairs_sensitive,

 stroller_required,

 wheelchair_access_needed,

 short_time,

 half_day_only,

 hotel_pickup_preferred,

 free_cancellation_preferred,

 strict_budget_cap,

 weather_sensitive

}
```

Constraint tags should be applied only when they are explicit or strongly implied by user behavior, filters, product data, or page context.

Constraints may be soft or hard depending on the tag.

Examples:

- `private_only` is a hard constraint.
- `wheelchair_access_needed` is a hard accessibility constraint.
- `strict_budget_cap` is a hard commercial constraint.
- `hotel_pickup_preferred` is usually a soft preference unless explicitly required.
- `small_group_preferred` is usually a soft preference.
- `low_walking_required` may become hard if the user explicitly selected it.

Constraints are used for:

- filtering
- eligibility checks
- product suppression
- fallback logic
- bundle feasibility
- Magic Finder behavior
- travel-agent search support
- accessibility and comfort logic

4.8.9.4 Definition Block: Eligibility state

Eligibility state describes how well a product fits a specific audience / intent / constraint profile.

```
None
eligibility_state ∈ {

 strong_fit,

 weak_fit,

 unknown,

 not_recommended

}
```

Eligibility state is not the same as product quality.

A high-quality product may still be `not_recommended` for a specific traveler profile if it does not fit the user's needs or constraints.

Examples:

- A highly rated long walking tour may be `not_recommended` for `comfort_easy_pace_traveler` with `low_walking_required`.
- A group tour may be `not_recommended` when `private_only` is active.
- A family-friendly museum tour may be `strong_fit` for `family_traveler`.
- A product with insufficient metadata may be `unknown`, not automatically preferred.

Hard rules:

- `private_only` is a hard filter: if the product is not private, it is OUT for that profile.
- `not_recommended` cannot win any main-stack slot for that profile.

- `unknown` never gives advantage.
- `strong_fit` may receive ranking preference where the product also passes quality, availability, and archetype rules.
- `weak_fit` may remain eligible, but should not beat a comparable `strong_fit` unless other deterministic rules justify it.

Eligibility state is used for:

- product-to-profile fit
- winner selection
- rail eligibility
- bundle product selection
- fallback logic
- Magic Finder filtering
- “why this pick” explanations
- suppression of unsuitable products

Important:
Numeric preference multipliers are configured in Chapter 3 Product Ops Settings.
Chapter 4.8 defines the deterministic decision order only.

4.8.9.5 Interpretation Rule

The four classification layers must always remain separate.

Audience answers:

Who is this traveler?

Intent answers:

What is this traveler trying to optimize for?

Constraints answer:

What must the system respect, avoid, or filter by?

Eligibility state answers:

How well does this product fit this specific profile?

This separation is critical because the same audience may have different intents.

Examples:

- A `family_traveler` may want `best_value`.
- A `family_traveler` may want `premium_comfort`.
- A `couple_traveler` may want `hidden_gems`.
- A `couple_traveler` may want `bucket_list_icons`.

- A `comfort_easy_pace_traveler` may still want `deep_dive_learning`.
- A `solo_social_traveler` may still have `strict_budget_cap`.

The same intent may also appear across multiple audiences.

Examples:

- `best_value` can apply to families, couples, solo travelers, and first-time visitors.
- `premium_comfort` can apply to couples, families, and comfort-focused travelers.
- `hidden_gems` can apply to deep-dive travelers, couples, or active travelers.
- `private_upgrade` can apply to couples, families, or premium first-time visitors.

Constraints may apply independently of both audience and intent.

Examples:

- `wheelchair_access_needed` may apply to any traveler type.
- `short_time` may apply to any audience.
- `private_only` may apply to any intent.
- `weather_sensitive` may apply to outdoor, family, or comfort-focused contexts.

Eligibility state applies after audience, intent, and constraints are understood.

It determines whether a product should be:

- preferred,
- allowed,
- treated neutrally,
- or suppressed for a specific profile.

4.8.9.6 Priority Rule

System logic should evaluate the classification layers in this order:

1. Audience determines the dominant traveler type.
2. Intent determines what the traveler wants to optimize for in this context.
3. Constraints determine what must be respected, avoided, suppressed, or filtered.
4. Eligibility state determines whether the product is a strong fit, weak fit, unknown fit, or not recommended for that profile.

The system should not treat audience as a replacement for intent, constraints, or eligibility.

For example:

- `family_traveler` should not automatically mean `best_value`.
- `couple_traveler` should not automatically mean `premium_comfort`.
- `comfort_easy_pace_traveler` should not automatically mean `senior`.

- `first_time_visitor` should not automatically mean `bucket_list_icons`, although it may often reinforce that direction.
- `active_adventure_traveler` should not automatically mean `hidden_gems`.
- `unknown` eligibility should not increase product ranking.
- `not_recommended` must not win main-stack positions.

4.8.9.7 Practical System Usage

This classification layer is used across the Decision Platform.

It must influence:

Product selection:

- eligibility
- archetype fit
- quality comparison
- winner logic
- profile fit

Product ranking:

- default ranking
- audience-aware ordering
- intent-aware tie-breaks
- constraint-based suppression
- eligibility-based preference

Rails:

- which products appear as winners
- whether Best Value / Premium / other rail logic is appropriate
- which products are suppressed
- how explanations are written

Bundles:

- product composition
- pacing
- traveler fit
- constraint feasibility
- “why these picks”
- trade-offs / what was skipped

DEE:

- snippet variant selection
- section emphasis
- ordering profiles

- hook visibility
- review-summary emphasis

Near pages:

- page topic classification
- decision angle
- matching to audience / intent
- SEO-safe content framing

Magic Finder:

- POI pack logic
- on-request inventory logic
- fallback handling
- explicit constraint handling
- eligibility-based product suppression

CRM / Retargeting:

- audience grouping
- destination-interest messaging
- product recommendation emails
- private upgrade campaigns
- abandoned planning flows

4.8.9.8 Governance Rule

These tag sets and eligibility states are final for the current specification version and should be treated as controlled vocabularies.

Developers must not introduce new audience, intent, constraint, or eligibility values ad hoc.

If a new tag or eligibility state is needed later, it must be added through a controlled taxonomy update process, including:

- definition
- allowed usage
- mapping rules
- effect on product selection
- effect on DEE
- effect on CRM / retargeting
- backward compatibility check

This is required to prevent taxonomy drift, inconsistent product classification, and unreliable decision logic.

4.8.10 Stage G — How exactly a product is chosen (winner selection algorithm)

This stage is the explicit selection logic (no shortcuts).

Inputs

- candidates = Pool products that are Set-eligible (INSTANT) for the relevant placement
- candidates are grouped by Archetype Core
- each candidate has: rating, review_count, classification, privateTour, pickupOptionType, duration_bucket, languages_count (if known), eligibility_state per profile, and flags

4.8.10.1 Definition Block: Base Quality Score (BQS)

BQS orders already high-quality candidates:
BQS = w_rating * rating_component + w_reviews * review_component

Weights are bounded and configured in Chapter 3.
Purpose: stable ordering, not “magic”.

4.8.10.2 Definition Block: Language advantage (ALWAYS ON)

Within the same Archetype Core:
If A.languages_count >= language_trigger_min AND B.languages_count <= language_low_ceiling,
then A beats B even if B has a higher rating, provided both passed gates.

If languages_count is UNKNOWN for either product, language advantage is ignored for that comparison.

4.8.10.3 Selection order (deterministic)

For each Archetype Core and target placement:

Step 1 — Hard filters

- If profile has private_only: keep only itinerary.privateTour = true
- Remove blocked products

- Remove MANUAL products (Set requires INSTANT)
- Remove candidates that violate classification requirement for placement:
    - Top Tickets placement accepts only classification ∈ {TICKET, PASS}
    - Ticket blocks reject TOUR always

Step 2 — Exclude not_recommended

- Remove candidates where eligibility_state == not_recommended for the active profile

Step 3 — Apply language advantage (if applicable)

- For comparisons where language rule triggers, the higher language coverage candidate wins

Step 4 — Order remaining by BQS

- Sort remaining candidates by BQS descending

Step 5 — Apply tie-break preference multipliers (bounded; Chapter 3)

- Apply only if still tied within a small margin:
    - pickup_preference_multiplier (for comfort/family)
    - private_preference_multiplier (for upgrade slots)
    - walking_penalty_multiplier (bundle-only; does not change winner ranking unless configured later)

Tie-breakers must never overrule hard rules (INSTANT, classification).

Step 6 — Choose winners per core

Compute the following, subject to availability of materially different options:

- default_winner: best overall for default profile (first_time_visitor + best_value + bucket_list_icons baseline)
- best_value_winner: best aligned to best_value

- premium_winner: best aligned to premium tier, only if materially different (R6); else provide 1 winner + “choose by intent” CTA

- private_upgrade_winner: max 1, only if private exists and is eligible; not shown as default unless private_only or explicit user selection

4.8.10.4 Private policy (locked intent)

- Private tours are primarily an upgrade layer

- Partner private is allowed only when thresholds are met and city private inventory policy allows (parameterized but bounded in Chapter 3)

- Rosotravel Originals policy: Originals are strategically preferred but not auto-winners; precedence rules for “same sightseeing archetype” are enforced (defined in Phase C policy; implementation uses pinned/priority within core)

4.8.11 Stage H — Build Sets (City / Things-to-do / Attraction) with explicit selection steps

This stage turns winners into page-ready curated Sets and enforces R1–R6.

4.8.11.1 City Set construction algorithm (high-level steps)

Inputs

- winners per archetype core

- city type (Iconic/Culture, Water/Cruise, etc.) determined from eligible archetype distribution (policy defined elsewhere; rails sets are fixed)

- section budgets (counts; configurable within bounds in Chapter 3)

Algorithm

1. Build Bundles (3 cards: 1/2/3 day) using Stage I (bundle builder).

2. Build Top Picks (6):

    - Start from the highest-priority archetypes for the city type (e.g., top attraction access, city highlights, food, etc.)

- Select default_winner from distinct archetype cores, enforcing:
    - R1 no product twice
    - avoid repeating same attraction product across Top Picks (R2)
- If a chosen Top Pick would repeat an already-used POI/archetype, select next candidate within that core

3. Build Private Upgrade section:
    - Show up to 2 CTAs only (no list):
        - “Upgrade to Private”
        - “See all private tours”
    - No products displayed here
4. Build Category Rails (max 4 rails × 2 products):
    - For each rail archetype group, select:
        - Best Value winner
        - Premium winner (only if materially different; else single pick + CTA)
    - Enforce:
        - R3 Rails must not repeat Top Picks (pick next best eligible)
        - R6 “2-of-many must be real”
5. Build Top Attractions navigation (6 cards):
    - Navigation only; no buy CTAs
    - Can reference attractions even if a related product is in Top Picks (allowed), but do not display product cards here
6. Build Top Tickets (4–6):
    - Only classification ∈ {TICKET, PASS}
    - Never include TOUR (R4)
    - Must not repeat any product already shown elsewhere on page (R1)

Output

CITY_{city}_SET_CORE membership list + section assignments + portfolio_version/hash

4.8.11.2 Things-to-do Set construction algorithm

Similar to City Set, with different budgets:

- Top Picks first
- Bundles second
- Rails count 6 × 2
- Top Tickets 4–6
  Same anti-duplicate rules apply.

4.8.11.3 Attraction Page Set construction algorithm

Inputs: POI-specific candidate set (POI cluster; no POI merge allowed).

Steps:

1. Our 2 picks:
    - Best Value + Premium/Bucket-list if eligible and materially different
    - If only one qualifies, show one (no weaker fill)
2. Intent mini-rails:
    - Only show intents that have eligible options (Family/Less crowded/Private upgrade/Hidden gems)
3. Private upgrade: 1 CTA
4. Tickets-only block:
    - Only TICKET/PASS
    - Never TOUR

4.8.12 Stage I — Bundle Builder (explicit, profile-aware)

Bundles are duration-based (1/2/3 day). No segment explosion.

Bundle structure (locked)

- Anchor + Complement + Flex

Difficulty index and Time Geometry are bundle-only.

Definition Block: Difficulty weights (bundle-only)

Difficulty weights per product class are configured in Chapter 3 (bounded).
Difficulty does not alter winners; it validates bundle feasibility.

Definition Block: Time Geometry Engine (bundle-only; no clock times)

- Day has three structural slots: A/B/C
- Block size derived from duration: SMALL/MEDIUM/LARGE/FULL_DAY
- FULL_DAY occupies A+B+C
- LARGE occupies A+B (C only light/relax)
- Timed item caps per audience apply (configured)
- High difficulty adjacency rules apply (configured)

Anchor selection principle (commercial trust)

For large cities (600+ eligible inventory), first_time_visitor default anchors prioritize:

- top attraction access with guide (TOUR) over generic city highlights as primary anchor,
  unless city inventory lacks sufficient top attraction guided options.

Day 2 strategy (city-size-aware)

- For large cities (600+ eligible inventory): first_time_visitor defaults to CITY_DEEP (in-city top attractions + thematic)
- For smaller cities: first_time_visitor defaults to DAY_TRIP/CAR strategy
  Exact threshold and parameters are configured in Chapter 3.

4.8.13 Stage J — Phase B behavior (explicit; no batch)

Phase B is activated only after explicit user intent (Show more options / Magic Finder).

Phase B consumes:

- Pool candidates (including MANUAL and below-standard “on your request” pools as defined in Phase B spec)

Phase B must:

- never change Set membership
- never trigger batch
- clearly separate “on your request” items from curated picks

---

4.8.14 Batch trigger boundary (clarity table)

Only Set membership triggers SEO/AI batch.

Layer / Collection	Main stack eligible	Phase B eligible	Triggers SEO/AI batch?	Notes
RAW_ALL_PRODUCTS	No	No	No	Provider mirror
POOL_ELIGIBLE	No	Yes	No	Dynamic eligibility
SET_* (City/Things/POI Sets)	Yes	Yes (may re-surface)	Yes	Only Set triggers batch
PhaseB_Curated_Catalog	No	Yes	No	Band-limited browse
PhaseB_OnYourRequest	No	Yes	No	Below-standard + MANUAL
B2B_Expanded_View	No	Yes	No	Partner view; no batch
Runtime Availability Fallback	Yes (runtime only)	N/A	No	Does not change Set

---

4.8.15 Single Product after Batch (reference; no workflow duplication)

If a product is added to Set after city batch has already completed:

- a single-product processing run must be triggered (governed)
- it must not rebuild the city batch
- it updates product-level offer_version only

Execution steps are defined in Chapter 4 (“Processing a New Single Product (End-to-End)”) and logging fields in 4.7 domains.

4.8.16 Worked Example (how a product is chosen end-to-end)

Example scenario: City = Vienna (large city; 600+ eligible), Profile = default first_time_visitor, no explicit intents chosen.

Goal

Select winners for:

- Top Attraction Access rail (Schönbrunn-like POI core)
- Night Experience rail (concert-like ticket/pass)
  and produce Top Picks and Bundles without violating locks.

Step 0 — Inputs

- Start from RAW_ALL_PRODUCTS (Vienna validated start city).
- Pool eligibility computed (Band C thresholds for Group01/02).
- Set requires confirmationType == INSTANT.

Step 1 — Candidate extraction for a target archetype core (Top Attraction Access)

- From Pool, filter to INSTANT only (Set eligible).
- Classification:
    - if any languageGuides.type == GUIDE → TOUR

    - else ticket/pass logic applies

- Build Archetype Core:
    - group_family=01
    - experience_mode=tour_like
    - poi_cluster=Schönbrunn attractionId (if available) else UNKNOWN_POI (city-level only)
    - format_core=shared + pickup/meeting/unknown
    - duration_bucket=half_day/full_day/unknown

If Core candidate_count < 4:

- merge duration_bucket away
- then merge pickup dimension
- POI merge is forbidden on attraction page; allowed at city-level only if still <4

Step 2 — Apply profile fit filters

- If private_only: keep private only (not active here)
- Remove not_recommended candidates for this profile
- UNKNOWN does not help

Step 3 — Apply language advantage (always-on)

- If candidate A has languages_count >= 3 and candidate B <= 1 → A wins comparison

Step 4 — Sort by BQS

- Compute BQS for remaining candidates
- Pick top as default_winner for this core

Step 5 — Select Best Value and Premium winners

- best_value_winner: best aligned to best_value among remaining
- premium_winner: best aligned to premium tier, only if materially different (not near-duplicate); else single winner + CTA “choose by intent”

Step 6 — Repeat for Night Experience core

- Candidate core likely uses Group02 and classification TICKET/PASS
- Apply INSTANT gate (exclude MANUAL from Set)
- Top Tickets block accepts only TICKET/PASS; TOUR is excluded

Step 7 — Build City Set

- Top Picks: choose 6 winners from distinct archetype cores, enforcing R1–R3
- Rails: select 4 rails ×2, enforcing R6 (“2-of-many must be real”)
- Top Tickets: fill 4–6 with TICKET/PASS only, enforcing R1 and excluding TOUR

Step 8 — Build Bundles (3 cards)

- Use bundle builder:
    - 1-day: Anchor = top attraction guided (TOUR) + Complement = Night ticket (TICKET/PASS) if feasible by time geometry and timed caps
    - 2-day: Day 1 anchor top attraction + night; Day 2 CITY_DEEP (another top POI tour + tasting)
    - 3-day: include day trip if it exists as eligible FULL_DAY and does not violate timed/difficulty constraints; else a relax cruise + second top POI

Outputs

- Set membership lists per page type
- winners per archetype core
- portfolio_version and portfolio_hash(city_id)
- flags for review (if any Set items violate Pool thresholds later)

Derived Product Tags (HARD vs SOFT) — Contract

HARD tags (structural, API-derived) may influence:
- product classification (TICKET/PASS/TOUR)
- archetype core/variant assignment
- eligibility and constraints filtering (only when data is KNOWN)
- bundle feasibility (difficulty + time geometry)
- INSTANT gate enforcement

SOFT tags (semantic/merchandising) must NEVER influence:
- Pool eligibility gates
- Set eligibility
- product exclusion decisions

SOFT tags may be used only for:
- rail naming and UI copy
- “why this pick” explanations
- optional last tie-break within already-qualified candidates

All derived tags must carry coverage state:
- KNOWN: derived from explicit provider fields or confirmed mapping
- UNKNOWN: not available; must not affect ranking or selection

4.9 Batch Lifecycle (Create RAW Batch → Append Records → Manual Activation → Restricted Full Re-Run)

4.9.0 Purpose

A batch is the controlled execution container for destination-scope processing.

It is a persistent container for RAW records and becomes the authoritative baseline for SEO ownership outcomes once activated.

Hard principles

- The first product that becomes a Set member for a destination scope MUST create a RAW batch automatically.

- A RAW batch is inactive by default; no AI batch processing runs automatically.

- New Set-member products are appended to the existing batch history without triggering processing.

- Activation is manual (“Run AI Batch Processing”).

- Full batch re-run is restricted (Marketing Manager/Admin only) and used only for strategy-level mistakes.

- After activation, late-arriving products are processed as single records and MUST respect existing ownership locks (cannot reclaim keywords or create colliding near-pages).

4.9.0A Relationship to Chapter 4.8 (Portfolio Choosing / Product Classification) — HARD LINK

Chapter 4.8 defines:

- how products move through RAW → Pool → Set (membership flags, no duplicated records),

- how products are classified and deduplicated (archetypes, ticket vs tour-like, etc.),

- how Model C outputs are computed for templates (Top Picks, Rails, Top Tickets, Bundles, Attraction picks),

- and which products are Set-eligible for the “main stake” (including INSTANT-only policy).

Chapter 4.9 defines:

- when and how those 4.8 outputs are processed as an execution unit (batch),

- how baseline SEO ownership is created (on activation),

- and how late-arriving products are handled without destabilizing the baseline.

Hard scope rule

- Pool membership changes are automatic and do NOT create or expand batch scope by themselves.

- Batch scope is Set-driven:

A product participates in batch processing only when it is a Set member for the destination scope (set_member = true) and meets Set eligibility rules defined in 4.8 (including INSTANT-only for main stake).

4.9.1 Initial batch creation (RAW batch is created immediately; no cycle, no staging)

Trigger (hard)

- The first product that becomes a Set member for a destination scope (as defined in 4.8) MUST create a batch automatically.

Behavior

- System creates batch_{scope_id}_{timestamp} as a persistent container for RAW records.

- Initial state:

RAW / inactive

- RAW batch stores:

- scope_id (e.g., city_id / destination scope)

- created_at, created_by (system)

- initial Set member product_id (first record)

- version domain references (raw_version / offer_version / realtime_offer_version)

- RAW batch remains inactive until a human triggers processing.

Hard

- No staging cycles. No “draft batch”. No automatic scheduled run.

- RAW batch creation does not run AI, does not publish, and does not write keyword ownership.

4.9.2 Adding records to an existing RAW batch (append-only; no triggers)

Trigger

- As more products become Set members for the same destination scope, they are appended to the same RAW batch history.

Behavior

- Products are appended to batch history:

- appended_at timestamp

- appended_reason (auto eligibility / manual Set add / pin)

- membership flags snapshot (pool_member / set_member / pinned / blocked)

- No automatic batch AI processing is triggered by appends.

Hard

- Appends to RAW must be side-effect free:

- no AI drafting

- no SEO ownership writes

- no near-page generation

- no publishing actions

Hard note (scope)

- Appending records to batch history is triggered by Set membership changes (set_member=true) for that scope.

- Pool-only changes (pool_member=true, set_member=false) must NOT append to batch scope; they remain eligible for Phase B only.

4.9.3 Manual activation (Run AI Batch Processing) — authoritative baseline creation

Trigger (hard)

- A user triggers “Run AI Batch Processing”.

Who can activate (recommended policy)

- Marketing Manager / Admin (optionally SEO Lead if explicitly allowed in Roles & Permissions).

Behavior

- Batch transitions:

RAW / inactive → ACTIVE

- The batch run produces the authoritative baseline for that destination scope:

- keyword_map writes (ownership locks) become baseline

- near-page creation decisions follow governance rules

- the destination scope enters “post-activation” mode for ongoing changes

Governance note

- Once activated, batch-level SEO ownership outcomes are treated as authoritative.

- Future changes must not rewrite ownership locks except through explicit allowed governance actions (defined in SEO governance rules).

4.9.3A Batch Scope Snapshot (from 4.8 outputs) — REQUIRED ON ACTIVATION

Purpose

A batch run must have deterministic inputs. On manual activation, the system must capture a snapshot of 4.8 outputs used by templates.

On activation (RAW → ACTIVE), create:

batch_scope_snapshot (immutable for the run_id), including at minimum:

- scope_id (city_id / destination scope)

- set_member_product_ids[] (the Set membership list at activation time)

- pinned/blocked overrides affecting Set ordering (if any)

Template outputs computed from 4.8 at activation time:

- City Page outputs:

- bundles (1/2/3 baseline)

- top_picks (6)

- rails (up to 4×2)

- top_attraction_tours cap (band-sized)

- top_tickets cap (band-sized)

- Things-to-do outputs:

- top_picks (6)

- bundles (1/2/3 baseline)

- rails (6×2)

- top_attraction_tours cap (band-sized)

- top_activities_feed cap (band-sized)

- Attraction page seeds (optional scope):

- pick sets / eligibility snapshot for attraction pages, if included in batch scope

Bundles included in scope:

- bundle_ids[] and proof presence flags:

- bundle_decision_proof_present (true/false per bundle)

Hard rule (bundle proof)

If required bundle decision proof objects are missing for baseline bundles,

activation must raise a blocking flag or require explicit override (policy choice),

because bundle pages must not publish without explainability/trade-offs.

4.9.4 Restricted Full Re-Run (batch-level reset) — MM/Admin only

Definition

Full re-run = reprocessing the entire destination batch scope end-to-end (resetting/recomputing outputs).

Hard rule

- Full re-run is restricted to Marketing Manager/Admin only.

Allowed use cases (hard)

- Strategy-level mistakes only, e.g.:

- wrong SEMrush settings

- wrong keyword retrieval configuration that caused ownership errors

- incorrect destination scope configuration that must be rebuilt

Not allowed use case (hard)

- Regular new products arriving after activation.

These must be handled by single-record processing.

Requirements

- Full re-run must require:

- explicit reason entry (mandatory)

- audit logging

- optional approval chain (recommended for high-scale scopes)

4.9.5 Single-record processing (late-arriving products after batch activation)

When a batch is ACTIVE and a new product arrives:

- the product still enters RAW and is appended to batch history

- AI processing is executed per single record (not batch-wide)

Purpose

Enable continuous growth without destabilizing the destination baseline.

Hard rule: single-record processing MUST respect existing ownership locks

Single-record processing cannot:

- reclaim keywords already locked by earlier batch run

- rewrite keyword ownership for the destination scope

- spawn near-pages that collide with already-consumed keyword ownership for that destination

Allowed outputs (single-record run may)

- regenerate product-level AI draft fields for that product only

- update product-level structured data for that product

- update embeddings for that product

- recompute this product's classification/archetype assignment (4.8) and store explainability

- raise flags/alerts for review if the product would materially impact existing winners/rails/bundles

Not allowed outputs (single-record run must NOT)

- rebuild the entire destination batch

- recompute destination-wide rails/bundles/top picks baseline captured in batch_scope_snapshot

- create ownership-colliding near pages

- modify other products' outputs

Collision handling (hard)

If a single-record run detects that a potential near-page or keyword would collide with existing ownership:

- it must fail-closed for that candidate

- log a collision event

- raise an alert for SEO review (recommended)

4.9.6 Audit & Traceability (batch + single record)

Every action must be logged:

- batch creation (auto)

- record append (auto/manual)

- activation (manual)

- full re-run (restricted)

- single-record run triggers (manual/auto)

- collision events (keyword/near-page)

Each log entry includes:

- actor (system/user), role

- timestamp

- scope_id

- product_id(s) (if applicable)

- action_type

- reason (required for restricted actions)

4.10 Execution Seeds (City / Major Attraction / Optional Product) — Rules + Gates

Tu ma być: seeds dla city/major POI/wyjątek product; tylko scope+cost control, bez powtarzania template'ów.

01. Generate seed from products, cities, attractions, POI for keywords retrieval

SEED GENERATION & CONTROL SYSTEM

Purpose of the Seed Layer

The Seed Generation System defines **which phrases are sent to keyword discovery tools** (e.g. SEMrush) to retrieve searchable demand signals.
Its purpose is to:

• control SEO scope and cost
• prevent low-value keyword noise
• enforce clean keyword ownership
• align keyword discovery with real product availability
• ensure scalability across thousands of cities and attractions

Seeds are **inputs only**. They do not define final keywords, clustering, or ownership.

Seed Scope & Core Principles

1. Seeds are **minimal, high-intent, canonical**
2. Seeds are **entity-driven**, never free-text
3. Seeds are generated **only from authoritative internal data**
4. Seeds are **independent of supplier-specific semantics**
5. English is always the **source language**
6. All expansion happens **inside the keyword tool**, not in seed logic

Seed Entity Types

Seeds are generated **only** for the following entities:

• City
• Major Attraction
• Product (limited, gated)

Programmatic Seeds are generated only for City / Major Attraction / Product (gated). Destination Seeds for Country/Region/Island are handled separately by Run SEO Auto (see Destination Seed section)

Canonical City Entity (Seed Source)

Required City Fields

Each city used for seeding must contain:

• city_id (internal canonical ID)
• city_name_en (canonical English name)
• country_id (internal relation)
• products_count (active, published)
• viator_destination_id (nullable, linkage only)

City Activation Rule

A city is eligible for seed generation **only if**:

```
products_count ≥ 1
```

Inactive cities are stored but never seeded.

City Seed Phrases

Exactly **three** seeds per city:

• things to do in {city_name_en}
• {city_name_en} attractions
• {city_name_en} tours

No other city-level variants are allowed in MVP.

Attraction Entity & Major Attraction Logic

Required Attraction Fields

Each attraction must contain:

• attraction_id (internal canonical ID)
• attraction_name_en (canonical English)
• city_id (relation)
• attraction_type (museum | landmark | palace | site | activity | other)
• product_count
• total_reviews
• average_rating

Major Attraction Classification

An attraction is classified as **Major** if **any one** of the following is true:

• product_count ≥ 10
• total_reviews ≥ 1,000
• total_reviews ≥ 300 AND average_rating ≥ 4.6
• is_major_attraction_override = true (manual SEO admin override)

Automatic Exclusions

An attraction is **never** major if:

• product_count = 0
• attraction is non-visitable
• attraction is administrative or generic (e.g. “city center”)

Major Attraction — Storage (Required Fields)
For every attraction entity, store:

- is_major_attraction (bool, computed)
- is_major_attraction_override (bool, manual SEO admin)
- major_attraction_reason (enum, system):
    - product_count_threshold
    - reviews_threshold
    - rating_reviews_combo
    - manual_override
    - excluded_non_visitable
    - excluded_generic_admin

- major_attraction_last_evaluated_at (datetime)

CMS controls (SEO admin only):

- toggle: is_major_attraction_override
- field: override_note (text, required if override is true)

Major Attraction - Effect on Keyword Generation

Major attractions unlock deeper long-tail expansion while still respecting D.05 similarity bands and D.06 ownership rules.

Effects:

1. Lower long-tail volume threshold for near-page candidates linked to this attraction:
- standard POI long-tail minimum volume = 30
- major attraction POI long-tail minimum volume = 10

2. Expanded seed template set for POI keyword retrieval (enabled only when is_major_attraction = true):
- “{POI} tickets” (transactional → product routing)
- “{POI} guided tour” (transactional → product routing)
- “best {POI} tours” (commercial → near-page routing)
- “{POI} with kids” (commercial → near-page routing)
- “{POI} opening hours” (informational → attraction routing)
- “{POI} how to get there” (informational → attraction routing)
3. Near-page cap scaling for a major attraction:
- max 10 near-pages per major attraction per 5-week cycle
- max 3 hub near-pages per major attraction per 5-week cycle (high-KD head themes)
4. If attraction is not major:

- do not generate POI-specific near-pages unless volume ≥ 30 and inventory coverage is strong (≥ 8 eligible products).

Manual Override Rules

CMS fields:

• is_major_attraction_override (boolean)
• major_override_reason (required text)

Only SEO Admin may apply overrides.

Attraction Seed Generation Rules

Seeds are generated **only for Major Attractions**.

Core Attraction Seeds (MVP)

Exactly **three** seeds per major attraction:

• {attraction_name_en}
• {attraction_name_en} tickets
• {attraction_name_en} guided tour

No adjectives, no modifiers, no location duplication.

Controlled Near-Intent Seed

Exactly **three** informational seeds per major attraction:

• {attraction_name_en} opening hours
• {attraction_name_en} ticket price
• {attraction_name_en} how to get there

These are used **only** to:

• route long-tail queries
• support near-page eligibility
• populate FAQ discovery

They do **not** imply indexation.

Product Entity (Transactional Routing Only)

Products **do not generate independent seeds** by default.

Product Fields Used for Routing

• product_id
• product_title_en
• city_id
• attraction_id (nullable)
• product_category (tour | ticket | experience)
• bookable (true | false)

Product Seed Exception (Optional, Limited)

Product-level seeding may be enabled **only** if:

• product is bookable
• product is linked to a major attraction
• SEO Admin explicitly enables product seeding

When enabled, **one seed only**:

• {attraction_name_en} tour

No product titles, no variants, no brand names.

Language & Database Rules

• All seeds are generated in **English only**
• English keyword discovery is authoritative
• All other languages are translations of indexed content
• SEMrush (or alternative provider) database is selected by:
– English + country context

What Is Explicitly NOT Included in Seeds

The following data is **never** used for seed generation:

• opening hours
• addresses
• geo coordinates
• popularity scores
• seasonality
• pricing values
• itineraries
• product descriptions
• reviews text

These belong to **factual enrichment**, not keyword discovery.

Seed Generation Timing & Batching

Seeds are generated:

• per batch (grouped by country or region)
• after product ingestion and destination activation
• before embeddings, clustering, or ownership

Seeds are immutable per batch version.

SEO Tooling (Provider-Agnostic)

Seed logic is independent of keyword provider.

The SEO team must select **one** provider:

• SEMrush API
• SE Ranking API
• Oxylabs / Bright Data (SERP scraping)

Regardless of provider:

• seed structure remains unchanged
• ownership rules remain unchanged
• clustering logic remains unchanged

Providers are interchangeable data sources.

System Guarantees

This seed system guarantees:

• no keyword explosion
• no city/attraction cannibalization
• predictable SEO growth
• controllable AI costs
• clean semantic graph expansion

Summary

Seeds are **deliberately minimal**, **entity-controlled**, and **demand-weighted**.
Depth is earned through performance, not assumed.

This layer protects the entire programmatic SEO system from scale failure.

Example: SUMMARY TABLE (One-Glance Rules)

- Keyword → Page Type Mapping example

Keyword Type	Page Type
“guided tour”, “from krakow”	Product
Attraction name	Attraction
“things to do in city”	Things-to-do
“best tours”, plural	Near-page
opening hours / price / how to get	Attraction FAQ
city discovery	City
advice / tips	Blog

4.11 Keyword Retrieval + Ownership Locking (SEMrush + GSC → keyword_map writes)

Purpose

This section defines the operational pipeline that retrieves keywords (SEMrush), validates signals (GSC), and writes deterministic SEO ownership decisions to keyword_map (SSOT). The canonical routing rules themselves are defined in D.06; this 4.11.0 defines inputs, process, writes, audit, and failure behavior.

Authoritative SSOT

All keyword ownership outcomes MUST be written to keyword_map. No system may bypass it.

All intent allowlists, routing constraints, and forbidden actions are governed by the SEO SSOT (5.4 SEO Rules and metrics).

Inputs (deterministic)

A) Seed Sets (generated)

● Seeds MUST be generated from canonical entities: city, attraction/POI, product, and destination hierarchy.

● Each seed includes: seed_entity_id, seed_entity_type, locale, and eligible page types for that entity.

B) SEMrush Retrieval (primary)

● For each seed, retrieve keyword candidates with: intent label (including Mixed), volume, KD, CPC, and SERP feature hints when available.

● Store raw results versioned by run_id/batch_id (provenance).

C) GSC Validation (DQCP signal)

● Pull GSC signals to validate demand and prevent mis-ownership:

  ○ query feed per URL/entity scope (impressions/clicks/CTR/position),

  ○ and optionally observed queries per owner URL cluster.

● If GSC is unavailable, dqcp_state must be recorded as unknown (fail-closed for auto-index promotion).

D) Embeddings / Similarity (supporting)

● Compute keyword embeddings and use cosine similarity for:

  ○ duplicate detection (≥ 0.92),

  ○ variant band (0.75–0.91),

  ○ cluster band (≥ 0.78) for secondary selection.

(Thresholds and routing priorities are enforced by D.06 and D.06.CONFIG.)

Deterministic Processing (high-level)

Step 1 — Normalize keyword

● Normalize keyword text (lowercase, punctuation normalization, whitespace normalization).

● Store both raw and normalized forms.

Step 2 — Store intent (authoritative from SEMrush)

● Store SEMrush intent as intent_primary.

● If SEMrush returns Mixed, store exactly two values: intent_primary + intent_secondary.

Step 3 — Run D.06 deterministic routing

- Apply D.06 rules in order:
  - duplicate lock check,
  - intent allowlist enforcement,
  - entity detection + product/bundle fit gates,
  - priority resolution,
  - near-page gates/caps where applicable.

Step 4 — Write to keyword_map (SSOT)

For each keyword, the system MUST write/upsert a keyword_map row including at minimum:

- keyword_text (raw), keyword_normalized
- intent_primary (+ intent_secondary when Mixed)
- volume, kd, cpc
- owner_page_type (enum) and assigned_to_url (canonical owner) when owned
- status (candidate | owned | reserved | rejected)
- lock_state (locked | unlocked | legacy_locked)
- batch_id/run_id provenance
- last_decision_reason (required on any change)
- created_at, updated_at

And the system SHOULD also write:

- dqcp_state (passed/failed/unknown) from GSC validation
- collision_state (none/suspected/confirmed)
- serp_features (if available)
- similarity_class (duplicate/variant/new_topic) and duplicate_of_keyword_id (if applicable)

Audit (hard)

● Every ownership decision and any change must be auditable (actor, timestamp, old vs new, reason, affected URLs).

● No silent overwrite for ownership changes; ownership transitions must preserve history.

Fail-closed dependencies (hard)

● If SEMrush retrieval fails: do not create new ownership; keep previous keyword_map state; mark run as degraded.

● If GSC fails: allow proposals but set dqcp_state=unknown; require SEO approval before switching noindex→index for new pages.

D.06 Anti-Cannibalization Rules (Canonical SEO Logic)

D.06.0 Purpose (Why this exists)

This section defines the **canonical SEO governance layer** that prevents keyword and content cannibalization across all landing page types (12 templates). It ensures that:

- Every keyword has **exactly one canonical owner URL** (or is explicitly reserved/rejected).
- SEMrush search intent is treated as **authoritative**, and routing is **deterministic**.
- Programmatic growth (near-pages) is **controlled**, not explosive.
- AI content generation and regeneration are **subordinate to rules**, not the other way around.
- Conflicts are **blocked** before publishing and are **auditable**.

This section is written for developers: all rules must be enforceable in code, with clear thresholds and test cases.

D.06.1 Core objects and definitions

D.06.1.1 Keyword Map (single source of truth)

All keyword decisions must be written to keyword_map. No system may bypass it.

Minimum fields (required):

- keyword_text (raw)
- keyword_normalized
- intent_primary (SEMrush)
- intent_secondary (SEMrush, only if SEMrush returns Mixed)
- volume, kd, cpc (SEMrush)
- embedding_vector (keyword embedding)
- owner_page_type (enum)
- assigned_to_url (canonical owner)
- status (enum: candidate | owned | reserved | rejected)
- lock_state (enum: locked | unlocked | legacy_locked)
- batch_id (provenance)
- created_at, updated_at
- last_decision_reason (string, required on changes)

Extended fields (operational governance)

To support deterministic retrieval validation, collision handling, and downstream consumers (e.g., Gooru Topic Pool), keyword_map SHOULD include:

● dqcp_state (enum: passed | failed | unknown) — GSC validation state

● serp_features (optional) — snippet/PAA/local pack indicators if available from SEMrush

● collision_state (enum: none | suspected | confirmed) — ownership conflicts requiring review

● similarity_class (enum: duplicate | variant | new_topic)

● duplicate_of_keyword_id (nullable) — points to the canonical locked/owned keyword when similarity ≥ 0.92

● owner_candidate_url (nullable) — proposed owner prior to approval (useful for review UI)

● decision_history (array/log ref) — immutable references to audit entries (actor/time/reason)

D.06.1.2 Page type list (authoritative)

These are the only page types considered in ownership routing:

1. Homepage
2. Country Page
3. Region Page
4. City Page
5. Things-to-do Page
6. Attraction (POI) Page
7. Product Page
8. Near-page
9. Blog Page
10. Expert Page
11. Package / Bundle Page
12. Explore Attraction Map / AI Feed Context Page

D.06.1.3 Similarity thresholds (definitions)

All similarity checks use **cosine similarity** on embeddings.

- >= 0.92 → **Duplicate phrasing** (same keyword, reworded)
- 0.75 – 0.91 → **Variant** (same topic family, different wording)
- < 0.75 → **New topic** (meaningfully different intent/topic)

Additional “same-cluster” band used for secondary keyword selection:

- >= 0.78 → same semantic cluster (eligible to be secondary if intent allows)

D.06.2 Hard rules (always enforced)

Rule 1 — Keyword Locking (hard canonical ownership)

Once a keyword is assigned as **owned**:

- assigned_to_url = "<canonical url>"
- lock_state = locked

Hard effects:

- No other URL can own that keyword or any duplicate (similarity_to_locked >= 0.92).
- Any ownership change is **manual only** (via Keyword Ownership UI), audited, and must mark impacted pages as **OUTDATED** until regenerated/approved.

Example:

- If "schönbrunn palace tickets" is owned by /en/austria/vienna/schonbrunn-palace-guided-tour/ (locked) then "tickets for schonbrunn palace" with similarity 0.93 is discarded as duplicate and cannot be assigned elsewhere.

Rule 2: Intent Enforcement (SEMrush Intent Routing — Hard Rules)
SEMrush intent labels are authoritative. The system MUST route keywords using SEMrush intents, including the “Mixed” label.
SEMrush intent labels used by the system (authoritative):

- Informational
- Navigational
- Commercial
- Transactional
- Mixed (two intents)

1.

Rule 3 — FAQ Uniqueness (cross-template collision control)

FAQ **questions** must be unique across:

- Product
- Attraction
- Things-to-do
- Near-pages
- Blog

Before accepting any FAQ question, run Qdrant similarity against **all existing FAQ questions** platform-wide.

If faq_question_similarity >= 0.85 (recommended threshold):

- block insertion,
- require manual rewrite or reuse the canonical question.

Rule 4 — Duplicate Page Prevention (content-level cannibalization)

If page-to-page content similarity >= 0.90:

- block publishing,
- push into “Manual SEO Review Queue”,
- require SEO decision: merge / canonical / rewrite / de-index.

This applies even if keywords differ.

D.06.3 Page type → allowed SEMrush intent ownership
Mixed intent handling

Mixed intent handling (Hard, deterministic)

- SEMrush Mixed must be stored as: intent_primary + intent_secondary (exactly two).
- Mixed containing Transactional can be owned ONLY by:
  Product Page (if 1:1 product fit) OR Package/Bundle Page (if bundle-pattern fit).
  Otherwise → ineligible.

- Mixed = Commercial + Informational can be owned ONLY by:
  Things-to-do Page OR Near-page (never City, never POI, never Blog).
- Mixed containing Navigational is allowed ONLY on:
  Homepage / City / Expert / Explore (but never if it contains Transactional).

Page Type	Allowed Intents (Ownership)	Mixed intent allowed?	Notes (hard)
Homepage	Navigational, Informational	**Yes** (only if Mixed includes Navigational; excludes Transactional)	Brand hub. MUST NOT own Transactional. Commercial ownership is not allowed as target (route to Country/City/Things-to-do) .
Country Page	Informational, Navigational, Commercial	**Yes** (Info+Nav, Info+Commercial, Nav+Commercial)	Discovery hub at national level. MUST NOT own Transactional. No POI/product-specific ownership.
Region Page	Informational, Navigational, Commercial	**Yes** (Info+Nav, Info+Commercial, Nav+Commercial)	Mid-level discovery. MUST NOT own Transactional.
City Page	Informational, Navigational	**Yes** (Info+Nav only)	Trip-planning hub (“visit Vienna”, “Vienna travel guide”). MUST NOT own “things to do in {city}” head terms (reserved for Things-to-do). MUST NOT own Commercial discovery phrases about activities (route to Things-to-do / Near-page).

Things-to-do Page	Commercial	**Yes** (Commercial+Informational only; excludes Transactional)	Canonical owner for “things to do in {city}”, “best things to do”, “what to do”. MUST NOT own Transactional (tickets/book/price). This page is the “activity marketplace hub”, not checkout intent.
Attraction (POI) Page	Informational, Navigational	**No**	POI knowledge hub. MUST NOT own Commercial or Transactional. Any “tickets / tours / price / book” routes to Near-page (Commercial) or Product/Bundle (Transactional).
Product Page	Transactional	**Yes** (only if Mixed includes Transactional AND passes 1:1 product fit)	Strict conversion page. 1 keyword → 1 product URL. Once assigned, keyword is locked.
Near-page *(includes category pages like /{city}/{category}/ and keyword pages)*	Commercial, Informational	**Yes** (Info+Commercial only; excludes Transactional)	Long-tail expansion + comparison intent. MUST NOT own Transactional. If keyword becomes Transactional → route to Product/Bundle.
Blog Page	Informational	**No**	Editorial only. MUST NOT own Commercial/Transactional. Blog keywords are declared in Blog CMS and displayed as LOCKED

			(read-only) in Ownership UI.
Expert Page	Navigational, Informational	**Yes** (Info+Nav only; excludes Transactional)	Entity authority page. Not a conversion owner.
Package / Bundle Page	Commercial, Transactional, informational	**Yes** (Info+Commercial allowed only if explicitly “itinerary/plan/route”; Transactional allowed only if “package/booking” patterns match)	Can own “2 days in Rome itinerary + book/price” patterns. MUST NOT steal product transactional keywords like “{POI} tickets”. Bundle transactional is bundle-scoped (itinerary/package language).
Explore Attraction Map / AI Feed Context	Navigational, Informational	**Yes** (only if Mixed includes Navigational; excludes Transactional)	Discovery + dataset. MUST NOT own Commercial/Transactional

---

D.06.4 Deterministic routing (what wins when)

D.06.4.1 Routing always runs in this order (hard decision flow)

Step 0 — Normalize

- Normalize keyword (lowercase, punctuation removal, lemmatization if available).
- Compute keyword embedding.

Step 1 — Duplicate lock check

- If similarity_to_any_locked_keyword >= 0.92 → classify as duplicate.
- Action: discard as new ownership candidate (may still be stored for analytics, but not assigned).

Step 2 — Read SEMrush intent

- Store as intent_primary (+ intent_secondary if Mixed).
- If SEMrush intent missing → keyword cannot be owned automatically (status=candidate, requires manual review).

Step 3 — Entity detection
Detect whether keyword explicitly contains:

- a **City** (e.g., “vienna”, “rome”)
- a **POI / Attraction** name (e.g., “schönbrunn palace”, “colosseum”)
- **Booking/purchase modifiers** (tickets, book, price, entry, guided tour, skip-the-line, etc.)
- **Bundle patterns** (e.g., “2 days in rome”, “rome itinerary 3 days”, “vienna weekend plan”)

Step 4 — Product 1:1 fit test (mandatory for Product ownership)
A keyword can map to a Product page only if:

- intent is Transactional (or Mixed including Transactional),
- keyword maps 1:1 to a single product:
    - cosine(keyword_embedding, product_embedding) >= 0.78
    - AND no other product has similarity within 0.03 of the top match (to avoid ambiguous “multi-product” fit)
    - AND keyword is not a broad head hub query (e.g., “best tours in vienna”)

Step 5 — Apply page-type intent allowlist

- Page types that do not allow the intent are removed from candidates.

Step 6 — Apply hierarchy and specificity priority
When multiple allowed candidates remain, choose the most specific valid owner in this order:

1. Product Page (only if 1:1 fit passes and transactional-capable)

2. Package / Bundle Page (if itinerary/bundle pattern and commercial/transactional-capable)
3. Attraction (POI) Page (only informational/navigational)
4. Near-page (informational/commercial long-tail, inventory gates must pass)
5. Things-to-do Page (commercial hub)
6. City Page (broad commercial/navigational)
7. Region Page
8. Country Page
9. Homepage / Explore / Expert (only if navigational/informational and matches entity)

If none passes → mark keyword as reserved or rejected with reason.

D.06.4.2 Concrete scenarios (hard examples)

Scenario A — Transactional POI booking phrase

- Keyword: “schönbrunn palace tickets”
- SEMrush intent: Transactional
- Contains POI + tickets modifier → transactional-capable
- Product 1:1 fit passes → **Owner = Product Page**
- Lock keyword to that product URL.

Scenario B — POI informational query

- Keyword: “schönbrunn palace opening hours”
- SEMrush intent: Informational
- POI informational → **Owner = Attraction Page**
- Must not be assigned to product or near-page.

Scenario C — Broad city commercial query

- Keyword: “vienna sightseeing tours”
- SEMrush intent: Commercial
- Not 1:1 product fit → **Owner = Things-to-do Page** (primary)
  If Things-to-do not available, fallback to **City Page**.

Scenario D — Long-tail commercial comparison

- Keyword: “best vienna palace tours for families”
- SEMrush intent: Commercial
- Not 1:1 product fit; long-tail modifier; requires multiple products → **Owner = Near-page**
  Only if inventory coverage gate passes; otherwise fold into Things-to-do/City page modules (no new URL).

Scenario E — Mixed intent keyword

- Keyword: “blog platform free”
- SEMrush intent: Mixed (Informational + Commercial)
- Allowed owners: Near-page (Info+Commercial) or Country/Region (Info+Commercial) depending on entity relevance
- If city/entity not present and topic fits editorial only → **do not force Blog ownership** (Blog cannot own Mixed). Route to **Near-page** only if inventory/content structure supports; else reserved.

D.06.5 Keyword allocation lifecycle (pre-generation and ongoing)

Step 1 — Fetch keywords (pre-generation)

Before generating or updating any entity page:

- Fetch keywords from SEMrush (and optionally import GSC queries).
- Store each keyword as a record with intent + metrics + embeddings.

Hard rule:

- No page can become indexable unless it has a valid ownership decision in keyword_map.

Step 2 — Classify and route (ownership assignment)

Run deterministic routing (D.06.4) and write outcomes:

- owned → assigned_to_url + lock_state
- candidate → waiting gates/manual
- reserved → intentionally blocked
- rejected → forbidden patterns / conflicts

Step 3 — Select primary + secondary keywords (per owning page)

Use scoring (D.06.6), then write:

- primary_keyword (exactly 1)
- secondary_keywords (max 2; must be within semantic band and not duplicates)

Step 4 — Lock transactional ownership for products

When a Product page receives transactional ownership:

- set keyword(s) to locked
- prohibit reuse by near-pages, attractions, cities, blog

Step 5 — Regeneration linkage (governance only)

Ownership changes do not regenerate content here.
They only mark affected blocks/pages **OUTDATED** and create regeneration requirements (handled elsewhere).

D.06.6 Deterministic keyword scoring (used for primary/secondary selection)

Hard gates (apply before scoring)

A keyword is ineligible if any are true:

- intent is not allowed for page type (per matrix)
- similarity_to_any_locked_keyword >= 0.92 (duplicate)
- forbidden modifiers for the page type are present (defined in D.06.CONFIG)
- entity fit fails:
    - cosine(keyword_embedding, entity_embedding) < 0.78 for that entity

Scoring formula (deterministic)

Definitions:

- relevance_sim = cosine(keyword_embedding, entity_embedding)
- intent_fit = 1 if allowed else 0 (hard gate)
- volume_norm = log10(volume + 1)
- kd_norm = 1 - (kd / 100)
- cpc_norm = log10(cpc + 1)

Score:
keyword_score = (0.45 * relevance_sim) + (0.25 * volume_norm) + (0.20 * kd_norm) + (0.10 * cpc_norm)

Tie-breakers (in order):

1. higher relevance_sim
2. higher volume
3. lower kd
4. higher cpc

Primary vs secondary selection

- primary_keyword = highest scoring eligible keyword
- secondary_keywords = next best keywords that:
    - are not duplicates (< 0.92 similarity to primary and to each other)

    - are in the same semantic band (>= 0.78 similarity to primary)
    - have allowed intent for that page type

D.06.7 Near-page creation (spawn rules + gates)

Near-pages exist to expand topical coverage safely. They must never be created for duplicates or thin coverage.

Gate A — Inventory coverage (hard)

A near-page keyword must map to at least one of:

- >= 8 eligible products in the same city, OR
- >= 4 eligible products + >= 1 major attraction, OR
- >= 2 bundles (when bundles exist for that city/segment)

If gate fails:

- do not create near-page URL,
- route topic into existing City/Things-to-do modules or FAQ expansion.

Gate B — Volume (hard)

- Minimum volume: >= 30
- Exception: major attractions allowed down to >= 10

Gate C — Feasibility (kd/head handling)

- head_topic = true if volume >= 2000 OR matches head patterns (“things to do in X”, “best tours in X”, “X sightseeing”)
- If kd >= 70 AND head_topic = true → only eligible as **Hub Near-page** (stricter caps + required modules)
- Otherwise eligible as standard near-page.

Hub Near-page additional requirements:

- coverage: >= 20 eligible products OR >= 10 products + >= 3 bundles
- must include: Overview + Comparison + Top list + Strong internal linking

Gate D — Topic-distance throttle (prevents random long-tail explosion)

A near-page keyword must be:

- 0.62–0.78 similar to an existing owner topic (preferred), OR
- < 0.62 only if it can be routed to a known entity cluster and still passes coverage.

Gate E — “Head topic we cannot win” rule (hard)

If keyword is an iconic hard head topic and fails feasibility:

- do not create a dedicated near-page in MVP,
- include it only as a subsection inside a broader winnable near-page (non-owning).

Caps (hard)

Per city per 5-week cycle:

- max 30 new near-pages
- max 5 hub near-pages
  Overflow becomes candidate_queue (no URL or draft noindex depending on implementation).

D.06.8 Weekly keyword re-sync logic (SEMrush + GSC)

Inputs

- New SEMrush keywords and refreshed metrics
- New GSC queries (impressions/clicks)

Similarity classification (mandatory)

Compare each incoming keyword to all locked/owned keywords:

A) Duplicate (>= 0.92)

- Action: ignore for ownership changes (store as analytics only).
- No new page, no meta rewrite, no FAQ creation.

B) Variant (0.75–0.91)

- If intent = Transactional AND maps to same product 1:1:
    - attach as **secondary transactional keyword** to that product (if within limits)
    - allowed automatic updates (strict):
        - meta description allowed
        - FAQ answers allowed
    - forbidden automatic updates:
        - meta title
        - H1
        - URL / canonical
        - new headings
- If intent = Informational/Commercial:
    - attach as secondary to the owning non-product page type (if allowed)
    - may extend FAQ coverage (questions must pass uniqueness gate)

C) New topic (< 0.75)

- If Transactional:
    - if an existing product matches 1:1 → route to that product (lock)
    - else flag new_product_opportunity = true and require manual review
- If Commercial/Informational:

- if covered by existing City/Attraction/Things-to-do → expand existing content/FAQ (no new URL)
- else consider Near-page creation (must pass gates + caps)

D.06.9 Canonical conflict resolution (when overlap happens)

If two pages end up competing (detected via:

- keyword overlap conflict, or
- page similarity threshold breach, or
- SERP performance conflict signal if implemented)

Hard resolution flow:

1. Determine canonical winner using priority:
   - Product transactional ownership wins over all
   - Next: Package, Attraction, Near-page, Things-to-do, City, Region, Country
2. Apply canonical mapping:
   - weaker page → canonical to winner
3. Publishing rules:
   - if conflict is detected before publish → block weaker page
   - if conflict detected post-publish → keep live but mark “SEO REVIEW REQUIRED” until canonical is applied

No automatic deletion.

D.06.10 Audit and immutability

Every ownership decision and change must log:

- actor (system or user)

- timestamp
- old vs new owner
- reason code (required)
- affected URLs
- regeneration requirement flags (if change touches keyword-dependent blocks)

Audit logs are immutable.

D.06.11 Downstream Consumer: Gooru Topic Pool (Informational-only)

Purpose

Enable selected guides to write informational blog content without breaking keyword ownership rules. Topic availability is derived from keyword_map; no topic may be created outside SSOT.

Topic source (hard)

Topic Pool Generator MUST create topics ONLY from keyword_map rows where:

● intent_primary = Informational (or SSOT-approved informational mix)

● owner_page_type = Blog Page

● lock_state = unlocked

● status in (candidate | reserved | approved) as allowed by SEO policy

● not already owned by another canonical URL and not already claimed

Topic lifecycle

available → claimed → draft → submitted → published_noindex → approved_indexed (or returned_for_fixes)

Claim behavior (hard)

On claim:

● the system MUST write a deterministic reservation to keyword_map and lock ownership (audited),

● the topic becomes unavailable to other guides,

● claim TTL applies; if TTL expires without submission, the system releases the topic and unlocks the reservation (audited).

Indexation safety (hard)

All auto-published Gooru articles MUST default to noindex until SEO approval switches them to index.

D.06.12 Fail-Closed Dependencies (Hard)

● If SEMrush is unavailable: do not assign new ownership automatically; store keywords as candidate only and require manual review.

● If GSC is unavailable: dqcp_state=unknown; do not auto-promote new URLs from noindex to index.

● If collision_state=confirmed: block auto-creation and push to manual SEO review queue.

D.06.CONFIG — Implementation Configuration (Copy/Paste for Developers)

Purpose: Central, editable configuration for D.06 Anti-Cannibalization logic.
Rule: No thresholds, modifiers, or routing priorities may be hardcoded outside this config.

---

1) Numeric Thresholds (Hard Constants)

```
D06_THRESHOLDS:

  # Embedding similarity

  DUPLICATE_KEYWORD_SIM_THRESHOLD: 0.92        # >= means "duplicate phrasing" → cannot be owned by another URL

  VARIANT_KEYWORD_SIM_MIN: 0.75                # >= means "variant band"

  VARIANT_KEYWORD_SIM_MAX: 0.91                # <= means "variant band"

  CLUSTER_SIM_THRESHOLD: 0.78                  # >= means eligible as "secondary keyword" in same cluster

  NEW_TOPIC_SIM_THRESHOLD: 0.75                # < means "new topic"

  NEAR_PAGE_TOPIC_MIN_SIM: 0.62                # min similarity band for near-page "related topic" routing preference
```

Entity fit (keyword → entity)

ENTITY_FIT_MIN_SIM: 0.78 # keyword must match entity centroid to be eligible owner

Ambiguity guard for 1:1 product fit

PRODUCT_TOP2_SIM_MARGIN: 0.03 # if (top1 - top2) < margin → treat as ambiguous → fail 1:1 fit

Page-level duplicate / thin / overlap protection

PAGE_CONTENT_DUPLICATE_THRESHOLD: 0.90 # >= block publishing & push to manual queue

FAQ_QUESTION_DUPLICATE_THRESHOLD: 0.85 # >= reject new FAQ question (must rewrite or reuse canonical)

Near-page creation gates

NEAR_PAGE_MIN_VOLUME: 30

NEAR_PAGE_MIN_VOLUME_MAJOR_ATTRACTION: 10

NEAR_PAGE_KD_HUB_THRESHOLD: 70

HEAD_TOPIC_VOLUME_THRESHOLD: 2000

Near-page caps (per 5-week cycle)

NEAR_PAGE_MAX_NEW_PER_CITY_PER_CYCLE: 30

NEAR_PAGE_MAX_HUB_PER_CITY_PER_CYCLE: 5

NEAR_PAGE_MAX_NEW_PER_MAJOR_ATTRACTION_PER_CYCLE: 10

Coverage gates

NEAR_PAGE_MIN_PRODUCTS_STANDARD: 8

NEAR_PAGE_MIN_PRODUCTS_PLUS_ATTRACTION: 4

NEAR_PAGE_MIN_BUNDLES_STANDARD: 2

NEAR_PAGE_MIN_PRODUCTS_HUB: 20

NEAR_PAGE_MIN_PRODUCTS_PLUS_BUNDLES_HUB_PRODUCTS: 10

NEAR_PAGE_MIN_PRODUCTS_PLUS_BUNDLES_HUB_BUNDLES: 3

---

2) Page Type Priority (Winner Order)

D06_OWNER_PRIORITY:

Used when multiple candidates are valid after intent + fit checks

- PRODUCT

- PACKAGE_BUNDLE

- ATTRACTION

- NEAR_PAGE

- THINGS_TO_DO

- CITY

- REGION

- COUNTRY

- HOMEPAGE

- EXPLORE_MAP

- EXPERT

- BLOG

Hard rule: BLOG is last resort and only if intent is strictly informational and no programmatic owner is valid.

D06_MIXED_INTENT_RULES:

SEMrush Mixed stores (intent_primary, intent_secondary). Both must be allowed unless exception applies.

DEFAULT: REQUIRE_BOTH_ALLOWED

EXCEPTIONS:

Only these exceptions are allowed; otherwise block ownership and require manual review.

PRODUCT:

ALLOW_IF_INCLUDES: [TRANSACTIONAL] # still must pass 1:1 product fit

CITY:

ALLOW_ONLY_IF_PAIR: [NAVIGATIONAL, COMMERCIAL]

NEAR_PAGE:

ALLOW_ONLY_IF_PAIR: [INFORMATIONAL, COMMERCIAL]

PACKAGE_BUNDLE:

ALLOW_IF_INCLUDES: [COMMERCIAL, TRANSACTIONAL]

HOMEPAGE:

ALLOW_IF_INCLUDES: [NAVIGATIONAL]

EXPLORE_MAP:

ALLOW_IF_INCLUDES: [NAVIGATIONAL]

EXPERT:

ALLOW_IF_INCLUDES: [NAVIGATIONAL]

---

4) Keyword Modifier Dictionaries (Hard Routing Signals)

These lists are signals, not the intent source (intent comes from SEMrush).
They are used for entity detection, product 1:1 fit gating, and forbidden modifier checks.

D06_MODIFIERS:

TRANSACTIONAL:

- tickets

- ticket

- book

- booking

- buy

- price

- prices

- entry
- entrance
- admission
- skip the line
- skip-the-line
- guided tour
- private tour
- small group
- reservation
- reserve
- checkout
- discount code
- promo code

COMMERCIAL:

- best
- top
- compare
- comparison
- reviews
- rating
- worth it

- itinerary
- plan
- for families
- with kids
- romantic
- couples
- seniors
- budget
- cheap
- recommended
- alternatives
- vs

INFORMATIONAL:

- opening hours
- hours
- history
- how to
- tips
- guide
- what to see
- map

- directions
- address
- rules
- dress code
- accessibility
- wheelchair
- stroller
- queue time
- wait time
- time needed
- duration

NAVIGATIONAL:

- official
- official site
- website
- login
- contact
- customer service
- phone number
- email
- location

- near me

---

5) Bundle / Itinerary Pattern Rules (Hard)

D06_BUNDLE_PATTERNS:

Used to identify bundle/package intent candidates in addition to SEMrush intent.

REGEX:

- '(\\b1\\b|\\bone\\b)\\s*(day|days)\\s+in\\s+([a-z\\-\\s]+)'

- '(\\b2\\b|\\btwo\\b)\\s*(day|days)\\s+in\\s+([a-z\\-\\s]+)'

- '(\\b3\\b|\\bthree\\b)\\s*(day|days)\\s+in\\s+([a-z\\-\\s]+)'

- '([a-z\\-\\s]+)\\s+itinerary\\s+(\\b1\\b|\\b2\\b|\\b3\\b)\\s*(day|days)'

- '(weekend|long weekend)\\s+in\\s+([a-z\\-\\s]+)'

- '(\\bplan\\b|\\bitinerary\\b)\\s+([a-z\\-\\s]+)\\s+(\\b1\\b|\\b2\\b|\\b3\\b)\\s*(day|days)'

HARD_RULES:

- "If bundle pattern matches AND page type PACKAGE_BUNDLE is available for that city → prefer PACKAGE_BUNDLE over NEAR_PAGE for ownership, provided intent allowlist passes."

---

6) Forbidden Modifiers per Page Type (Hard Blocks)

If a keyword contains a forbidden modifier for the candidate page type, that page type is removed from candidates.

D06_FORBIDDEN_MODIFIERS_BY_PAGE_TYPE:

PRODUCT:

FORBID:

- best
- top
- compare
- itinerary
- plan
- what to do
- things to do
- attractions
- sightseeing
- opening hours
- history

ATTRACTION:

FORBID:

- tickets
- book
- booking
- buy
- promo code
- discount code
- price for tour

CITY:

FORBID:

- opening hours
- history of
- tickets
- skip the line
- admission

THINGS_TO_DO:

FORBID:

- opening hours
- history of
- official site
- login

NEAR_PAGE:

FORBID:

- checkout
- promo code
- discount code
- buy now

BLOG:

FORBID:

- tickets
- book
- booking
- buy
- skip the line
- promo code
- discount code

PACKAGE_BUNDLE:

FORBID:

- opening hours
- history of
- official site
- login

---

7) Head Topic Detection (Hard Patterns)

D06_HEAD_TOPIC_DETECTION:

REGEX:

- '^things to do in\\s+'

- '^best\\s+(things to do|tours|activities)\\s+in\\s+'

- '^([a-z\\-\\s]+)\\s+sightseeing'

- '^([a-z\\-\\s]+)\\s+attractions'

- '^best\\s+tours\\s+in\\s+'

RULES:

- "head_topic = true if SEMrush volume >= HEAD_TOPIC_VOLUME_THRESHOLD OR any REGEX matches"

---

8) Near-Page Gates (Hard Decision Functions)

D06_NEAR_PAGE_GATES:

INVENTORY_COVERAGE:

STANDARD_PASS_IF:

- "eligible_products_in_city >= NEAR_PAGE_MIN_PRODUCTS_STANDARD"

- "eligible_products_in_city >= NEAR_PAGE_MIN_PRODUCTS_PLUS_ATTRACTION AND major_attraction_count >= 1"

- "bundle_count >= NEAR_PAGE_MIN_BUNDLES_STANDARD"

HUB_PASS_IF:

- "eligible_products_in_city >= NEAR_PAGE_MIN_PRODUCTS_HUB"

- "eligible_products_in_city >= NEAR_PAGE_MIN_PRODUCTS_PLUS_BUNDLES_HUB_PRODUCTS AND

bundle_count >=
NEAR_PAGE_MIN_PRODUCTS_PLUS_BUNDLES_HUB_BUNDLES"

VOLUME:

STANDARD_MIN: NEAR_PAGE_MIN_VOLUME

MAJOR_ATTRACTION_MIN: NEAR_PAGE_MIN_VOLUME_MAJOR_ATTRACTION

FEASIBILITY:

HUB_IF:

- "kd >= NEAR_PAGE_KD_HUB_THRESHOLD AND head_topic == true"

TOPIC_DISTANCE:

PREFERRED_BAND_MIN: NEAR_PAGE_TOPIC_MIN_SIM

PREFERRED_BAND_MAX: CLUSTER_SIM_THRESHOLD # 0.78

ALLOW_NEW_TOPIC_IF:

- "similarity < NEAR_PAGE_TOPIC_MIN_SIM AND entity_route_exists == true AND
INVENTORY_COVERAGE passes"

CAPS:

CITY_STANDARD_MAX_PER_CYCLE:
NEAR_PAGE_MAX_NEW_PER_CITY_PER_CYCLE

CITY_HUB_MAX_PER_CYCLE: NEAR_PAGE_MAX_HUB_PER_CITY_PER_CYCLE

MAJOR_ATTRACTION_MAX_PER_CYCLE:
NEAR_PAGE_MAX_NEW_PER_MAJOR_ATTRACTION_PER_CYCLE

9) Deterministic Keyword Scoring Weights (Hard)

D06_SCORING:

WEIGHTS:

relevance_sim: 0.45

volume_norm: 0.25

kd_norm: 0.20

cpc_norm: 0.10

NORMALIZATION:

volume_norm: "log10(volume + 1)"

kd_norm: "1 - (kd / 100)"

cpc_norm: "log10(cpc + 1)"

TIEBREAKERS_ORDER:

- "higher relevance_sim"
- "higher volume"
- "lower kd"
- "higher cpc"

10) Product 1:1 Fit Gate (Hard)

D06_PRODUCT_ONE_TO_ONE_FIT:

REQUIRED:

- "intent includes TRANSACTIONAL (or is TRANSACTIONAL)"
- "cosine(keyword_embedding, top_product_embedding) >= ENTITY_FIT_MIN_SIM"
- "(top1_sim - top2_sim) >= PRODUCT_TOP2_SIM_MARGIN"
- "keyword is NOT head_topic (per D06_HEAD_TOPIC_DETECTION) unless keyword contains explicit POI + transactional modifier and still passes ambiguity gate"

FAIL_ACTION:

- "If fails → Product page removed from candidates; route via priority order"

---

11) FAQ Uniqueness Check (Hard)

D06_FAQ_UNIQUENESS:

VECTOR_STORE: "Qdrant"

QUERY_AGAINST:

- "product_faq_questions"
- "attraction_faq_questions"
- "things_to_do_faq_questions"
- "near_page_faq_questions"
- "blog_faq_questions"

REJECT_IF_SIM_GTE: FAQ_QUESTION_DUPLICATE_THRESHOLD

ON_REJECT:

- "block insert"

- "return canonical_question_id if available"

- "require manual rewrite"

---

12) Page Similarity Publish Block (Hard)

D06_PAGE_SIMILARITY_GUARD:

COMPUTE_ON:

- "pre-publish validation"

- "post-publish weekly audit (optional)"

REJECT_IF_SIM_GTE: PAGE_CONTENT_DUPLICATE_THRESHOLD

ON_REJECT:

- "block publishing"

- "push to seo_manual_review_queue"

- "require decision: merge | canonicalize | rewrite | noindex"

---

13) Reserved / Rejected Keyword Rules (Hard)

D06_KEYWORD_STATES:

RESERVED_REASON_CODES:

- "strategic_hold"

- "future_inventory_expected"

- "brand_protection"

- "conflict_requires_human"

REJECTED_REASON_CODES:

- "forbidden_modifier_for_all_candidates"

- "no_entity_route"

- "duplicate_of_locked"

- "intent_not_supported"

- "thin_inventory"

- "spam_or_policy"

---

14) Required Unit Test Pack (Developer Checklist)

Implement as test cases: (keyword, semrush_intent, entities_detected, metrics, similarity_context) → expected_owner, expected_state, expected_reason

Minimum required coverage: 60 tests

D06_TEST_SUITE_MINIMUM:

- "Duplicate threshold boundary: 0.91 vs 0.92"

- "Variant band boundaries: 0.74 / 0.75 and 0.91 / 0.92"

- "Entity fit boundary: 0.77 vs 0.78"

- "Product ambiguity guard: top1-top2 margin 0.02 vs 0.03"

- "Transactional POI → Product ownership"

- "Informational POI → Attraction ownership"

- "Commercial city head term → Things-to-do (fallback City)"

- "Mixed (Informational+Commercial) → Near-page if gates pass"

- "Mixed (Commercial+Transactional) → Package/Bundle if pattern matches"

- "Blog forbidden transactional modifiers → reject Blog ownership"

- "Near-page inventory gate pass/fail"

- "Hub near-page requirements"

- "Page similarity >= 0.90 publish block"

- "FAQ similarity >= 0.85 rejection"

---

15) Implementation Notes (Hard Requirements)

D06_IMPLEMENTATION_REQUIREMENTS:

- "All D06 thresholds must be loaded from config at runtime (no hardcoded constants)."

- "All routing decisions must write to keyword_map with last_decision_reason."

- "Any ownership change must be auditable and must set affected pages to OUTDATED (not cleared by notifications)."

- "No page becomes indexable unless it has an owned primary keyword in keyword_map."

- "Near-page creation must enforce caps and gates before URL creation."

D) Weekly Keyword Re-Sync Logic (Deterministic Decision Tree)

This section defines how Rosotravel weekly sync assigns new keywords to page types, prevents cannibalization, and safely expands coverage using SEMrush and Google Search Console (GSC) data.
All logic is deterministic, auditable, and subordinate to the canonical anti-cannibalization rules (D.06).
This weekly logic NEVER “guesses” intent, NEVER rewrites ownership rules, and NEVER bypasses keyword locks.

D.1 Inputs (weekly, deterministic)

Every weekly run processes the following inputs:

1. GSC Query Feed (weekly delta)

- New queries and rising queries per page and per entity scope.
- Stored as: keyword_text, impressions, clicks, CTR, avg_position, date_range.

2. SEMrush Keyword Metrics (for eligible queries/candidates)

- For each keyword line fetched: volume, KD, CPC, intent label (including Mixed).
- SEMrush intent is treated as authoritative.

3. keyword_map (single source of truth)

- Current ownership, lock state, canonical owner URL, timestamps.
- Used to prevent duplicates and resolve conflicts deterministically.

4. Entity embeddings + keyword embeddings

- Used for similarity checks and routing gates.
- All similarity checks use cosine similarity.

D.2 Intent Classification (hard rules)

The system does not invent intent types.
All intent classification is taken directly from SEMrush and treated as authoritative.

SEMrush intent values used by the system

- Informational
- Navigational
- Commercial
- Transactional
- Mixed (two intents)

Rules

- No manual overrides are allowed in the weekly sync.
- No custom intent definitions are allowed.
- Mixed must be stored as two explicit values: intent_primary + intent_secondary.

D.3 Page Type → Allowed Intent Ownership Matrix (hard enforcement)

Each page type has a strict allowlist of SEMrush intents it may own as SEO target keywords.

If a keyword's intent (or Mixed pair) is not allowed for a page type, assignment is blocked.

Allowed Intent Ownership (hard)

- Homepage: Navigational, Informational (Mixed allowed only if includes Navigational; excludes Transactional)
- Country Page: Navigational, Informational, Commercial (Mixed allowed only if both intents are allowed)
- Region Page: Navigational, Informational, Commercial (Mixed allowed only if both intents are allowed)
- City Page: Navigational, Informational (Mixed allowed only if Info+Nav; excludes Commercial/Transactional)
- Things-to-do Page: Commercial (Mixed allowed only if includes Commercial and excludes Transactional)
- Attraction (POI) Page: Navigational, Informational (no Mixed)

- Product Page: Transactional (Mixed allowed only if includes Transactional AND passes 1:1 product fit)
- Near-page: Commercial, Informational (Mixed allowed only if Info+Commercial; excludes Transactional)
- Blog Page: Informational only (no Mixed)
- Expert Page: Navigational, Informational (Mixed allowed only if Info+Nav)
- Package / Bundle Page: Informational, Commercial, Transactional (Mixed allowed only if it matches itinerary/bundle patterns; see D.5.3)
- Explore Map / AI Feed Context: Navigational, Informational (Mixed allowed only if includes Navigational; excludes Transactional)

D.4 Keyword Normalization (mandatory)

Before any matching or storage:

- lowercase
- trim spaces
- normalize punctuation
- remove duplicated whitespace
- store canonical keyword_text_normalized

All comparisons and map keys use normalized form.

D.5 Product Fit vs. “Broad Commercial” detection (deterministic)

D.5.1 What “Broad Commercial” means

“Broad Commercial” is not a SEMrush intent label.
It is a derived classification used only for routing decisions.

D.5.2 1:1 Product Fit Test (deterministic)

The system evaluates whether a keyword can map to exactly one Product URL.

A keyword FAILS 1:1 product fit if any of the following is true:

- the keyword implies multiple valid products
  example: “vienna palace tours”

- the keyword lacks a unique attraction + action pair
  example: “rome guided tour”

- the keyword can be satisfied by more than one product URL in inventory

- the keyword intent is comparison/selection rather than execution
  example: “best tours in rome”

Formal rule
IF intent == Transactional OR (Mixed includes Transactional)
AND product_fit == true
THEN candidate may route to Product Page ownership (subject to D.6 gates).

IF intent == Commercial
AND product_fit == false
THEN keyword is treated as Broad Commercial and may route to Things-to-do or Near-page (never Product).

D.5.3 Bundle Fit Test (itinerary-pattern gating)

Bundle pages may own itinerary-like queries only if the keyword matches bundle patterns.

A keyword PASSES bundle_fit if it matches at least one:

- duration pattern: “1 day / 2 days / 3 days / weekend / 48 hours / itinerary / plan”

- city-bound itinerary pattern: includes city name (or canonical city entity)

- itinerary intent pattern: “itinerary”, “route”, “plan”, “schedule”, “day-by-day”

Formal rule
IF (intent == Informational OR intent == Commercial OR intent == Mixed)
AND bundle_fit == true
THEN candidate may route to Package/Bundle Page.

Otherwise, itinerary-like keywords must route to City Page or Near-page according to intent matrix and similarity gates.

D.7 Deterministic actions by similarity class

D.7.1 Actions for Similarity ≥ 0.92 (Duplicate)

Meaning: the keyword is a pure rephrasing of an already owned keyword.

System actions:

- ignore keyword for ownership changes
- do not create new pages
- do not regenerate content
- log as duplicate_of = {owner_keyword_id}

D.7.2 Actions for 0.75 ≤ Similarity < 0.92 (Variant)

This range does NOT trigger page-type changes and does NOT allow scope expansion.
It enables only controlled micro-extensions.

Case A — Transactional variant (same Product intent)
Preconditions (ALL must be true):

- intent is Transactional OR Mixed includes Transactional
- product_fit == true
- owning Product already has a locked primary transactional keyword
- variant belongs to the same product entity cluster (keyword ↔ product similarity gate)

Actions:

- attach as secondary transactional keyword (max 2 per product)
- mark as locked in keyword_map
- allowed automatic content impact (ONLY):
  - meta description refresh (never meta title)
  - FAQ answer enrichment (never FAQ questions)
- forbidden:
  - new headings

- slug change
- canonical change
- long description rewrite

Example (allowed):

- primary locked: “schönbrunn palace tickets”
- variant: “schönbrunn palace ticket price”
- allowed: FAQ answer mentions pricing; meta description references price

Case B — Commercial/Informational variant
Actions:

- attach variant as “secondary_targets” to the existing eligible owner page type:
  - Things-to-do (Commercial)
  - Near-page (Info/Commercial)
  - City (Info/Nav only)
  - POI (Info/Nav only)
- may be used for FAQ expansion ONLY within the same page type uniqueness rules
- do not create a new URL

Hard rule:

- Product pages are NEVER updated for non-transactional variants.

D.7.3 Actions for Similarity < 0.75 (New Topic)

This represents a semantically distinct user intent.

Path 1 — Transactional new topic
System checks:

- does any single product match with 1:1 product_fit?

If YES:

- assign to that Product as primary or secondary (respecting max limits)
- lock keyword
- allow only the controlled impacts listed in D.7.2 Case A

If NO:

- set new_product_opportunity = true
- escalate to manual SEO/Product review
- no new page is created automatically

Path 2 — Commercial / Informational new topic
System checks (in order):

1. can it be owned by Things-to-do? (Commercial city activity head terms)
2. can it be owned by an existing Near-page? (Info/Commercial long-tail)
3. does it require a NEW Near-page? (only if passes creation gates below)
4. can it be owned by Bundle? (if bundle_fit == true)

If none apply:

- route to “Candidate Queue” (no URL, noindex draft optional)

D.8 Near-page creation gates (hard, required)

A new Near-page may be created only if ALL gates pass:

1. Inventory coverage gate

- must map to:

○ ≥ 8 eligible products in the city OR

○ ≥ 4 eligible products + 1 major attraction OR

○ ≥ 2 bundles (if bundles exist for that city)

2. Volume gate

- minimum volume: ≥ 30
- exception: ≥ 10 only for major-attraction scoped topics

3. Cost / scale caps

- per city per cycle: max 30 new near-pages
- hub near-pages: max 5 per city per cycle
- overflow goes to candidate queue

4. Transactional exclusion gate

- if intent is Transactional or Mixed includes Transactional → ineligible as Near-page owner

---

D.9 Structural safety controls (always enforced)

1. Keyword Locking

- any keyword owned by a Product Page is locked (1 keyword → 1 product URL)
- locked keywords cannot be reused by Near-pages, POI pages, City, Things-to-do, Blog

2. FAQ Uniqueness
   FAQ questions must be unique across:

- Product, POI, Things-to-do, Near-pages, Blog, Bundle
  A similarity check must run before acceptance.

3. Duplicate Prevention / Publishing Block
   If page similarity ≥ 0.90 against an existing indexable page:

- block publishing
- push to manual review queue
- SEO decides: merge, canonical, or keep separate

4. Canonical Mapping
   If overlap is later detected:

- weaker page receives canonical to stronger page
- pages are never deleted automatically

5. Human Validation Trigger
   Escalate for manual review if ALL are true:

- intent is Transactional (or Mixed includes Transactional)
- volume ≥ 10,000
- no 1:1 product match exists

D.10 keyword_map updates (single source of truth)

Every weekly action must write to keyword_map:

- keyword_text_normalized
- semrush_intent (or mixed pair)
- owner_url (or null if candidate)
- owner_page_type
- lock_status (locked/unlocked/reserved)
- similarity_class (duplicate/variant/new_topic)
- duplicate_of_keyword_id (if applicable)
- introduced_by_run_id
- timestamps

No system may bypass keyword_map.

D.11 Outputs (what weekly run produces)

At the end of the weekly run, the system produces:

- updated keyword_map (authoritative ownership state)
- candidate queue items (for future creation or manual decision)
- pipeline tasks (only when a new page draft is created or when keyword attachments require controlled regeneration)
- audit log entry for every change (who/what/when/why/run_id)

Weekly sync does NOT:

- regenerate content directly
- publish pages directly
- change slugs directly
- bypass locks

D.12 Strategic outcome (guaranteed priorities)

- Transactional intent is owned by Product (strict 1:1) and Bundle (bundle-fit only).
- Things-to-do owns city-level activity discovery (Commercial), not checkout intent.
- Informational coverage expands through POI, City, Near-pages, Blog (informational only).
- Near-pages grow long-tail safely under strict creation gates and caps.
- Cannibalization is prevented by locks, similarity gates, and canonical safety controls.
- All decisions are deterministic, auditable, and aligned with D.06.

4.12 Embeddings + Clustering + Topic Graph (Routing Confidence, Collision Prevention)

Tu ma być: embeddings po retrieval, similarity gates, clustering w scope destination, topic graph update triggers.

01. Purpose of This Layer (Full Definition)

The Semantic Layer is the central intelligence module that ensures all programmatic SEO, content generation, keyword allocation, anti-cannibalization, topic graph management, and dynamic near-page generation work consistently across the entire domain.

This layer provides:

1. Semantic Memory

It stores AI embeddings for:

- SEMrush keywords
- GSC queries
- product names
- attraction names
- cities, regions, countries
- titles, metas, H1/H2/H3
- long descriptions
- highlights
- FAQ questions
- blog topics
- near-pages
- dynamic emerging topics

This memory allows the system to:

- detect semantic similarity
- avoid duplicate content
- maintain clean architecture

- preserve topical consistency

---

2. Anti-Cannibalization Engine

This layer enforces:

- strict keyword ownership
- page-type constraints
- uniqueness of primary topics
- unique FAQ clusters
- no repeated queries across page types
- no competing URLs with the same intent

---

3. Topic Graph Engine

Builds a semantic graph for the entire site:

- country → region → city → attraction → product → near-pages
- backlinks: nearby / related
- entity clusters (e.g., “Rome museums”, “Paris skip-the-line tours”)
- long-tail intent clusters

---

4. Retrieval Layer for AI (RAG)

Supports LangChain by enabling:

- contextual retrieval based on embeddings
- avoiding regeneration of duplicate text
- enriching drafts with factual context

- improving rewrite consistency

---

5. Dynamic Near-Page Generation

After 5 weeks of data:

- compares new GSC queries
- identifies missing coverage
- detects semantic gaps
- automatically drafts new near-pages

D.02 Embedding Model Specification (Mandatory Technical Standard)

Model

- **OpenAI `text-embedding-3-large` (or newest available at implementation)**

Vector Size

- **3072-dimension float32 vectors**

Normalization Rules

Every item must be:

- **lowercased**
- **punctuation-stripped**
- **stopword-normalized**
- **whitespace-normalized**
- **UTF-8 normalized**

- **locale-aware (e.g., ü → ue where appropriate)**

Batch Rules

- **Keywords → batch of 200**
- **FAQs → batch of 100**
- **Page text → batch of 50**
- **Long-form paragraphs → 1 at a time**

When Embeddings Are Generated

Event
→ Generate embeddings?
RAW ingestion
→ ❌ No (RAW is stored only)
After factual enrichment
→ ✅ Yes
After keyword allocation
→ ✅ Yes
After AI drafts the FINAL page
→ ✅ Yes (final authoritative vector)
After publishing
→ Final embedding override
Weekly optimization
→ Only pages changed

D.03 Qdrant Collections (Vector Database Architecture)

Qdrant is the single semantic storage engine for all embeddings.

Each collection has:

- **full payload schema**
- **cosine similarity metric**
- **indexes on text + metadata fields**
- **timestamps**

- **versioning**

D.03.1 Collection: `keywords_vectors`

Used for:

- **similarity checks**
- **keyword ownership**
- **cannibalization prevention**
- **near-page detection**

Field	Type	Description
keyword	**string**	**raw keyword**
normalized_keyword	**string**	**normalized form**
embedding	**vector**	**3072-dim**
intent	**enum**	**navigational, informational, commercial, transactional, mix of 2**
language	**string**	**ISO**
volume	**int**	**SEMrush**
kd	**int**	**keyword difficulty**

cpc	**float**	**CPC**
recommended_page_type	**enum**	**product / attraction / things-to-do / near-page / blog**
assigned_to_url	**string/null**	**locked owner**
assigned_type	**enum**	**product / attraction / near-page / blog**
timestamp_created	**datetime**	**initial ingestion**
timestamp_updated	**datetime**	**last update**

Indexes required:

- **keyword (text)**
- **normalized_keyword (text)**
- **assigned_to_url (text)**
- **intent (enum)**

D.03.2 Collection: `pages_vectors`

Used for:

- **detecting duplicates**
- **canonical rules**
- **deciding where topics belong**
- **optimizing internal linking**

Field	Type
url	**string**
entity_type	**enum**
h1	**string**
primary_keyword	**string**
secondary_keywords[]	**array**
embedding	**vector**
impressions_last_30d	**number**
ctr_last_30d	**number**
bounce_rate	**number**
conversion_rate	**number**
last_modified	**datetime**

D.03.3 Collection: `faq_vectors`

Used to prevent duplicate questions across:

- **specific products**

- **attractions**
- **things-to-do pages**
- **near-pages**
- **blog**

Field	Description
question	**raw question**
normalized_question	**cleaned**
embedding	**3072-dim**
assigned_to_url	**owner**
intent_source	**SEMrush / GSC / AI**
locked	**bool**

D.03.4 Collection: `poi_vectors`

Used for:

- **Explore attraction map**
- **Nearby attractions**
- **LLM datasets**

Field	Description
poi_id	**attraction internal ID**
poi_slug	**seo slug**
name	**POI name**
country	**string**
region	**string**
city	**string**
lat	**float**
lng	**float**
categories	**array**
embedding	**vector**

D.04 The Topic Graph (Formal Specification)

The Topic Graph defines the semantic architecture of the entire Rosotravel domain.

Each node contains:

- **topic_id**
- **primary_keyword**
- **secondary_keywords[]**
- **representative_embedding (centroid)**
- **assigned_entity_url**
- **entity_type**
- **funnel_intent**
- **children_topics[]**
- **parent_topic**

Relationship Types

Relationship	Meaning
isPartOf	**hierarchical parent**
hasPart	**children topics**
isSimilarTo	**semantic similarity (>0.75)**
overlapsWith	**cannibalization risk (>0.92)**
nearby	**geo-distance**
related	**editorial or category-based**

Graph Update Triggers

- **batch publication**
- **new near-pages**
- **weekly keyword resync**
- **major landing page changes**
- **new region/city/attraction added**

D.05 Similarity Thresholds

Keyword similarity

D.05.1 Keyword Similarity Thresholds

Keyword similarity (cosine)	Action (mandatory)
≥ 0.92	**Duplicate phrasing** → block + ignore (no fetch, no metadata update, no new URL)
0.78 – 0.92	**Variant phrasing** → attach ONLY to existing **owner_url** as secondary keyword (no new URL, no new page type)
0.62 – 0.78	**Borderline topic** → create `topic_candidate` requiring **owner decision**: (A) absorb as FAQ/section on owner OR (B) near-page draft only if near-page gates pass
< 0.62	**New topic** → eligible to create a new page draft (routing rules + intent allowlists + index gates apply)

Hard rule: No new URL may be created from the **Variant band (0.78–0.92)**.

D.05.2 Page Similarity Thresholds

Page similarity (cosine)	Action (mandatory)

≥ 0.90	**Duplicate page** → block publishing as indexable + force **winner selection** (one owner_url). Loser must be `noindex,follow` + canonical to owner (or 301 if deprecated).
0.70 – 0.90	**Same cluster** → do NOT create a new indexable page. Unify by: (A) merge content into existing owner OR (B) publish as non-indexable variant (noindex + canonical).
0.40 – 0.70	**Related but allowed** → allowed to publish if intent ownership does not overlap. Must still pass keyword ownership + canonical rules.
< 0.40	**Unrelated** → no similarity constraint (standard governance rules apply).

Page similarity — scope rule (add this 1-liner under the table)

Page similarity MUST be computed on "SEO body only" (exclude global templates, navigation, boilerplate policy blocks, repetitive UI sections).

D.08 Error Handling & Recovery

1. Embedding Failures

Retry 3 times → log → skip entity but keep version.

2. Qdrant Failure

Fallback to cached embeddings.
Disable:

- near-page creation
- keyword allocation

3. Keyword Collision

At publish time:

- abort publish
- push to manual review queue

D.09 Performance Requirements

- **Qdrant top-k search: <200ms**
- **Embedding lookup: <150ms**
- **Topic Graph rebuild: <2 minutes per country, nightly**
- **Maximum embedding batch: 200 items**
- **Write throughput: 8,000 vectors/min**

A.10.7 Topic Graph Integration (SEO + LLM)

All entities and owned keywords MUST be represented in the **Topic Graph**.

Each node contains:

- topic_id
- primary_keyword
- secondary_keywords[]
- entity_id
- entity_type
- representative_embedding
- parent_topic
- children_topics[]

The Topic Graph is used for:

- internal linking logic
- near-page candidate discovery
- LLM grounding and retrieval
- content gap detection

No page may be generated, indexed, or exposed to LLMs unless it exists as a valid node in the Topic Graph.

A.10.4 Keyword Ownership Mapping (keyword_map)

The keyword_map table defines **exclusive ownership** of keywords across the entire system.

Each keyword record MUST include:

- keyword
- normalized_keyword
- intent (SEMrush-provided)
- assigned_entity_id
- assigned_entity_type
- assigned_url
- locked (true/false)
- source (SEMrush / GSC / manual)
- created_at

Rules:

- A keyword may be owned by **only one entity**.
- Transactional keywords may only be owned by **Product pages**.
- Broad commercial keywords may only be owned by **City / Region / Country pages**.
- Near-pages may own **long-tail informational or commercial keywords only**.
- Once locked, a keyword cannot be reassigned without manual SEO override.

This table is the **primary anti-cannibalization mechanism**.

J.08 Automatic “Nearby” & “Related” Blocks (How They Work)

Nearby (Geo-based)

Used on:

- Attraction pages
- City pages
- Explore Map
- Sometimes product pages (if multi-POI)

Logic:

- Use `poi_vectors` + lat/lng to find top N closest POIs within a configurable radius (e.g. 5–10 km).
- Filter by `status = published`.
- Do NOT show more than 6 items to keep UX clean.
- Cache per POI to minimize load.

Related (Semantic-based)

Used on:

- Product pages
- Near-pages
- Blog posts
- Packages

Logic:

- Use `pages_vectors` with language-specific collection.
- Query top 30 by cosine similarity, then filter by:
    - same city OR same attraction OR same category
    - NOT same URL
    - NOT higher than similarity threshold for duplicates (`>=0.90`)

- From remaining results, choose:
    - 3–6 for “Similar tours”
    - 2–3 for “You may also like” near-pages

4.13 Factual Enrichment (Google Places + Wikidata): Cost-Controlled Refresh + factual_hash

Tu ma być: priorytety źródeł, cadence per field, factual_hash, brak AI factual rewrites, kiedy factual changes triggerują schema refresh

Factual / Local

- **Google Places API** – opening hours, address, phone, website, rating.
- **Wikidata API**

Global Schema & Factual Rules

- Schema is generated only from tokens + Schema Builder Table.
- AI cannot fabricate coordinates, times, links, or offers.
- AI may only paraphrase human-readable text, never factual values.
- Must include hasPart / isPartOf where applicable.
- Must include breadcrumb, inLanguage, canonical, and alternate/hreflang.

Editorial Consistency Rules

Applicable to **all page types**, enforced by validator:

- Exactly one <h1>
- Logical hierarchy: H2 → H3 → H4 (never skip levels)

- No empty sections
- No duplicate highlight bullets
- No duplicated FAQ across pages
- Internal links must point to canonical URLs
- No soft 404 pages (intro must be ≥ 160 chars)

FAQ Rules (Integrated Into Chapter 1)

FAQ Count by Page Type

Page Type	Count
Product	10
Attraction	7
City	9
Region	7
Country	7
Things to Do	7
Near Page	7
Blog	5

FAQ Purpose

- Answer user intent
- Feed rich results
- Provide factual clarity
- Avoid cannibalization

FAQ Data Sources

- **Products:** Feed → SEMrush → GSC
- **Attractions:** Google Places + Wikidata + Tourism boards
- **Cities:** Official tourism + GSC volume
- **Near Pages:** GSC new queries + SEMrush long-tail
- **Blog:** Informational intent only

FAQ Rules

- No duplication between pages
- No contradictory facts
- No promotional language
- Product FAQ must match product's actual inclusions / meeting point / ticket policy
- Attraction FAQ must match opening hours, ticketing, rules, accessibility
- FAQ answers must be **220–350 chars**
- FAQ question similarity < **0.85** (embedding safety check)
- FAQ answers must be rewritten for each language with semantic preservation
- Locked FAQs can never be overwritten by AI

Universal JSON Output Rules (Updated)

Applied across *all page types*.

```
{
  "meta_title": "",
  "meta_description": "",
  "h1": "",
```

```
  "sections": {
    "intro": "",
    "body": "",
    "snippet": ""
  },
  "highlights": ["..."],
  "options": [
    {
      "name": "",
      "description": ""
    }
  ],
  "faq": [
    {"q": "", "a": ""}
  ],
  "schema": {
    "type": "REFERENCE_ONLY",
    "tokens": []
  }
}
```

E.06 Factual Data Enforcement Rules

Factual fields are NEVER AI-generated:

- opening hours
- address
- phone
- coordinates
- place_id
- website
- holiday openings
- price baseline
- availability

They come ONLY from:

1. Google Places API
2. Wikidata
3. Wikipedia (fallback, sections only)

If missing → AI generates neutral text:

> "Opening hours vary depending on the season; verify current hours before visiting."

4.14 Diff Engine → Pipeline Actions (Compare vN vs vN-1 → MINOR/MEDIUM/MAJOR → enqueue)

Tu ma być: severity + affected_blocks + pipeline_actions[] + twarda zasada: tylko MAJOR odpala AI drafting.

7) Differentiation Engine (RAW vN-1 → RAW vN comparison) — aligned to 3 hashes

The Differentiation Engine compares RAW versions using only:

- `content_hash`
- `factual_hash`
- `media_hash`
  plus manual locks.

7.1 Inputs

- RAW_v(N-1) hashes: content_hash, factual_hash, media_hash
- RAW_vN hashes: content_hash, factual_hash, media_hash
- manual lock flags

7.2 Outputs

- `change_severity = MINOR | MEDIUM | MAJOR`

- `affected_blocks[] = content | factual | media`
- `pipeline_actions[]`

7.3 Severity definition (deterministic)

MINOR

- No hash changed (`content_hash`, `factual_hash`, `media_hash` unchanged)
  → No pipeline actions (except optional cache refresh / housekeeping)

MEDIUM

- `factual_hash` changed and/or `media_hash` changed
- AND `content_hash` unchanged
  → Update factual/media downstream, no AI drafting

MAJOR

- `content_hash` changed
- AND affected content fields are not manual-locked
  → Eligible to enqueue AI drafting modules (selective)

Hard rule: Only MAJOR changes can enqueue AI drafting. MEDIUM/MINOR must not trigger AI drafting.

---

8) Diff Logic Table (system actions) — aligned to 3 hashes

Change detected (hash)	System action (deterministic)	AI drafting allowed?
No hash changed	Skip pipeline; keep current Published winners	No

factual_hash changed	Update factual blocks + factual schema + factual-derived FAQ answers (answers only) + maps/relations	No
media_hash changed	Re-run media optimization (e.g., Cloudinary), update refs/thumbnails	No
content_hash changed AND not locked	Regenerate only changed content sections via AI (dependency-based), keep locks	Yes (selective)
content_hash changed BUT locked	Keep manual content; raise “lock prevented update” flag; allow non-locked dependent updates only	No

Important: The system NEVER regenerates the entire entity unless explicitly forced by admin action.

---

9) Trigger Logic (What starts the pipeline?) — aligned to 3 hashes

9.1 New snapshot ingested (External API / internal / Google / Wikidata / media)

Triggers:

- raw_version++ only if content_hash/factual_hash/media_hash changes
- Diff Engine run
- Selective downstream processing based on severity

9.2 Manual marketer actions (no version increments)

Manual actions can trigger selective regeneration for the current RAW vN without changing raw_version, e.g.:

- Regenerate selected AI blocks (title/snippet/long desc/FAQ) using current prompts

- Rebuild Published Record (winner selection)
- Re-translate from Published EN winners (if enabled)

Hard rule: manual triggers must respect dependencies (schema/meta/translation depend on winners).

9.3 Scheduled tasks

- Daily: external refresh + factual refresh (only increments RAW vN if hashes change)
- Weekly: GSC/SEMrush/FAQ monitoring (does not touch RAW vN directly; may create alerts/tasks)
- Post-publish: near-page generation (separate module; does not affect RAW vN)

K) Data Source Labels in Tables (naming convention, no Viator endpoint details)

To avoid mixing scopes, tables must use the following source labels:

- External API (OTA) — content (details in OTA scope; do not list endpoint fields here)
- External API (OTA) — operational (details in OTA scope; do not list endpoint fields here)
- External API (OTA) — factual (only if used; otherwise prefer Google/Wikidata for indexable facts)
- Google Places API (paid; factual truth for name/address/hours/coords/phone/website/categories)
- Wikidata API (factual/historical/relations/aliases/classification; lower cost; cached aggressively)

L) Cost-Safe Fetch Strategy (Google Places + Wikidata) — to avoid unnecessary spend

L.1 Core rule

Google Places and Wikidata must be fetched using cache + TTL + dedupe so the system never refetches the same entity repeatedly because many products reference it.

L.2 Entity-level caching (mandatory)

For each entity (City / Attraction):

- store `google_fetched_at`, `wikidata_fetched_at`
- store `google_cache_payload_hash`, `wikidata_cache_payload_hash`
- reuse cached results across all products referencing the same entity

L.3 Default TTL policy (editable config)

- Google Places TTL (cities/attractions): 7–30 days (configurable; default 14 days)
- Wikidata TTL: 30–90 days (configurable; default 60 days)

Hard rule: do not refresh if TTL not expired unless forced by an admin action.

L.4 Dedupe rule (mandatory)

If 200 products reference the same attraction:

- fetch Google/Wikidata once
- apply the result to the shared entity record
- never multiply calls per product

L.5 Batch + queue discipline

- Requests are queued
- Fetches are batched by entity type and locale
- Backoff and retry logic is applied centrally (not per product)

---

5) API Rate Limits or Outages (System behavior)

5.1 Affected APIs

- External API (OTA feeds)
- Google Places
- Wikidata
- SEMrush
- Google Search Console

5.2 Handling (hard rules)

- Queue ingestion (no blocking writes)
- Retry with exponential backoff
- Use last valid cached data (serve existing winners safely)
- No data loss (idempotent ingestion + durable queue)
- Non-blocking CMS banner (visible alert; does not break UI workflows)

5.3 Indexation impact (hard rule)

- No automatic index changes during outages
- Existing indexable pages remain indexable

No auto-demotions or emergency noindex unless explicitly triggered by manual SEO/admin action

Error Handling & Fallback for Diff Engine:

Data domains (must be treated separately)

A) CITY/ATTRACTION factual domain (place truth)

- coordinates, address, categories, official website, phone, opening hours (where available), identifiers.

B) PRODUCT offer / operational domain (booking truth)

- duration, meeting point, cancellation policy, inclusions/exclusions, product options, confirmation type.

C) PRODUCT realtime offer domain (volatile signals)

- availability refs, start times, price_from, currency.

Hard rule:

- realtime_offer changes MUST NOT force raw_version increments.
- offer/realtime are handled by offer_version / realtime_offer_version domains.

2.1 “Acknowledged” is not “resolved”

➢ Clicking or acknowledging a notification only marks it as seen.
➢ The underlying state flag remains until the system confirms resolution.

2.2 Registry:

➢ RAW_CONTENT_MISSING, FACTUAL_MISSING, OFFER_MISSING, RELATIONSHIP_MISSING, AI_FAILED, MEDIA_REVIEW_REQUIRED

Stamp: for full information ref. to 4.7

3.1 Definition

Missing or invalid supplier-provided content seeds such as:

- title_seed
- short_description_seed
- long_description_seed
- highlights_seed
- itinerary_overview_seed

Note:

- supplier raw itineraries / raw reviews / supplier media are not used for indexing and are not required for SEO assembly.

3.2 RAW behavior (always)

- A new RAW snapshot is stored (raw_version++ only if the snapshot actually changed per hashes).
- Missing fields are flagged:
    - raw_missing_fields[] = {field_name...}
    - raw_missing = true

3.5 Allowed AI remediation

AI may regenerate ONLY these blocks (if enabled for the entity/page):

- product overview / long description (AI-owned)
- highlights (AI-owned)
- FAQ (AI-owned)
- itinerary_summary (high-level narrative, never itinerary steps)

AI must NOT:

- recreate inclusions/exclusions
- infer prices, duration, meeting point, cancellation policy, start times

4) Missing or Inconsistent FACTUAL Data — CITY/ATTRACTION only

4.1 Definition

Missing/invalid place fields:

- coordinates, address, opening hours (optional), phone, official website, categories, identifiers.

Hard rule:

- AI must never generate or “correct” factual values.
- AI may only describe factual data that exists and is validated.

5) Missing OFFER for Diff engine

5.1 Definition

Missing/invalid booking offer fields (medium cadence, offer_version):

- duration
- meeting point (or verified pickup/meeting instructions)
- cancellation policy
- inclusions/exclusions
- product options structure (if product requires options)
- confirmation type

6) Missing REALTIME Offer Fields (Availability/Pricing) — PRODUCT realtime domain - DIFF engine

6.1 Definition

Missing realtime fields (high cadence, realtime_offer_version):

- price_from, currency
- availability_refs
- start_times[] (optional)

6.2 Handling rules (do not blow up RAW)

- realtime_offer updates must not touch raw_version.
- Store realtime in separate cache/store and update realtime_offer_version.

6.3 Job rules

➢ Retry realtime fetch with exponential backoff.

7) Missing or Broken RELATIONSHIP Fields (Hierarchy / Mapping) — ALL entity types - Diff Engine

7.1 Definition (contract-level failure)

Any of the following:

- missing canonical parent_relations required by hierarchy
- missing relationship_edges[] for hierarchy graph
- PRODUCT missing canonical city relation (PRODUCT_IN_CITY)
- ATTRACTION missing canonical city relation (ATTRACTION_IN_CITY)
- conflicting dedupe keys causing ambiguous parent mapping

7.2 Fallback order (deterministic)

1. Use last_valid_cache relationship snapshot if present and still consistent.
2. Attempt re-resolution using available IDs (Viator graph, Google place, Wikidata) within budget:
    - resolve city
    - resolve parent chain country→region→city

9) AI Generation Failures (Any entity where AI blocks exist) - Diff engine

9.1 Cases

- timeout, empty output, duplication violations, hallucination/policy failure.

9.2 Fallback order

1. Keep last valid AI output for that block (if exists).

11) Regeneration Triggers & What is allowed to regenerate (UI + Pipeline alignment) - diff engine

11.1 What may trigger regeneration

A) Diff Engine signals (based on version domains + hashes)

- raw_version change:
    - content seeds changed → content AI blocks may be queued (only those not locked)

    - factual/media/relationships changed → schema/maps/relations update; no AI narrative unless policy says required

- offer_version change:
    - offer schema update; may trigger FAQ answer refresh (answers only) if derived from offer fields

- realtime_offer_version change:
    - UI refresh only (price/availability); no SEO narrative regeneration

B) Manual initiative (not only “errors”)

- Admin/SEO may trigger regeneration of selected blocks in Pipeline, even if not flagged as outdated.

11.2 Pipeline caps exists per block and must be enforced

11.3 Locks (global rule)

- Locks are per block, not global.
- Locked blocks are skipped by regeneration automatically.
- Unlock requires role + reason + audit event.

4.15 AI Drafting + Schema Drafting + Validation Gates (Allowed Blocks Only)

Tu ma być: allowed blocks, schema z verified winners, walidacje (uniqueness/policy/schema), status outputs.

01. Global Structured Data Requirements

Structured data is a hard requirement for SEO and LLM visibility. Every **indexable** page type must emit **JSON-LD** that is consistent with the rendered UI and the canonical URL.

a. **Mandatory rules (apply to all indexable page types):**

1. **JSON-LD must be present** on every indexable page type.
2. **Stable @id policy:** Every emitted schema node must use stable `@id` values derived from the canonical URL (e.g., `{{Canonical_URL}}#article`, `{{Canonical_URL}}#tour`, `{{Canonical_URL}}#person`).
3. **Language declaration:** Every schema block must include `inLanguage` matching the page locale.

4. **mainEntityOfPage:** Every page must declare `mainEntityOfPage` pointing to `{{Canonical_URL}}`.
5. **Publisher identity:** Every content schema (Article/Guide/Expert content) must include `publisher` as `Organization` (Rosotravel) with `logo` and `sameAs[]`.
6. **Breadcrumbs on hierarchical pages:** Country/Region/City/Things-to-do/Attraction/Tour/Package must emit a JSON-LD `BreadcrumbList` that matches visible breadcrumbs.
7. **No hidden schema:** Do not emit schema for content that is not rendered (e.g., FAQPage without an FAQ block in UI is forbidden).
8. **Validation Gate:** If required schema fields are missing or invalid for a page type, publishing must be forced into `noindex` with a CMS alert, per governance rules.

b. Author attribution

➢ Schema requirements (Blog/Guides/Expert editorial pages):

1. Expert/Guide must include `author` as `Person` and reference the author profile via `@id`.
2. If the page emits a separate `Person` schema block, it must use the same `@id` as `Article.author.@id`.
3. `publisher` must be `Organization` (Rosotravel) with `logo` and `sameAs[]`.

c. Definition of Done (DoD):

1. 100% of pages per supported page type pass schema validation rules (no critical errors).
2. Schema content matches UI content for key blocks (FAQ, rating/count, author line, published/updated dates).
3. Publishing is blocked when a mandatory schema fails.

02. Date Metadata Policy (Published / Updated / Reviewed)

All editorial content and LLM-facing content must expose publication and update timing signals in both **schema** and **UX**.

a. Required schema fields (content pages):

- `datePublished` = `{{Date_Published_ISO_US}}`
- `dateModified` = `{{Date_Modified_ISO_US}}`
- `lastReviewed` = `{{Last_Reviewed_ISO_US}}`

ISO format requirement (US profile):

- Use ISO 8601 with US time zone offset, e.g. `2026-02-14T09:30:00-05:00`.

- If the site uses DST, offsets may be `-04:00` during daylight time.
- Date-only values are allowed only when time is not available (e.g. `2026-02-14`), but preferred format is full timestamp with offset.

b. DoD:

03. Content pages cannot be published as indexable unless `datePublished` and `dateModified` are present and valid.

04. The visible date line matches schema values.

05. AI DRAFTING LAYER (FULL SPECIFICATION)

E.01 Purpose of This Layer

The AI Drafting Layer converts structured data (RAW product feed, factual data, keyword ownership, semantic context) into:

- product pages
- attraction pages
- city/region/country pages
- category pages
- things-to-do pages
- immediate near-pages
- expert pages
- blog topic templates (content written manually)
- FAQs
- meta titles + descriptions
- schema JSON-LD
- LLM snippets
- alt text for images

This layer must:

- respect keyword ownership (no cannibalization)
- respect manual locks
- respect factual data (no hallucinations)
- integrate semantic retrieval via Qdrant
- produce content that passes AI-detector naturalness checks
- use a strict template hierarchy
- output machine-readable drafts

This is **the single most critical layer** for programmatic SEO quality.

E.01.1 Extended Purpose: Dynamic Experience Engine + Human Face Outputs (MVP)

The AI Drafting Layer must also generate and maintain additional non-SEO variants and “human face” text blocks that power the Dynamic Experience Engine and Rosotravel Live. These outputs must never alter canonical SEO text (H1/H2 body, meta, schema, keyword ownership).

This layer produces:

- Segment-ready UI variants (Dynamic Snippets, Review Summaries, Ordering Profiles hints) used only for on-page personalization and module ordering.
- Human-face narrative blocks tied to real operational data (Last Tour Snapshot copy, “This Week in City” copy, guide micro-bios), without fabricating facts.

All dynamic outputs must be:

- stored per entity in CMS fields,
- versioned and lockable like any AI content,
- safe by default (fallback to default variants when signals are missing).

E.02 Inputs Into AI Drafting

AI generation ONLY starts after these inputs are ready:

1. `raw_data_enriched.json`

Contains:

- product metadata
- attraction metadata
- factual API data
- options
- operational data (price, duration, languages)

2. Keyword Map (Locked)

Includes:

- primary keyword
- secondary keyword(s)
- forbidden keywords (keyword ownership from other pages)
- funnel type
- semantic cluster ID

3. Embedding Context (Qdrant Retrieval)

Used by LangChain, including:

- similar pages
- related attractions
- related tours
- city context
- FAQ clusters

4. Prompt Templates Library

One template per entity type:

- Product template
- Attraction template
- City template
- Category template
- Things-to-do template
- Near-page template
- Expert profile template
- Package template

Stored as:
`/prompts/templates/{entity_type}.json`

5. CMS Field Locks

Per-section:

- title_locked
- short_description_locked
- long_description_locked
- highlights_locked
- itinerary_locked
- faq_locked
- meta_locked
- schema_locked

6. SEO Tokens (Global Token Library)

E.g.:

- {{Attraction_Name}}
- {{City_Name}}
- {{Primary_Keyword}}
- {{Duration}}
- {{Included_List}}
- {{FAQ_1_Question}}

In addition to E.02 inputs, AI Drafting must receive these sources when available:

1. session_intent_config.json (SYSTEM)
   Contains:

- supported segments: according to **4.8.9 Stage F — Profile model**
- segment inference rules (keyword/referrer/behavior mappings)
- locale weighting rules (optional, configurable, not hard-coded)
- allowed dynamic sections per page type

2. reviews_aggregates.json (SYSTEM)
   Contains:

- rating distribution
- topic clusters (precomputed, e.g., guide_quality, pace, logistics, value, accessibility)
- per-segment cluster weights (family vs couple vs solo etc.)
- most_recent_reviews_by_tour_date (if available)

3. tour_run_events.json (SYSTEM, for Rosotravel-owned where available)
   Contains:

- last_departure_datetime
- group_size (if available)
- guide_id (if available)
- post_tour_mood_score (if available)

- media_ids tagged with tour_date (if available)

4. guide_profiles.json (CMS / SYSTEM)
   Contains:

- guide_id, display_name, photo, languages, credentials, short_bio
- allowed products mapping (product_id ↔ guide_id)
- fallback “Rosotravel Guide Team” profile for OTA imports

5. bundles_city_index.json (SYSTEM)
   Contains:

- bundle_id list for city (1/2/3-day) per segment if present
- bundle curated_by_type, curated_by_name
- bundle highlights, trip_dna, short_description
  Used only for “bundles preview” sections and ordering hints on city/attraction pages.

E.03 Output of the AI Drafting Layer

AI generates a structured draft:

`draft_content_vX.json` containing:

A. Content Sections

- Title
- Short description
- Long description
- Highlights (min 5)
- Itinerary (if applicable)
- Expert Tips
- Local Insights

- Pro Tip

B. SEO Sections

- H1
- H2/H3 hierarchy
- Meta title
- Meta description

C. FAQ Block

- 5–12 questions
- Structured for schema

D. Alt Text Block

- 1 alt text per image

E. Snippets for LLMs

- 240–300 character neutral summary

F. Schema JSON-LD (with URL placeholders)

G. Internal Linking Suggestions

- list of recommended related URLs
  (based on embeddings)

H. Entity Relations

- isPartOf
- hasPart
- nearby
- related

03.1 Extended Output: Variant Blocks + Human-Face Blocks (Stored in CMS)

The AI Drafting Layer must generate and store these additional outputs per entity, with the same lock + versioning behavior as other AI fields.

A) Dynamic Snippet Variants (MVP, mandatory where snippet exists)
Stored in CMS fields per page:

- first_time_visitor, (it is default audience)
- snippet_family_traveler,
- couple_traveler,
- comfort_easy_pace_traveler,
- solo_social_traveler,
- interest_deep_dive_traveler,
- active_adventure_travele
- snippet_explorers
  Constraints:

- each 180–260 chars

- 1 paragraph

- summarizes what/where/why it matters

- not used in meta; not indexation-critical

- if no segment detected at runtime → snippet_default

B) Review Summary Variants (MVP, where reviews exist)
Stored in CMS fields per page:

- review_first_time_visitor, (it is default audience)
- family_traveler,
- couple_traveler,
- comfort_easy_pace_traveler,
- solo_social_traveler,
- interest_deep_dive_traveler,
- active_adventure_traveler

Constraints:

- short theme summary derived from review clusters

- no direct quotes
- cannot invent facts; must reflect cluster evidence
- if segment variant missing → fallback to default

C) Ordering Profile Hints (MVP, non-SEO ordering only)
Stored in CMS fields per page (as structured JSON or normalized tables):

- ordering_first_time_visitor, (it is default audience)
- family_traveler,
- couple_traveler,
- comfort_easy_pace_traveler,
- solo_social_traveler,
- interest_deep_dive_traveler,
- active_adventure_traveler

- ordering_profile_explorers
  Defines:

- ranking weights to apply in listing modules (top tours, related tours, bundles block, category tiles)

- optional pin/boost/avoid lists (editor overrides take precedence)
  Constraints:

- must never change URL creation, canonical rules, or keyword ownership

- affects only ordering of already-eligible items

D) Human-Face / Live Text Blocks (MVP)

1. Last Tour Snapshot Copy (product page, if tour_run_events available)
   Stored fields:

- last_tour_snapshot_title

- last_tour_snapshot_summary_line

- last_tour_guests_loved_topics[] (2–3)

- last_tour_photo_strip_caption (optional)
  Constraints:

- factual fields (date, guide name, group size) come only from tour_run_events / reviews feed

- AI may only summarize themes (“guests loved…”) from that tour_date window
  Fallback:

- if no recent data → store and render last_30_days_summary instead (separate field)

2. “This Week in {{City}}” Copy (city page) / “Recent at {{Attraction}}” Copy (attraction page)
   Stored fields:

- live_week_summary_line

- live_week_guests_loved_topics[] (2–3)

- live_week_top_guides_line (optional, if guide_ids available)
  Constraints:

- counts/ratings derived from aggregates; AI only summarizes themes and wording

3. Guide Micro-Bio Blocks (product page)
   Stored fields:

- primary_guide_card_copy (if guide known)

- guide_team_fallback_copy (if OTA import / unknown guide)
  Constraints:

- credentials and experience years must come from guide_profiles, not AI invention

---

E.04 Prompt Orchestration (LangChain)

Generation always uses a **three-layer chain**:

Chain 1 — Retrieval Chain

Fetches:

- factual blocks
- keyword cluster
- similar content context
- nearby attractions
- entity graph relations
- competitor snippet patterns (SEMrush SERP extract)

This ensures:

- no hallucinations
- SEO-consistent structure
- correct factual values

Chain 2 — Drafting Chain

Combines:

- global template
- keyword map
- tokens
- retrieved context

Example internal call:

```
generate_product_page(
  tokens,
  keyword_set,
  factual_block,
  similar_pages_context,
  template="product_v3"
)
```

Chain 3 — Refinement Chain

Validates against 13 rules:

1. No duplicated sentences
2. No repeated keywords
3. Tone natural, not “AI robotic”
4. No factual contradictions
5. Title 65–75 chars
6. Meta description 140–160 chars
7. H1 exactly 1 per page
8. H2/H3 structure respected
9. No trademark issues
10. No hallucinated opening hours
11. No invented building names
12. No invented accessibility info
13. Alt texts unique per image

Refinement output overwrites the draft.

Chain 4 Prompt Orchestration Extension: Variant Generation Chains (MVP-safe)

In addition to the existing 3-chain flow, Dynamic Experience Engine outputs use an isolated “Variant Chain” that cannot edit SEO-core fields.

Chain 4 — Variant Generation Chain (Dynamic Engine)
Inputs:

- base canonical content (read-only)

- segment list + segment intent rules
- review clusters + per-segment weighting
- page type + allowed dynamic sections
  Outputs (only):
- snippet_* variants
- review_summary_* variants
- ordering_profile_* hints
  Rules:
- must not modify: H1, meta_title, meta_description, canonical long_description, schema JSON-LD, keyword ownership fields
- must pass the same tone + prohibited phrases validator
- must pass “no hallucination” checks for any factual references

Chain 5 — Live Layer Summarization Chain (Human Face)
Inputs:

- tour_run_events (if available)
- reviews in the relevant time window (tour_date or last 7 days)
- guide_profiles (if available)
  Outputs (only):
- Last Tour Snapshot copy fields
- City/Attraction weekly live summary copy fields
  Rules:
- factual values must be templated from system fields, not generated
- AI generates only: short summaries, topic labels, captions

E.07 AI Drafting Rules by Field

Title

- max 80 characters
- must include primary keyword
- must include attraction/city
- no generic adjectives
- no year

Short Description

- 22–35 words
- high information density
- no SEO stuffing

Long Description

- 230–320 words
- broken into 3 paragraphs
- must use:
    - primary keyword
    - 1–2 secondary keywords
    - contextual synonyms

Highlights

- 5–8 points
- each <12 words
- each must start with a verb

Meta Title

- 65–75 characters
- must include primary keyword
- must contain city
- should contain CTA element: “Guided Tour”, “Tickets”, “Entry”

Meta Description

- 145–160 characters
- must include emotional prompt + USP
- must include city + attraction

E.08 Drafting Fallback Logic

Case
System response

Missing keyword cluster
Stop draft → send to “keyword allocation queue”

Missing factual data
Fill with verified-neutral template
Flag: factual_needed

Insufficient context from embeddings
Fallback to template without “similar pages”

AI generation fails
Try 3 attempts → send to “manual review queue”

E.09 CMS Integration Requirements

AI Drafting Layer must write into CMS fields:

Visible to Marketer

- AI-generated content (editable)
- Locks shown (AI/Human)
- Keyword ownership block
- FAQ block
- Schema preview
- AI log (timestamp + prompt ID)

Marketer Controls:

Toggle buttons:

- Regenerate Description
- Regenerate Meta
- Regenerate FAQ
- Rebuild Schema

Each section also shows:

- “Generated by AI”
- “Edited by human — locked”
- “Safe to regenerate”

Hidden System Fields

- ai_version
- ai_generated_timestamp
- used_prompt_template
- semantic_context_used
- factual_block_id
- qdrant_lookup_id

CMS Integration Extension: Editor Controls for Dynamic + Human Face

In addition to existing E.09 controls, CMS must expose:

A) Dynamic Variants Panel (per entity page)

- editable fields for all snippet_* variants
- editable fields for all review_summary_* variants (if applicable)
- ordering_profile_* viewer + override editor (pin/boost/avoid)
  Buttons:
- Regenerate Dynamic Snippets
- Regenerate Review Summaries
- Rebuild Ordering Profiles (suggestion only; editor override optional)
  Locks:
- per-field lock (variant-level locking)

B) Human-Face / Live Panel (per entity page)
Product:

- toggle: show_last_tour_snapshot
- preview: last tour date window used
- rebuild action: Recompute Last Tour Snapshot
- moderation: hide/show guest photos used in snapshot strip (if supported)
  Guides:
- assign primary guide(s) (Rosotravel-owned)
- fallback selection for OTA (guide team card)
  City/Attraction:
- toggle: show_live_week_block
- rebuild action: Recompute Weekly Live Summary

C) Variant Preview Switch (required)
In CMS preview mode, editor can switch between:
first_time_visitor, (it is default audience)

family_traveler,

couple_traveler,

comfort_easy_pace_traveler,

solo_social_traveler,

interest_deep_dive_traveler,

active_adventure_traveler

The preview must reflect:

- snippet variant
- review summary variant
- ordering profile result inside page modules (ordering only, no URL changes)

E.10 Validation + Status Outputs

Each draft receives a status:

Status	Meaning
`ai_draft_ready`	Successfully generated
`ai_draft_failed`	AI generation failed
`requires_factual_data`	Google Places/Wikidata missing
`requires_keyword_allocation`	No keywords detected
`manual_review_required`	AI uncertainty too high

E.11 Example Internal Payload (JSON)

Here is exactly what the AI Drafting Layer outputs (simplified example):

```
{
  "entity_type": "product",
  "url_placeholder": "/italy/rome/colosseum-guided-tour",
  "content": {
    "title": "Colosseum Guided Tour with Roman Forum Entry",
    "short_description": "Explore the Colosseum with a licensed
guide and enjoy priority access to the Roman Forum.",
    "long_description": "<p>...</p>",
    "highlights": [
      "Skip-the-line entry",
      "Expert local guide",
      "Roman Forum access",
      "Small group experience",
      "Fast-track entrance"
    ],
    "itinerary": "<p>...</p>",
    "expert_tip": "Arrive 15 minutes early to enjoy the outer arches
without crowds.",
    "local_insight": "The Colosseum once held 50,000 spectators
cheering gladiators."
  },
  "seo": {
    "h1": "Colosseum Guided Tour",
    "meta_title": "Colosseum Guided Tour in Rome | Skip-the-Line
Entry",
    "meta_description": "Book a guided Colosseum tour with priority
entry and Roman Forum access. Small groups and expert guides
included."
  },
  "faq": [
    {
      "question": "How long is the Colosseum tour?",
      "answer": "The guided tour lasts approximately 90 minutes
including the Roman Forum."
    }
  ],
  "schema": {
    "json_ld": "{ ... }"
```

```
  },
  "embeddings_ready": false
}
```

F. VALIDATION LAYER (FULL SPECIFICATION)

F.01 Purpose & Position in the Pipeline
The Validation Layer is the last automated gate before Human QA and Publishing.
It guarantees that every AI-generated draft is:

- structurally valid (all required fields present and well-formed)
- linguistically natural (no AI-ish artifacts, no spam, no boilerplate)
- SEO-safe (no keyword cannibalization, over-optimization, or missing intent)
- factually consistent (never contradicting Google Places / Wikidata factual block)
- schema-valid (JSON-LD correct, rich-result ready)
- non-duplicated (internally & externally)
- FAQ-safe (no repeated questions across product/attraction/city layers)

It runs twice per entity:

1. after the main AI drafting step (before FAQ retrieval/drafting)
2. after FAQ has been generated and attached.

F.02 Validation Stages Overview

For each draft page (product, attraction, city, region, country, things-to-do, near-page, expert page) the system runs these stages in order:

1. Structural & syntax validation

2. Content quality & language validation
3. SEO & keyword validation
4. Factual integrity validation (against factual_block)
5. Duplication & template detection (internal + external)
6. FAQ validation & collision prevention
7. Schema (JSON-LD) validation

Each stage produces machine-readable errors/warnings that are stored and exposed in CMS.

F.03 Input/Output Contracts

F.03.1 Input: DraftContent JSON (simplified but realistic)

Example payload for a tour/product page draft (after AI drafting, before FAQ stage):


```
{
  "entity_id": "tour_12345",
  "entity_type": "product",
  "locale": "en",
  "url_slug": "italy/rome/colosseum-guided-tour",
  "primary_keyword": "colosseum guided tour",
  "secondary_keywords": [
    "rome colosseum tour",
    "skip the line colosseum"
  ],
  "title": "Colosseum Guided Tour with Skip-the-Line Entry",
  "h1": "Colosseum Guided Tour in Rome",
  "meta_title": "Colosseum Guided Tour in Rome | Skip-the-Line Tickets",
  "meta_description": "Discover the Colosseum with a guided tour in Rome. Skip the line, hear expert stories and explore ancient history with a local guide.",
  "short_description": "Explore the Colosseum with a local guide, skip the long queues and uncover the stories behind Rome’s most iconic arena.",
  "long_description": "Full body text with sections, paragraphs...",
```

```
"highlights": [
  "Skip-the-line entry to the Colosseum",
  "Expert local guide in your chosen language",
  "Small group for a more personal experience"
],
"itinerary": [
  "Meet your guide near the Colosseum entrance",
  "Guided tour inside the Colosseum",
  "Time for photos and questions"
],
"faq": [],               // filled after FAQ module
"pro_tips": [
  "Arrive 15 minutes early to find your guide easily."
],
"alt_texts": [
  {
    "image_url": "/images/colosseum_1.webp",
    "alt": "View of the Colosseum in Rome at sunset"
  }
],
"schema_blocks": [
  {
    "type": "TouristTrip",
    "json_ld": { "...": "..." }
  },
  {
    "type": "FAQPage",
    "json_ld": { "...": "..." }
  }
],
"factual_block": {
  "place_id": "ChIJ2QeB5YBfLxMRxAq5LFz4Z4Q",
  "source": "google_places",
  "opening_hours": "Mo-Su 08:30-19:15",
  "geo_lat": 41.8902,
  "geo_lng": 12.4922,
  "address": "Piazza del Colosseo, 1, 00184 Roma RM, Italy",
  "official_website": "https://parcocolosseo.it"
},
"locks": {
  "title_locked": false,
```

```
    "short_description_locked": false,
    "long_description_locked": false,
    "highlights_locked": false,
    "itinerary_locked": false,
    "faq_locked": false,
    "meta_locked": false,
    "schema_locked": false
  }
}
```

F.03.2 Output: ValidationResult JSON

```
{
  "entity_id": "tour_12345",
  "entity_type": "product",
  "locale": "en",
  "status": "needs_review",           // ok | needs_review | blocked
  "severity": "warning",              // info | warning | error
  "errors": [
    {
      "code": "SEO_PRIMARY_KEYWORD_MISSING_IN_H1",
      "field": "h1",
      "message": "Primary keyword 'colosseum guided tour' not found
in H1.",
      "severity": "warning"
    },
    {
      "code": "CONTENT_TOO_SHORT_LONG_DESCRIPTION",
      "field": "long_description",
      "message": "Long description is under 200 words.",
      "severity": "warning"
    }
  ],
  "schema_errors": [],
  "faq_errors": [],
  "duplication": {
    "internal_similarity_score": 0.68,
    "external_plagiarism_score": 0.00,
    "collision_with_url": null
  },
  "factual_check": {
```

```
    "status": "ok",                    // ok | mismatch | missing
    "mismatched_fields": []
  },
  "created_at": "2025-11-23T10:15:00Z"
}
```

This object is stored, and a summary is surfaced in CMS.

F.04 Structural & Syntax Validation

Goal: ensure that each draft entity has all required fields, correct types, and basic length constraints before deeper checks.

F.04.1 JSON Schema for Product Draft (simplified)

```
{
  "$schema": "https://json-schema.org/draft-07/schema#",
  "title": "ProductDraft",
  "type": "object",
  "required": [
    "entity_id",
    "entity_type",
    "locale",
    "url_slug",
    "primary_keyword",
    "title",
    "h1",
    "meta_title",
    "meta_description",
    "short_description",
    "long_description",
    "highlights",
    "schema_blocks",
    "factual_block"
  ],
  "properties": {
    "entity_id": { "type": "string", "minLength": 1 },
    "entity_type": { "type": "string", "enum": ["product"] },
    "locale": { "type": "string", "pattern":
"^[a-z]{2}(-[A-Z]{2})?$" },
    "url_slug": { "type": "string", "minLength": 3 },
```

```
    "primary_keyword": { "type": "string", "minLength": 3 },
    "secondary_keywords": {
      "type": "array",
      "items": { "type": "string" }
    },
    "title": { "type": "string", "minLength": 10, "maxLength": 110
},
    "h1": { "type": "string", "minLength": 10, "maxLength": 120 },
    "meta_title": { "type": "string", "minLength": 20, "maxLength":
60 },
    "meta_description": { "type": "string", "minLength": 80,
"maxLength": 160 },
    "short_description": { "type": "string", "minLength": 50,
"maxLength": 300 },
    "long_description": { "type": "string", "minLength": 200 },
    "highlights": {
      "type": "array",
      "minItems": 3,
      "maxItems": 7,
      "items": { "type": "string", "minLength": 10, "maxLength": 160
}
    },
    "itinerary": {
      "type": "array",
      "items": { "type": "string", "minLength": 10, "maxLength": 200
}
    },
    "faq": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["question", "answer"],
        "properties": {
          "question": { "type": "string", "minLength": 10,
"maxLength": 160 },
          "answer": { "type": "string", "minLength": 40,
"maxLength": 600 }
        }
      }
    },
    "schema_blocks": {
```

```
      "type": "array",
      "minItems": 1
    },
    "factual_block": {
      "type": "object",
      "required": ["place_id", "geo_lat", "geo_lng"],
      "properties": {
        "place_id": { "type": "string", "minLength": 1 },
        "geo_lat": { "type": "number" },
        "geo_lng": { "type": "number" }
      }
    }
  }
}
```

If JSON schema validation fails → status = "blocked" and drafts do not proceed further.

F.05 Content Quality & Language Validation

These rules ensure content is natural, human-like, and not spammy.

F.05.1 General text rules

- All main text fields (title, h1, meta, short_description, long_description, FAQ answers) must:
    - not contain full ALL CAPS sentences
    - not have more than 3 identical sentences in a row
    - not contain banned boilerplate phrases (we define a list)

Example banned phrases array:

```
[
  "in today's fast-paced world",
  "unforgettable experience of a lifetime",
  "nestled in the heart of",
  "look no further",
  "without breaking the bank"
]
```

Regex examples:

1. Detect full uppercase sentence (for small fields, not abbreviations):

```
^[A-Z0-9 ,.'!?-]{20,}$
```

2. Detect triple repetition of the same sentence (pseudo logic, not directly regex-only):

- Split long_description into sentences.
- If any sentence appears ≥ 3 times → error: CONTENT_REPETITIVE.

3. Detect too high percentage of the same word (simple heuristic):

- For each word w with length ≥ 3, if count(w) / total_words > 0.07 → warning: WORD_OVERUSED.

F.05.2 Length rules per field

Example (product page, EN):

- title: 40–90 chars (warning if out of range)
- h1: 40–100 chars
- meta_title: 45–60 chars (for SEO)
- meta_description: 110–160 chars
- short_description: 60–240 chars
- long_description: 250–800 words (below → warning; above 1200 → warning)
- FAQ answer: 40–300 words

If long_description < 200 words → ERROR (blocked)
If long_description between 200–250 → WARNING.

F.05.3 Language consistency

- locale-specific checks (e.g. for "en"):
    - disallow mixing with too many foreign words except brand/POI names (heuristic: if >10% tokens look non-English → warning).

F.06 SEO & Keyword Validation

Goal: ensure the drafted content aligns with keyword ownership rules and funnel intent, without over-optimization.

F.06.1 Primary keyword placement rules

For each page:

- primary_keyword must appear at least once in:
    - h1 (exact or close variant, embedding similarity ≥ 0.92)
    - meta_title
    - first 150 words of long_description

Simple regex-ish presence (string contains), plus fallback to embedding similarity if variation exists.

F.06.2 Over-optimization / density

For each primary + secondary keyword:

- calculate density = occurrences / total_words

Thresholds (for EN):

- primary_keyword density:
    - <0.2% → warning: KEYWORD_TOO_WEAK
    - 2.5% → warning: KEYWORD_TOO_STRONG
- any single word (excluding stopwords) used >7% of total words → warning: WORD_OVERUSED.

F.06.3 Intent and entity type check

Cross-check with keyword_map:

- if entity_type = "product":

    - only transactional keywords allowed as primary/secondary.
- if entity_type = "attraction":
    - mainly informational; transactional wording only in CTA, not in title/H1.
- if entity_type = "blog":
    - only informational; no "book now" meta/title.

Violations → error codes like:

- SEO_INTENT_MISMATCH_PRODUCT
- SEO_INTENT_MISMATCH_ATTRACTION.

F.07 Factual Integrity Validation

Goal: AI must never override factual data sourced from Google Places/Wikidata.

F.07.1 Protected factual fields (non-AI editable)

- opening_hours
- geo_lat
- geo_lng
- address
- official_website
- phone (if present)
- categories
- holiday_opening

Rules:

1. In page schema JSON-LD, these values must match factual_block exactly (string or normalized).

2. Long_description and FAQ may *paraphrase* but not contradict.

Example checks:

- If long_description says “open daily from 10:00” but factual_block.opening_hours starts at 09:00 → factual mismatch.
- If FAQ says “closed on Mondays” but factual_block.weekday_text includes Monday as open → factual mismatch.

These mismatches create:

```
"factual_check": {
  "status": "mismatch",
  "mismatched_fields": ["opening_hours"]
}
```

Severity: error (blocked) until fixed or explicitly overridden by human (very rare).

F.08 Duplication & Template Detection

F.08.1 Internal duplication — embeddings

Using pages_vectors collection:

- compute similarity between this draft and all existing pages in same city/locale.

Thresholds:

- similarity ≥ 0.90 → error: INTERNAL_DUPLICATE_PAGE
- 0.75–0.90 + same primary_keyword → warning: INTERNAL_CONTENT_TOO_SIMILAR

F.08.2 Template repetition — N-gram (Screaming Frog or internal tool)

Process:

- Extract 3–5 word n-grams from long_description and highlights.
- Calculate overlap against a sample of existing pages of same type.

If >15% n-gram overlap with any single template → warning: TEMPLATE_OVERUSED.

F.08.3 External duplication — Copyscape (EN product pages only)

Rule:

- only run for entity_type = product and locale = en.
- Copyscape returns a plagiarism percentage; mapping:
    - 0–5% → ok
    - 5–20% → warning: EXTERNAL_SIMILARITY_MODERATE
    - 20% → error: EXTERNAL_SIMILARITY_HIGH (blocked until rewrite)

F.10 Schema Validation

F.10.1 Schema JSON structure validation

Each schema_blocks item is validated against:

- JSON syntax
- required fields by type

Example: TouristTrip minimum:

```
{
  "@context": "https://schema.org",
  "@type": "TouristTrip",
  "name": "...",
  "description": "...",
  "image": ["..."],
  "offers": {
    "@type": "Offer",
    "price": "39.00",
    "priceCurrency": "EUR",
    "availability": "https://schema.org/InStock"
  },
  "provider": {
    "@type": "Organization",
    "name": "Rosotravel"
```

```
  }
}
```

Required fields per type:

- TouristAttraction: name, description, image, geo, address
- TouristDestination: name, description, geo or address
- FAQPage: mainEntity[] with Question/Answer pairs

If missing any required field → error: SCHEMA_MISSING_REQUIRED_FIELDS.

F.10.2 External schema validation

Optional but recommended:

- send JSON-LD to Google Rich Results API (or similar internal validator) in staging environment.
- parse any errors/warnings and store in schema_errors[].

Severity mapping:

- structured data critical error → blocks publishing for that schema type only.
- minor warnings (e.g., missing optional fields) → warning, not blocking.

F.11 Gating Logic (When Page Can Proceed)

For each draft, after all checks:

- If any error with severity = error and code in BLOCKING_RULES → status = "blocked".
- If only warnings → status = "needs_review".
- If no errors & warnings below configured threshold → status = "ok".

BLOCKING_RULES examples:

- STRUCTURAL_JSON_INVALID

- CONTENT_TOO_SHORT_LONG_DESCRIPTION
- FACTUAL_MISMATCH_CRITICAL
- INTERNAL_DUPLICATE_PAGE
- EXTERNAL_SIMILARITY_HIGH
- SCHEMA_MISSING_REQUIRED_FIELDS

Only status = "ok" or "needs_review" may go to Human QA sampling; "blocked" remains in system QA until fixed or overridden.

Draft-Time Validation (Block-Level + Schema Drafting)

a. **Scope**
This layer validates *generated blocks* (AI outputs) and *schema drafts* before publishing. It does **not** validate full-page rendering completeness.

b. **HowTo / Steps Draft Validation (Product + Package)**

- Generate the HowTo block only if eligibility is met (page is bookable, required factual inputs exist).
- Validate the HowTo block structure:
    - step_count within allowed bounds (Product: 3–6, Package: 3–7)
    - each step has non-empty title/text
    - length limits enforced
- Draft HowTo schema only when `howto_enabled=true` and the HowTo UI module is expected to render.
- Output `draft_validation_status` + `schema_draft_status` + reasons.

c. **Article Field Validation (Blog/Guides) — Draft-Time**

- Validate formats only (ISO 8601 date formats, author identifiers format).
- Produce structured errors for missing fields, but do not decide index/noindex here.

d. **Status Outputs (mandatory)**

- `draft_validation_status`: OK / FAIL
- `schema_draft_status`: OK / FAIL
- `validation_errors[]`: {code, severity, field, message}

Metadata Completeness Gate (Title / Meta / H1 — Per Language)

Missing metadata is treated as a publish safety defect.

Required fields per page (per language variant):

- Page_Title (title tag)
- H1 (visible)
- Canonical_URL
- inLanguage
- Meta_Description (required if page is eligible for indexation; optional for noindex pages)

Enforcement:

- If Page_Title or H1 is missing → block publish OR force noindex + send to CMS Inbox (severity: HIGH)
- If Meta_Description missing on an indexable page → block publish OR force noindex + send to Inbox
- If localized variant missing required fields → block that language release (ties into 4.18)

F.13 Error Handling & Logging

For each validation run:

- store: entity_id, entity_type, locale, validation_result JSON, duration, timestamp.

- errors in validator itself (exceptions, timeouts) → log to `validation_failures` table with stack trace; draft remains in previous state.

Retry policies:

- transient errors (e.g. external schema API timeout) → retry up to 3 times.
- persistent errors → flag and require human investigation.

3.1 Extended Schema.org Data Structure

Purpose
Expose machine-readable entities so search engines and LLMs fully understand tours, offers, reviews, people, organization, and FAQs.

Data/Schema

- Types: TouristTrip, Offer, Review, Person, Organization, FAQPage.
- Custom attributes:
  - `reviewSource` (real / itinerary / ai-seed)
  - `geo` (coordinates of meeting point)
    `imageObject` (URL, author, date, EXIF)
  - `knowsAbout` (guide's expertise)
  - `aggregateRating` (computed average)

Backend

- Generate JSON-LD blocks per page type (product, city, attraction, organization).
- Map feed fields → schema fields; maintain versioning; validate against schema.org.
- Store custom attributes; enrich where available (EXIF, coordinates).
- Localize schema fields per locale after publication (translations).

-

APIs/Integration

- Internal generator service `GET /schema/:entityId?type=TouristTrip|Offer|…`
- Validation pipeline (schema linter) in CI.

Validation

- No missing required fields, no conflicting values; one Organization global profile.
- Test in Google Rich Results; log errors.

KPIs

- % pages with valid schema = 100%.
- Rich result eligibility by template type.

Acceptance (DoD)

- All page types output valid JSON-LD; custom attributes present where applicable.

Notes

- Keep schema minimal but complete; avoid duplication across multiple blocks.

A.10.6 Schema Ownership Mapping (schema_map)

The schema_map defines which entity owns which structured data blocks.

Rules:

- Each schema block has a single owner entity.
- Schema MUST reflect the Published Record winners only.
- Product schema may not inherit schema blocks from City or Attraction.
- Aggregate schemas (e.g. city-level Offer summaries) must not expose supplier-only data.

Schema ownership prevents:

- duplicate structured data
- conflicting rich results
- invalid nesting across hierarchy levels

4.16 FAQ Pipeline (Questions Retrieval → Dedupe → AI Answers → QA Signals)

Tu ma być: SEMrush Questions + GSC queries, dedupe, global uniqueness, generowanie odpowiedzi, QA signals.

01. FAQ Structured Data Consistency Gate

FAQPage schema must reflect only FAQs that are rendered and visible on the page.

Hard rules:

1. If the page does not render an FAQ block, **FAQPage schema must not be emitted**.
2. If an FAQ block is rendered, schema questions/answers must match the UI content (no hidden questions).
3. FAQ generation must follow the FAQ pipeline constraints (dedupe, uniqueness, QA signals).
4. FAQ schema errors (invalid structure, empty values) must **block publishing** (or force noindex + alert).

DoD:

- Zero cases where FAQPage schema exists without an FAQ UI block.
- FAQ schema is always valid and consistent with page content.

02. AEO Schema Blocks (FAQ + HowTo) — Conditional, UI-Bound

a. FAQPage

FAQPage schema is allowed only if an FAQ block is rendered on the page.

b. HowTo (optional — only when it fits)

HowTo schema is allowed only when:

- the page renders a visible **Steps** module (minimum 3 steps),
- each step is user-actionable (not marketing copy),
- steps are stable and reviewed (editable + lockable in CMS).

c. Typical eligible pages:

- Travel Guide – Itinerary / Topic (e.g., “3 days in Rome itinerary”)
- Ticket usage / redemption guides (only if written as steps)
- “How to meet your guide / how boarding works” sections (only if truly step-based)

d. Validation gates (blocking):

- If HowTo schema is emitted but < 3 steps → block publishing
- If HowTo steps exist in UI but schema missing → warn (non-blocking) and enqueue fix

3.4 FAQ Sections

Purpose

Expose question-answer content that serves both users and search/LLMs.

Data/Schema

- **FAQPage JSON-LD attached to: Product, Attraction, City pages.**
- **Natural-language questions (e.g., “Where does the tour start?”).**

Backend

- **AI generation from factual fields (duration, tickets, meeting point, refund); min 3 Q&A per level.**

- **Store per level (city/attraction/product); support modular re-gen.**

Front-End

- **Collapsible accordions; per-level display; schema injection.**

APIs/Integration

- **`GET /faq/:scope(city|attraction|product)/:id` with locales.**

Validation

- **No contradictions with base data; language quality threshold.**

KPIs

- **Rich results coverage for FAQ; CTR uplift where FAQ shown.**

Acceptance (DoD)

- **All three levels available where applicable; JSON-LD valid.**
-

A.10.5 FAQ Ownership Mapping (faq_map)

The faq_map table ensures **global FAQ uniqueness** and prevents duplication across entities.

Each FAQ record MUST include:

- question

- normalized_question

- embedding

- assigned_entity_id
- assigned_entity_type
- intent_source (SEMrush / GSC / AI)
- locked
- created_at

Rules:

- A question may appear on **only one indexable page**.
- FAQ similarity checks MUST run before assignment.
- FAQs derived from supplier content must be rewritten by AI to be owned.

Near-pages may not reuse FAQs owned by Products or Attractions.

F.09 FAQ Validation & Collision Prevention

F.09.1 FAQ count rules per page type

Guidelines per entity_type (EN):

- product: 5–7 FAQs
- attraction: 7–10 FAQs
- city: 8–12 FAQs
- country: 8–15 FAQs
- near-page: 5–8 FAQs
- blog/travel guide: 3–7 FAQs

If less than minimum → warning; if >20 FAQs → warning (too heavy).

F.09.2 FAQ semantic uniqueness

Before accepting AI-generated FAQ list:

- embed each question into faq_vectors

- for each question, search top-k (e.g. 10) similar questions in same locale.

Rules:

- if similarity ≥ 0.92 with any FAQ on:
    - parent attraction
    - parent city
    - parent things-to-do page
      → collision → either:
        - auto-adjust wording, or
        - skip and generate alternative question.

FAQ with similarity ≥ 0.92 and same answer logic must not be duplicated across levels.

F.09.3 FAQ type coverage

For product pages, at least:

- 1 booking/cancellation question
- 1 “what’s included” / “what to expect”
- 1 logistics (meeting point, duration)
- 1 language/guide question

Validation ensures content types are covered (simple intent classifier on the question).

4.17 Publish Outputs: Assemble Published Record + Website Sitemaps + Public Rendering Rules

Tu ma być: deterministic winners, index_state, standard sitemaps (web), public rendering rules.

AI & Published Tracking (per RAW version N)

H.1 AI State (ai_state_for_raw_version = N)

AI state indicates whether AI outputs exist for the current RAW vN.

Stored fields (per entity, per RAW vN):

- `ai_processed_for_raw_version = N | null`
- `ai_status = NOT_REQUIRED | REQUIRED | GENERATED | FAILED`
- `ai_generated_at`
- `ai_prompt_template_id`
- `ai_outputs` (stored as draft/CMS fields)
- `ai_regeneration_log[]` (audit only; no new version numbers)

Key rule: AI can regenerate multiple times within the same RAW vN; version number does not change.

H.2 Published Record (published_for_raw_version = N)

Published Record is the **only** object served publicly (front-end, APIs, AI feeds, sitemaps, MCP).

Stored fields:

- `published_for_raw_version = N | null`
- `published_at`
- `published_by`
- `index_state`
- `published_record` (materialized deterministic winners)

H.2.1 Mismatch logic (must be visible in UI)

- If `published_for_raw_version < raw_version` → “Out-of-date vs source snapshot” alert
- If `ai_processed_for_raw_version < raw_version` AND AI is required → “AI backlog” alert

I) Manual Edits & Locks (do not change version number)

Manual work must be enforceable without inflating version numbers.

Manual lock flags (per entity):

- `manual_locked.title`
- `manual_locked.short_description`
- `manual_locked.long_description`
- `manual_locked.highlights`
- `manual_locked.itinerary_overview`
- `manual_locked.faq`
- `manual_locked.meta`
- `manual_locked.media_override`
- `manual_locked.factual_override`

Each lock stores:

- `locked_by`
- `locked_at`
- `lock_scope`
- `lock_reason`

- `requires_approval` (role-configurable)

Hard rule: Manual edits do not create new RAW versions. They only affect Published Record winner selection and audit.

J) Published Record Assembly (deterministic winner rules)

Winner precedence per field group:

1. Manual-locked wins
2. Else if AI output exists for current RAW vN → publish AI
3. Else publish RAW-normalized value (only if license/index rules allow)

Clarification (critical):
"AI wins over RAW" is a publishing precedent rule, not a rule to regenerate everything.

Error Handling & Fallback for Publish outputs:

1) Definitions (Critical for correct handling)

1.1 RAW vs Published Assembly vs Cache

- RAW vN: immutable snapshot of source payloads and normalized RAW fields for that version.
- Published Record: the only object served publicly (HTML, public APIs, AI feeds, sitemaps).
- last_valid_cache: "previous valid value used only for Published assembly", never written back into RAW.

3.3 Fallback order (Published assembly only)

For each missing content seed field:

1. Use last_valid_cache value for the same field, if available and valid.
2. If allowed and required for indexing rules: AI may generate an owned replacement block (see 3.5).

3. If neither exists: leave empty and raise an error flag.

3.4 Indexation impact (PRODUCT content blocks)

Blocking vs non-blocking:

- Blocking (forces noindex, exclude from sitemaps):
    - Missing title_seed AND no last_valid_cache AND no valid AI title available
- Non-blocking (allowed to stay indexable if AI-owned blocks exist where required):

Missing long_description_seed, highlights_seed, itinerary_overview_seed (because AI can generate owned blocks)

4.2 Fallback order (Published assembly only)

For each factual field:

1. Use last_valid_cache factual value if available and still valid.
2. Re-fetch authoritative factual source according to the factual priority rules (Google Places / Wikidata) within budget and rate limits.
3. If still missing: leave empty + flag factual_missing_fields[].

4.3 Indexation impact (CITY/ATTRACTION)

Blocking:

- coordinates missing (map + geo relations break) → forced noindex, exclude from sitemaps
  Non-blocking (warning):
- opening hours missing (often unavailable) → no index change

phone / website missing → no index change

5) Missing OFFER for Publish - PRODUCT offer domain

5.2 Fallback order (Published assembly only)

1. Use last_valid_cache offer fields if available and not expired by policy.
2. Retry offer ingestion (respect API budgets and backoff).
3. If still missing: block indexing.

5.3 Indexation impact (PRODUCT offer)

Blocking (forced noindex, exclude from sitemaps):

- duration missing
- meeting point missing

cancellation policy missing
Reason: these are core offer contract fields.

6) Missing REALTIME Offer Fields (Availability/Pricing) — PRODUCT realtime domain - Publish outputs

6.3 Fallback order

1. Use last_valid_cache realtime values within a short TTL.
2. Retry realtime fetch with exponential backoff.
3. If still missing beyond TTL:
    - keep page online
    - block indexing if booking cannot be performed safely

6.4 Indexation impact

Blocking (forced noindex, exclude from sitemaps):

- price_from missing AND availability_refs missing (cannot transact)
  Warning only:
- start_times missing (not always explicit)

7) Missing or Broken RELATIONSHIP Fields (Hierarchy / Mapping) — ALL entity types - Publish

7. 2 Fall back If still unresolved:

    - assemble Published Record only with safe minimal fields

- force noindex + exclude from sitemaps
- block promotion and AI feed exposure

7.3 Indexation impact

Blocking (always):

- RELATIONSHIP_MISSING → forced noindex, exclude from sitemaps
  Reason: canonical ownership, internal linking, and sitemap eligibility depend on hierarchy integrity.

8.3 SEO rules (binding) - Publish outputs

- Supplier media must not be included in image sitemaps.

9) AI Generation Failures (Any entity where AI blocks exist) - Publish

1. If AI output is required for indexation for that page type/block:
   - force noindex, exclude from sitemaps until AI output exists or block is manually authored/locked.
2. Otherwise:
   - no index change; keep page stable.

10) API Rate Limits / Outages (System-level)

- No automatic index changes unless required fields become missing beyond allowed TTL windows.
- Existing indexable pages remain indexable as long as required fields remain valid via cache.

11) Deterministic Summary Table (Single Source of Truth) for 4.14 and 4.17

Error Type	Domain	AI Allowed	Primary Fallback	Indexation Result	UI Severity

RAW_CONTENT_MISSING (non-title)	raw_version/content seeds	Yes (limited)	last_valid_cache → AI	Indexable only if required AI blocks exist	Warning/Blocking (depends)
RAW_CONTENT_MISSING (title)	raw_version/content seeds	Yes (if title AI allowed)	last_valid_cache → AI	Forced noindex if no safe title	Blocking
FACTUAL_MISSING (coords)	factual	No	cache → refetch	Forced noindex	Blocking
FACTUAL_MISSING (other)	factual	No	cache → refetch	No change	Warning
OFFER_MISSING	offer_version	No	cache → retry	Forced noindex	Blocking
REALTIME_OFFER_MISSING (price+availability)	realtime_offer_version	No	cache TTL → retry	Forced noindex if cannot transact	Blocking
REALTIME_OFFER_MISSING (start_times only)	realtime_offer_version	No	cache → retry	No change	Warning
RELATIONSHIP_MISSING	relationships	No	cache → re-resolve	Forced noindex	Blocking

MEDIA_REVIEW_REQUIRED	media	No	cache → placeholder	No change unless policy blocks	Warning/Blocking
AI_GENERATION_FAILED	AI layer	Retry only	last valid AI	Blocking only if required AI-owned blocks missing	Warning/Blocking
API_DEGRADED	any	No	queue + cache	No change unless required fields expire	Warning

---

13) Final Enforcement Notes (Developer checklist) for 4.14 and 4.17

1. RAW snapshots remain immutable; any fallback uses last_valid_cache for Published assembly only.
2. Domain separation is mandatory:
    - raw_version ≠ offer_version ≠ realtime_offer_version
3. PRODUCT required for indexability:
    - duration + meeting point + cancellation policy + (price_from or availability_refs) must be valid.
4. RELATIONSHIPS are contract-critical:
    - missing parent chain forces noindex and blocks promotion.

5. Notifications do not clear flags:
    - flags persist until resolved and validated.

14) Examples: System Response to Updates (Updated to final versioning) for 4.14 and 4.17

These examples illustrate exact behavior driven by hash/diff + ownership rules.

Example 1 — Price change only (Realtime domain)

Change detected:

- price_from changed (availability/pricing signal)

Diff:

- realtime_offer_hash changed
- realtime_offer_version increments
- raw_version unchanged
- offer_version unchanged

System actions:

- Update booking UI pricing
- Update Offer/availability runtime blocks
- No AI regeneration
- No meta regeneration
- No keyword governance changes
- No sitemap lastmod churn (policy: realtime changes do not touch sitemap)

Indexing impact:

- Page remains indexable if booking contract fields remain valid.

Example 2 — Long description seed changed (RAW content domain)

Change detected:

- long_description_seed changed in RAW snapshot

Diff:

- content_hash changed → raw_version increments
- field not locked

System actions:

- Regenerate long_description (AI-owned), only for that block
- Refresh embeddings for that entity
- Preserve all locked blocks
- Do not touch keywords
- Meta regeneration only if configured as dependent on content change (optional policy)

Indexing impact:

- Page remains indexable.

Example 3 — Manual edit exists, supplier update arrives (Lock blocks overwrite)

Change detected:

- supplier seed changed
- corresponding block is Human-owned (locked)

Diff:

- content_hash may change (raw_version increments), but publishing winners are unchanged for locked blocks

System actions:

- Keep human-written block

- Raise change signal: SUPPLIER_UPDATE_IGNORED_DUE_TO_LOCK
- No AI regeneration for locked block

Indexing impact:

- No change.

Example 4 — Opening hours updated (CITY/ATTRACTION factual domain)

Change detected:

- factual snapshot updated (Google Places)

Diff:

- factual_hash changed → raw_version increments (raw domain includes factual for CITY/ATTRACTION)
- content_hash unchanged

System actions:

- Update factual UI blocks
- Update schema fields related to opening hours
- Update factual-derived FAQ answers if such module exists
- No AI narrative regeneration
- No keyword changes

Indexing impact:

- Page remains indexable (unless missing required factuals like coordinates).

Publishing & CMS Output Layer

G.01 Purpose of This Layer

The Publishing Layer is responsible for turning validated content objects (products, attractions, cities, regions, countries, near-pages, categories, experts) into live, crawlable and indexable pages on rosotravel.com.

Its goals are:

- Enforce a clean, stable URL and slug strategy across all entities.
- Guarantee correct canonical, hreflang and meta robots settings on every page.
- Inject the correct schema blocks per page type based on the Schema Mapping table.
- Update XML sitemaps, AI sitemaps and AI summary feeds on every publish.
- Keep the CMS simple for marketers (clear “Draft → Published → Noindex” controls).
- Be fully idempotent: the same publish operation must always produce the same output given the same input.

G.02 Entities & Publishing States

G.02.1 Entities that can be published
The Publishing Layer must support at least:

- Product / Tour pages
- Attraction (POI) pages
- City destination pages
- Region destination pages
- Country destination pages
- City “Things to do” pages
- Experience category pages (e.g. family-friendly, day trips)
- Package / Bundle pages
- Expert hub pages and expert profiles
- Near-pages (immediate + dynamic)
- Blog / Travel guide articles
- Explore map and AI dataset endpoints (API, not classic HTML)
- Support pages (About, B2B, Contact, Help Center, etc.)

G.02.2 Page lifecycle states (per locale)
Every entity in the CMS must have at least these states per language:

- draft – not visible to public, not in sitemaps.
- scheduled – has a publish_at timestamp; goes live automatically when time is reached.
- published – live, indexable (unless meta_robots overrides).
- unpublished – previously live, now hidden (HTTP 404 or 410, depending on strategy).
- noindex – live but explicitly marked meta robots noindex (e.g. thin pages, temporary experiments).

State transitions must be logged in the audit trail and visible in the CMS for each entity.

G.03 URL & Slug Generation Rules

G.03.1 General rules

- Slugs are generated once at first publish and are not auto-changed when titles change.
- Slugs are lowercase, URL-safe, hyphen-separated, ASCII-normalized (ę → e, ü → u).
- If a slug collision happens, system appends a numeric suffix (“-2”, “-3”).
- Locale is handled by language prefix (e.g. `/en/`, `/de/`) or by subdomain, depending on final decision. Content spec must support both.

G.03.3 Transitional rules vs current CMS
Given current CMS behavior:

- Country & city pages are currently created manually.
- Attraction pages are auto-created from attraction field on product, but SEO fields are manual.

Publishing Layer must:

- Preserve existing manual slugs for countries/cities where present.
- For attractions created historically, backfill slugs to match the new pattern but keep redirections from old URLs.
- Introduce slug fields per entity in CMS (country, city, attraction, product) with “Generate from name” + manual override.

-

G.05 Injected SEO & Schema Artifacts

G.05.1 HTML `<head>` injection
On every published page, Publishing Layer must inject:

- Meta title (from AI drafting + keyword locking).
- Meta description.
- Open Graph tags (og:title, og:description, og:image, og:type, og:url).
- Twitter cards (summary_large_image).
- Canonical link tag.
- hreflang alternates (if multilingual).

G.05.2 JSON-LD schema injection
Using the Schema Mapping by Page Type, Publishing Layer must:

- Build appropriate JSON-LD blocks per page type (TouristDestination, TouristAttraction, TouristTrip, Offer, FAQPage, Person, Organization, Article, ItemList, etc.).
- Include relationships: `isPartOf`, `hasPart`, `nearby`, `tourBookingPage`, `sameAs`, `hasOfferCatalog`, `review`, etc.
- Use the Global Token Library to fill fields: names, slugs, geo, prices, opening hours, FAQs, ratings, etc.
- Ensure one mainEntityOfPage per page to avoid ambiguity.

G.05.3 FAQ injection

- FAQ markup (FAQPage) must be included where FAQ content exists and passes validation.
- FAQ questions must be unique per entity type (no collision with attraction FAQ if already used on product) according to the FAQ vector layer.

G.06 Sitemaps, AI Sitemaps & AI Feeds

G.06.1 Standard sitemaps
On publish/unpublish events, the system must update:

- `sitemap.xml` – index pointing to multiple sitemaps:
  - `sitemap-products.xml`
  - `sitemap-attractions.xml`
  - `sitemap-destinations.xml` (countries/regions/cities)
  - `sitemap-near-pages.xml`
  - `sitemap-blog.xml`
  - `sitemap-experts.xml`

Each entry must contain:

- `<loc>` – canonical URL
- `<lastmod>` – from `last_modified` field
- Optional `<changefreq>` (e.g. weekly for dynamic pages, monthly for static)
- Optional `<priority>` (e.g. higher for top cities/products)

G.09 Error Handling & Idempotency

G.09.1 Publish blocking rules
Publishing must be blocked if:

- No canonical URL could be generated.
- Required schema fields are missing (e.g. price, currency, duration on product).
- Keyword ownership collision is detected (another page already owns that primary keyword).
- Vector layer reports page similarity ≥ 0.90 to an existing published URL without explicit override.

If validation fails for Title/Meta/H1:

- Do not publish as indexable content
- Default to: noindex,follow + canonical to owner/parent (as per governance)
- Create Inbox alert with fix actions (III/3.7)

In such cases, CMS must show descriptive error messages and not push the page live.

G.09.2 Idempotent publishing

- Re-publishing the same version of a page must not create multiple sitemap entries or inconsistent schema.
- If the content (hash) has not changed, Publishing Layer should update `lastmod` only if there is a structural change (e.g. status, hreflang set).

G.10 Minimal Example Outputs

G.10.1 Sitemap entry example

```
<url>

<loc>https://www.rosotravel.com/destinations/italy/lazio/rome/colosseum-guided-tour/</loc>
  <lastmod>2025-01-15</lastmod>
  <changefreq>weekly</changefreq>
  <priority>0.8</priority>
</url>
```

03. HowTo Emission Rules (Deterministic)

Publishing assembles schema as follows:

- Product page: `TouristTrip` + `Offer` (always when bookable) + `HowTo` *(only when howto_enabled=true)*

- Package page: `ItemList` + `Offer` (purchasable) + `HowTo` *(only when howto_enabled=true)*

No hidden schema: do not emit HowTo if the module is not visible.

04. Publishing Hard Gates (UI ↔ Schema Parity + Page Completeness)

a. **No Hidden Schema Rule (mandatory)**
Any schema type emitted must correspond to a **visible UI module**. If the module is not rendered, schema emission is forbidden.

b. **HowTo Publish Gate (Product + Package)**
Block publishing if any of the following is true:

- HowTo schema emitted but HowTo module is not rendered.

- HowTo module rendered but invalid step structure (out of bounds, empty fields).

- HowTo instructions contradict booking reality (e.g., “book now” but no CTA/Offer present).

- Offer/price/availability required for booking pages is missing or invalid when the page is marked indexable.

c. **Article Completeness Gate (Blog/Guides)**
Block publishing (or force `noindex,follow` + Inbox item) if missing:

- Title, Meta Description (if indexable), H1

- Author name + author profile URL (must be clickable in UI)

- datePublished and dateModified (ISO 8601 with timezone, en-US)

- inLanguage and canonical URL

d. **TOC Warning (non-blocking)**
If TOC trigger is met (length/headings threshold) but TOC fails to render → create Inbox warning with fix instructions.

05. Relations Assembly Rules (Deterministic, UI-Consistent)

Publishing assembles JSON-LD relations deterministically from verified winners + visible UI modules.

Hard rule (No Hidden Schema):
If a relation is not visible (breadcrumbs, lists, nearby, included tours), it must not be emitted.

Minimum publish set by page type:

- Always emit **canonical URL**, **inLanguage**, and **BreadcrumbList** (if breadcrumbs exist in UI).
- Emit **ItemList** only if the page renders a list module whose items are stable and auditable.
- Emit **offers** only when the UI shows purchasable price/currency/availability.
- Emit **sameAs** only from trusted entity sources (Wikidata/official site), never invented by AI.

4.18 Translation & Multilanguage Release (From EN Winners, Locks, hreflang)

Tu ma być: EN jako source, translation jobs, locks per language, hreflang.

I.01 Purpose of This Layer (Full Definition)

This layer ensures your system can:

1. **Translate all programmatically generated pages** (product, attraction, city, region, country, things-to-do, near-pages, guides).
2. **Preserve SEO integrity** across languages (canonical alignment, hreflang, keyword lock inheritance).
3. **Maintain factual accuracy** (opening hours, addresses, place IDs must NOT be translated).